UNIVERSITE DE FIANARANTSOA

ECOLE NATIONALE D'INFORMATIQUE

RAPPORT DE PROJET DE FIN D'ANNEE

Mention : Informatique

Parcours : Informatique Générale

Intitulé :

Chiffrement des communications Client-serveur

« Serveur et client TCP sécurisés par RSA, AES-GCM et

Encadreur

Monsieur RANDRIANOMENJANAHARY Lala Ferdinand, Maitre de conférences

Présenter par :

- 458H-TOL : RABENARISON Bien Christophe
- 455H-TOL : ODONARDO DESIRE Andriamaidana Avenir Retsala
- 478H-TOL : RAVELOMANANTSOA Nantenaina Sitraka
- 420H-TOL : ANDRIANASOLO Andriamanendrika Safidy

Année Universitaire 2025 – 2026

CURRICULUM VITAE



ETAT CIVIL

Nom : RABENARSON

Prénoms : Bien Christophe

Date et lieu de naissance : 29 avril 1999 à Ambolofoty

Situation familiale : Célibataire

Contact : 034 78 785 51

Email : bienchristopherabenarson@gmail.com

FORMATIONS ET DIPLÔME OBTENU

2024-2025 : Troisième année en Licence Professionnelle, Parcours : Informatique Générale, à l'Ecole Nationale d'Informatique, Université de Fianarantsoa

2023-2024 : Deuxième année en Licence Professionnelle, Parcours : Informatique Générale, à l'Ecole Nationale d'Informatique, Université de Fianarantsoa

2022-2023 : Première année en Licence Professionnelle, Parcours : Informatique Générale, à l'Ecole Nationale d'Informatique, Université de Fianarantsoa

2021-2022 : Première année en Licence Professionnelle, Parcours Physiques et Chimie, à la Faculté des Sciences, Université de Toliara .

2020-2021 : Obtention du diplôme de Baccalauréat SERIE D au Lycée Laurent Botokeky de Toliara

STAGES ET EXPERIENCES PROFESSIONNELS

01 Septembre 2025 – 22 Novembre 2025 : Stage au sein de l'Université de Toliara en service informatique

Thème : « Mise en place d'un système de détection d'intrusion réseau »

Moyens Technologies : VirtualBox, Kali Linux, Ubuntu Server, Snort

19 Août 2024 – 22 Novembre 2024 : Stage en pratique au sein de JI.RA.MA

Thème : « Active directory et contrôle de domaine avec Windows serveur 2022 »

Moyens Logiciels: VirtualBox, Windows serveur 2022, windows 10, windows 8

Juin 2025 : **Projet Scolaire au sein de l'Ecole Nationale de l'Informatique ENI**

Thème : Développement d'un application mobile « Mini-réseaux Sociaux »

Moyens Logiciels : React Native, Supabase, expo Go

COMPETENCES EN INFORMATIQUE

Technologies web: HTML5, CSS3, Tailwind CSS, React

Framework web: Express

Framework mobile: React Native

Framework desktop: QT

Langages de programmation : Javascript, Python, PHP, Java

Systèmes d'exploitation : Windows, Linux

Outils bureautiques : Microsoft Word, Excel, Powerpoint

Réseaux : LAN, MAN, WAN, Protocole TCP/IP

CONNAISSANCES LINGUISTIQUES

Langues	Comprendre à l'audition	Lire	Ecrire	Parler et communiquer
Français	B	B	B	B
Anglais	AB	AB	AB	AB

Notes : B : Bien AB : Assez Bien P : Passable

CURRICULUM VITAE



ETAT CIVIL

Nom : ODONARDO DESIRE

Prénoms : Andriamaidana Avenir Retsala

Date et lieu de naissance : 23 Aout 2004 à Toliara

Situation familiale : Célibataire

Contact : +261 34 91 066 07

Email : odonardoavenir@gmail.com

FORMATIONS ET DIPLÔME OBTENU

2024-2025 : Troisième année en formation Licence à Ecole Nationale d'Informatique parcours Informatique Générale

2023-2024 : Deuxième année en formation Licence à Ecole Nationale d'Informatique parcours Informatique Générale

2022-2023 : Première année en formation Licence à Ecole Nationale d'Informatique parcours Informatique Générale

2021-2022 : Lycée Privée Sacrée Coeur Mahavatse II BACCALAURÉAT Série D

STAGES ET EXPERIENCES PROFESSIONNELS

Septembre - Novembre 2025 : Stage de trois mois effectués au sein de l'entreprise DREN Atsimo Andrefana.

Thème : Gestion des ressources informatiques

Outils: Backend: Framework Django, Frontend: Framework React Js, SGBD: PostgreSQL

2023 – 2024 : Projet au sein de l'Ecole Nationale d'Informatique

Septembre - Novembre 2024 : Stageau sein de l'UCP Ambohimiandra Antananarivo.

Thème : Renouvellement et Mise en place de Site Web

Outlies: Backend: Framework Django, Frontend: Framework React Js, **SGBD**: PostgreSQL.

2022 – 2023 : Projet au sein de l'Ecole Nationale d'Informatique.

COMPETENCES EN INFORMATIQUE

Langages de programmations : C, PHP, Python, Java.
Framework: Bootstrap, Django Rest Framework, React JS.
Technologies web :HTML, CSS, Java Script.
Systèmes d'exploitation : Windows, Linux.
Maintenance : Installation et dépannage de système informatiques.
SGBD : MySQL, PostgreSQL.
Moyens de conception : MERISE, UML.

CONNAISSANCES LINGUISTIQUES

Langue	Ecrire	Parler	Comprendre	Certificat
Français	B	B	TB	DALF B1
Anglais	AB	AB	AB	

Notes : TB : Très Bien B : Bien AB : Assez Bien P : Passable

CURRICULUM VITAE



ETAT CIVIL

Nom : RAVELOMANANTSOA

Prénoms : Nantenaina Sitraka

Date et lieu de naissance : 11juin 2004 à Toliara

Situation familiale : Célibataire

Contact : 033 97 453 24

Email : 11ravelomanantsoa@gmail.com

FORMATIONS ET DIPLÔME OBTENU

2024-2025 : Troisième année en licence Professionnelle à l'Ecole Nationale de l'Informatique Fianarantsoa centre de Formation Tuléar. Parcours : Informatique Générale.

2023-2024 : Deuxième année en licence Professionnelle à l'Ecole Nationale de l'Informatique Fianarantsoa centre de Formation Tuléar. Parcours : Informatique Générale.

2022-2023 : Première année en licence Professionnelle à l'Ecole Nationale de l'Informatique Fianarantsoa centre de Formation Tuléar. Parcours : Informatique Générale.

2021-2022 : BACC GENERALE Série D à Toliara

STAGES ET EXPERIENCES PROFESSIONNELS

1 Septembre 2025 – 22 Novembre 2025 : Stage en pratique au sien de Présidence Toliara

Thème : Conception et réalisation d'une application web pour la gestion des projets en entreprise

Technologies : Langage de programmation Javascript, Framework React, Express, SGBD MySQL, environnement Node.js

Mai 2025 – Juin 2025 : Projet scolaire au sein de l'École Nationale d'Informatique

Thème : Gestion d'application web des congés et permissions en entreprise

Technologies : Langage de programmation Javascript, Framework React, Express, SGBD MySQL, environnement Node.js

19 Août 2024 – 22 Novembre 2024 : Stage en pratique au sien de COSTM Toliara

Thème : Conception et réalisation d'une application web pour un serveur mail local

Technologies : Langage de programmation Javascript, Framework React, Express, SGBD MySQL, environnement Node.js

Avril 2024 – Mai 2024 : Projet scolaire au sein de l'École Nationale d'Informatique

Thème : Gestion de l'application desktop des réservations de chambre d'un hôtel

Technologies : MySQL, Java, NetBeans IDE.

COMPETENCES EN INFORMATIQUE

Technologies web: HTML, CSS, Tailwind CSS, Bootstrap5, React

Langage de programmation : Javascript, Java, PHP, Python

Framework web: Express

Framework mobile: React Native

SGBD : MySQL, SQLite, PostgreSQL

Systèmes d'exploitation : Windows, Linux

Bureautique : Microsoft Word, Excel, Powerpoint

Réseaux : LAN, WAN, Protocole TCP/IP

CONNAISSANCES LINGUISTIQUES

Aptitudes / Langues	Comprendre				Lire				Ecrire				Parler			
Mention	TB	B	AB	P	TB	B	AB	P	TB	B	AB	P	TB	B	AB	P
Français	X					X					X				X	
Anglais		X					X					X			X	

Notes : TB : Très Bien B : Bien AB : Assez Bien P : Passable

CURRICULUM VITAE



ETAT CIVIL

Nom : ANDRIANASOLO

Prénoms : Andriamanendrika Safidy

Date et lieu de naissance : 17 Juin 2002 à Sakaraha

Situation familiale : Célibataire

Contact : 033 66 629 25 / 038 49 756 06

Email : safidyandrianasolo118@gmail.com

FORMATIONS ET DIPLÔME OBTENU

2024-2025 : Troisième année en licence Professionnelle à l'Ecole Nationale de l'Informatique Fianarantsoa centre de Formation Tuléar. Parcours : Informatique Générale.

Décembre 2024 : Obtention d'attestation au SELF (Speak English Language Fluently)

2023-2024 : Deuxième année en licence Professionnelle à l'Ecole Nationale de l'Informatique Fianarantsoa centre de Formation Tuléar. Parcours : Informatique Générale.

2022-2023 : Première année en licence Professionnelle à l'Ecole Nationale de l'Informatique Fianarantsoa centre de Formation Tuléar. Parcours : Informatique Générale.

2020-2021 : BACC TECHNIQUE Série Industriel au Lycée Technique Professionnelle / Toliara

STAGES ET EXPÉRIENCES PROFESSIONNELLES

01 Septembre 2025 – 22 Novembre 2025 : Stage au sein de l'Assurances ARO

Thème : Système de Gestion de Suivi des Sinistres

Moyens Technologies : Vite + react, Express.js, Node.js, MySQL, Gantt Projet, Visual studio code, GitHub, Microsoft Office.

19 Août 2024 – 22 Novembre 2024 : Stage en pratique au sein de C.O.S.T.M

Thème : « Serveur VPN de Type Client-to-Site »

Moyens Logiciels : Vite + reactjs, PgAdmin, VirtualBox, PfSense, Windows serveur, Serveur des fichiers, Serveur OpenVPN

Juin 2025 : **Projet Scolaire au sein de l'Ecole Nationale de l'Informatique ENI**

Thème : Développement d'un application mobile « Mini-réseaux Sociaux »

Moyens Logiciels : React Native, Supabase, expo Go

COMPETENCES EN INFORMATIQUE

Technologies web: HTML5, CSS3, Tailwind CSS, React

Framework web: Express

Framework mobile: React Native

Framework desktop: QT

Langage de programmation : Javascript, Python, PHP, Java

Systèmes d'exploitation : Windows, Linux

Bureautique : Microsoft Word, Excel, Powerpoint

Réseaux : LAN, MAN, WAN, Protocole TCP/IP

CONNAISSANCES LINGUISTIQUES

Tableau 1 : Compétence linguistique

Langues	Comprendre à l'audition					Lire					Ecrire					Parler et communiquer				
	TB	B	AB	P	NS	TB	B	AB	P	NS	TB	B	AB	P	NS	TB	B	AB	P	NS
Français	X						X				X						X			
Anglais	X					X					X					X				
Allemand				X						X					X				X	

TB : Très Bien

B : Bien

AB : Assez-Bien

P : Passable

NS : Niveau Scolaire

SOMMAIRE

CURRICULUM VITAE 2
CURRICULUM VITAE 4
CURRICULUM VITAE I
CURRICULUM VITAE III
SOMMAIRE V
REMERCIEMENTS VIII
LISTE DES FIGURES IX
LISTE DES TABLEAUX XII
LISTE DES ABREVIATIONS XIV
INTRODUCTION GENERALE 1
PARTIE I : PRESENTATIONS 2
Chapitre 1. Présentation de l'Ecole Nationale d'Informatique 3
1.1. Information d'ordre générale 3
1.2. Missions et historiques 3
1.3. Organigramme institutionnel 5
1.4. Domaine de spécialisation 6
1.5. Architecture des formations pédagogiques 7
1.6. Relation de l'ENI avec les organismes externes 8
1.7. Débouchés professionnels et diplômés 9
1.8. Ressources humaines 11
Chapitre 2. Description du projet 12
3.1. Formulation 12
3.2. Objectif et besoin de l'utilisateur 12
3.3. Moyens nécessaires à la réalisation du projet 13
3.4. Résultats attendus 15

3.5. Chronogramme de travail 15
PARTIE II : ANALYSE PREALABLE 17
Chapitre 4. Analyse de l'existant 18
4.1. Analyse de l'existant 18
4.2. Critique de l'existant 21
4.3. Proposition de solution 24
Chapitre 5. Études et choix des outils 25
5.1. Étude des notions de sécurité et de cryptographie 25
5.2. Comparaison et choix des outils 30
5.3. Présentation des outils retenus 32
PARTIE III : REALISATION 34
Chapitre 6. Présentation de l'environnement 35
6.1. Environnement matériel 35
6.2. Environnement réseau 35
6.3. Environnement logiciel 36
6.4. Architecture de la solution 36
Chapitre 7. Installation et configuration 39
7.1. Installation de Python 39
7.2. Installation et utilisation de Visual Studio Code 41
7.3. Installation de la bibliothèque cryptography et de Tkinter 43
7.4. Installation et utilisation de Wireshark 44
7.5. Outils réseau du système 46
7.6. Conception du protocole de communication 48
7.7. Développement du code 53
7.8. Interfaces graphiques 66
7.9. Déploiement sur deux machines 74
Chapitre 8. Test et validation 81

8.1. Tests 81
8.2. Validation 89
CONCLUSION GENERALE 93
REFERENCES BIBLIOGRAPHIQUES IX
REFERENCES WEBOGRAPHIQUES X
GLOSSAIRE XI
ANNEXES XIII
Annexe A : Guide de démonstration express entre deux machines XIV
TABLE DES MATIERES XV
RESUME XXII
ABSTRACT XXIII

REMERCIEMENTS

Avant tout, nous tenons à exprimer notre profonde gratitude envers la Providence divine, qui nous a accordé la force, le courage et la santé nécessaires pour mener à bien ce travail tout au long de notre période de stage.

Nous adressons également nos remerciements les plus sincères à toutes les personnes ayant contribué, de près ou de loin, à la réalisation de ce mémoire. Qu'elles trouvent ici le témoignage de notre profonde reconnaissance.

Nous souhaitons exprimer notre gratitude aux personnes suivantes :

- Monsieur **HAJALALAINA Aimé Richard**, Professeur et Président de l'Université de Fianarantsoa, pour son soutien et son engagement envers la promotion de l'enseignement supérieur, ainsi que pour avoir permis la réalisation de mes études dans cette prestigieuse université ;
- Monsieur **VERIZA Roberto Francis** Maître de conférences, Président de l'Université de Toliara, pour son engagement envers l'Université de Toliara et pour son soutien à la communauté académique ;
- Monsieur **MAHATODY Thomas**, Professeur, Directeur de l'Ecole Nationale d'Informatique, d'avoir accepté mon inscription au sein de son établissement ;
- Monsieur **RANDRIANOMENJANAHARY Lala Ferdinand**, Maitre de conférences, Encadreur pédagogique, de nous avoir bien encadré tout au longue de ce projet.

LISTE DES FIGURES

Figure 1 : Organigramme de l'Ecole Nationale d'Informatique 6

Figure 2 : Architecture des études correspondant au système LMD 7

Figure 3 : Chronogramme indicatif du travail 16

Figure 4 : Topologie du réseau utilisé pour le projet 18

Figure 5 : Architecture générale : client, réseau, serveur 37

Figure 6 : Installateur de Python avec la case « Add python.exe to PATH » cochée 40

Figure 7 : Vérification des versions de Python et de pip dans le terminal 40

Figure 8 : Le projet ouvert dans Visual Studio Code 41

Figure 9 : Le terminal intégré de VS Code 42

Figure 10 : Lancement du serveur depuis le terminal de Visual Studio Code 42

Figure 11 : Installation des dépendances avec pip 43

Figure 12 : Vérification des bibliothèques 43

Figure 13 : Installation de Wireshark (étape Npcap) 44

Figure 14 : Écran d'accueil de Wireshark et liste des interfaces 45

Figure 15 : Adresse IPv4 de la machine serveur avec ipconfig 47

Figure 16 : Autorisation du pare-feu Windows pour le serveur 48

Figure 17 : Diagramme de séquence du protocole (handshake, authentification, messages) 49

Figure 18 : Dérivation des clés de session à partir du secret X25519 50

Figure 19 : Structure d'un message chiffré sur le réseau 51

Figure 20 : common.py : fonctions send_msg, _recv_exact et recv_msg 54

Figure 21 : common.py : fonctions aes_encrypt et aes_decrypt (lignes 47 à 60) 55

Figure 22 : common.py : fonctions du handshake 56

Figure 23 : common.py : classe Channel 57

Figure 24 : common.py : fonction derive_channel 57

Figure 25 : common.py : hash_password et auth_proof 58

Figure 26 : server.py : constantes de configuration 59
Figure 27 : server.py : fonction load_or_create_key 59
Figure 28 : server.py : limitation des échecs et recherche d'un utilisateur 60
Figure 29 : server.py : fonction authenticate 61
Figure 30 : server.py : fonction handle_client 62
Figure 31 : server.py : fonctions addrs_of_user et broadcast 63
Figure 32 : client.py : fonction verify_server 64
Figure 33 : client.py : fonction handshake 64
Figure 34 : client.py : fonction login 65
Figure 35 : chat_core.py : classe ClientCore 66
Figure 36 : Échanges entre le thread réseau et le thread principal de Tkinter 67
Figure 37 : gui_client.py : fonction network_thread 68
Figure 38 : Client graphique : écran de connexion 69
Figure 39 : Client graphique : fenêtre de confiance avec l'empreinte du serveur 69
Figure 40 : Client graphique : discussion avec affichage des messages chiffrés 70
Figure 41 : Serveur graphique au démarrage 71
Figure 42 : Ajout d'un utilisateur 71
Figure 43 : Serveur graphique avec deux clients connectés 72
Figure 44 : Sélection d'un client : le bouton devient « Envoyer à … » 73
Figure 45 : Message ciblé : reçu uniquement par le client choisi 73
Figure 46 : Code de la sélection du destinataire ..... **Erreur ! Signet non défini.**
Figure 47 : Code de la zone de discussion en bulles 74
Figure 48 : Version terminal : serveur et client 76
Figure 49 : Machine A : fenêtre du serveur avec son adresse IP réseau 77
Figure 50 : Machine B : test de connectivité avec ping 78
Figure 51 : Machine B : saisie de l'adresse du serveur 78
Figure 52 : Comparaison des empreintes entre les deux machines 79

Figure 53 : Échange de messages entre les deux machines 79

Figure 54 : Exécution des 19 tests automatiques 83

Figure 55 : Mot de passe incorrect : message du client et journal du serveur 84

Figure 56 : Blocage de l'adresse après trop d'échecs..... 85

Figure 57 : Capture Wireshark de la version en clair : liste des paquets 86

Figure 58 : Capture Wireshark du chat chiffré : liste des paquets 87

Figure 59 : Flux TCP chiffré : seuls des octets illisibles..... 87

LISTE DES TABLEAUX

Tableau 1 : Organisation du système de formation pédagogique de l'Ecole 6

Tableau 2 : Mention et parcours au sein de l'ENI 7

Tableau 3 : Liste des formations existantes à l'ENI 8

Tableau 4 : Débouchés éventuels des jeunes diplômés 10

Tableau 5 : Moyens humains 13

Tableau 6 : Moyens matériels 14

Tableau 7 : Moyens logiciels 14

Tableau 8 : Versions successives du projet 15

Tableau 9 : Caractéristiques du réseau 19

Tableau 10 : Menaces considérées et contre-mesures 22

Tableau 11 : Solutions envisagées 24

Tableau 12 : Propriétés de sécurité et mécanismes associés 25

Tableau 13 : Comparaison des modes opératoires d'AES 26

Tableau 14 : Algorithmes cryptographiques utilisés 29

Tableau 15 : Comparaison des choix cryptographiques 30

Tableau 16 : Comparaison des langages et des bibliothèques 31

Tableau 17 : Comparaison des éditeurs de code 31

Tableau 18 : Comparaison des solutions d'interface graphique 32

Tableau 19 : Comparaison des analyseurs de trafic 32

Tableau 20 : Configuration des deux machines de test 35

Tableau 21 : Ports et adresses utilisés 35

Tableau 22 : Environnement logiciel du projet 36

Tableau 23 : Fichiers du projet et rôle de chacun 37

Tableau 24 : Filtres d'affichage Wireshark utiles 46

Tableau 25 : Format d'un message chiffré 51

Tableau 26 : Choix de conception justifiés 53
Tableau 27 : Fichiers à copier ou non sur la machine cliente 76
Tableau 28 : Problèmes fréquents lors de la démonstration..... 80
Tableau 29 : Tests unitaires automatiques 81
Tableau 30 : Scénarios de tests fonctionnels..... 83
Tableau 31 : Comparaison entre la version en clair et le chat chiffré 88
Tableau 32 : Résultats des tests par menace 88
Tableau 33 : Faiblesses de la version 1 et corrections apportées 90
Tableau 34 : Difficultés rencontrées et solutions retenues..... 90

LISTE DES ABREVIATIONS

AAD : Additional Authenticated Data

AEAD : Authenticated Encryption with Associated Data

AES : Advanced Encryption Standard

DHCP : Dynamic Host Configuration Protocol

GCM : Galois/Counter Mode

HKDF : HMAC-based Key Derivation Function

HMAC : Hash-based Message Authentication Code

IP : Internet Protocol

JSON : JavaScript Object Notation

LAN : Local Area Network

MITM : Man-In-The-Middle

OAEP : Optimal Asymmetric Encryption Padding

PBKDF2 : Password-Based Key Derivation Function 2

PEM : Privacy-Enhanced Mail (format de fichier de clés)

PSS : Probabilistic Signature Scheme

RFC : Request For Comments

RSA : Rivest-Shamir-Adleman

SHA : Secure Hash Algorithm

SR : Systèmes et Réseaux

SSH : Secure Shell

TCP : Transmission Control Protocol

TLS : Transport Layer Security

TOFU : Trust On First Use

VPN : Virtual Private Network

X25519 : Échange de clés Diffie-Hellman sur la courbe Curve25519

INTRODUCTION GENERALE

Les échanges numériques (messagerie, banque en ligne, travail à distance, objets connectés) transitent par des réseaux que ni l'émetteur ni le destinataire ne contrôlent. Entre deux machines, un message traverse des routeurs, des commutateurs et des points d'accès Wi-Fi ; à chacune de ces étapes, une personne mal intentionnée peut lire le trafic, le modifier ou s'en servir pour usurper une identité. Le protocole TCP, sur lequel reposent la plupart des applications réseau, garantit que les octets arrivent complets et dans l'ordre, mais il ne garantit ni leur confidentialité, ni leur intégrité, ni l'identité de l'interlocuteur.

Un simple analyseur de trafic comme Wireshark suffit à lire en clair tout ce qu'un programme naïf envoie sur le réseau, mots de passe compris. C'est pourquoi les protocoles modernes (HTTPS, SSH, messageries chiffrées de bout en bout) ajoutent une couche cryptographique au-dessus de TCP. Le projet « Chiffrement des communications client-serveur » (sujet 8) a pour but d'étudier ces mécanismes en les réalisant à petite échelle : un serveur TCP, un client et une couche de chiffrement négociée par un handshake, c'est-à-dire une version simplifiée de ce que réalise TLS.

La question centrale est donc : comment permettre à deux machines d'échanger des messages sur un réseau non fiable, de façon qu'un attaquant capable de lire, de modifier, de rejouer ou d'injecter du trafic ne puisse ni lire les messages, ni les altérer sans être détecté, ni se faire passer pour le serveur ou pour un utilisateur légitime ?

Pour y répondre, le projet associe un échange de clés Diffie-Hellman sur courbe elliptique (X25519) signé par la clé RSA du serveur, un chiffrement authentifié AES-256-GCM, une authentification de l'utilisateur par défi/réponse et plusieurs protections contre les attaques classiques (rejeu, force brute, saturation). Deux interfaces graphiques en Python (Tkinter) complètent les versions en terminal, et la validation repose sur des tests automatiques, des attaques simulées et une analyse du trafic avec Wireshark.

Ce mémoire s'organise en trois parties. La première partie présente l'École Nationale d'Informatique, l'établissement d'accueil et le projet. La deuxième partie, consacrée à l'analyse préalable, étudie l'environnement réseau existant, ses insuffisances, les menaces à contrer, ainsi que les notions et les outils retenus. La troisième partie décrit la réalisation : présentation de l'environnement, installation et configuration des outils (dont Python et Visual Studio Code), conception du protocole, développement du code et des interfaces, puis tests et validation.

PARTIE I : PRESENTATIONS

Chapitre 1. Présentation de l'Ecole Nationale d'Informatique

L'École Nationale d'Informatique (ENI) est reconnue comme un établissement de référence à Madagascar dans le domaine de la formation en informatique.

1.1. Information d'ordre générale

L'Ecole Nationale d'Informatique, en abrégé ENI, est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Le siège de l'Ecole se trouve à Tanambao-Antaninarenina à Fianarantsoa. L'adresse pour la prise de contact avec l'Ecole est la suivante : Ecole Nationale d'Informatique (ENI) Tanambao, Fianarantsoa. Le numéro de sa boîte postale est 1487 avec le code postal 301. Téléphone : 038 96 566 96 ou 034 76 591 95. Son adresse électronique est la suivante : **eni@eni.mg**. Il dispose également d'un site web : **www.eni.mg**

1.2. Missions et historiques

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme étant le plus puissant secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques.

Cette Ecole Supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI était créée par le décret N° 83- 185 du 24 Mai 1983, comme étant le seul établissement Universitaire Professionnalisé au niveau national, destiné à former des techniciens et des Ingénieurs de haut niveau, aptes à répondre aux besoins et exigences d'Informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a pour conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux notamment :

- En fournissant à des étudiants des connaissances de base en informatique ;
- En leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et en essayant une meilleure adéquation des formations par rapport aux besoins évolutifs des sociétés et des entreprises ;
- En initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC) ;

La filière de formation d'Analystes Programmeurs a été mise en place à l'Ecole en 1983, et a été gelée par la suite en 1996, tandis que la filière de formation d'ingénieurs a été ouverte à l'Ecole en 1986.

Une formation de troisième cycle a été ouverte à l'Ecole a été ouverte à l'Ecole depuis l'année 2003 – 2004 grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa pour le compte de l'ENI et l'Université Paul Sabatier de Toulouse (UPST).

Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'Informatique, et notamment pour préparer la relève des Enseignants-Chercheurs qui étaient en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec les deux parcours de formation :

- Génie Logiciel et base de Données.
- Administration des Système et réseaux.

La mise en place à l'Ecole de ces deux options de formation devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés de limitation de l'effectif des étudiants accueillis à l'Ecole, notamment à cause du manque d'infrastructures, un système de « Formation Hybride » a été mise en place à partir de l'année 2010. Il s'agit en effet d'un système de formation semi présentielle et à distance avec l'utilisation de la visioconférence pour la formation à distance. Le système de formation hybride a été ainsi créé à Fianarantsoa ainsi qu'Université de Toliara. Cette formation est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouvert au sein de l'Ecole pour répondre les besoins des entreprises. La formation est destinée aux étudiants titulaires du diplôme de licence (Bac +3) en Mathématiques ou en Statistiques ou en Informatique, etc. La mention IA comporte deux parcours :

- Gouvernance et Ingénierie de Données (GID),
- Objets connectés et Cybersécurité (OCC).

Le principe de l'enseignement pour le parcours GID offre aux l'étudiants des compétences scientifiques et techniques spécialisées en Science de données. Pour le parcours OCC, les étudiants octroient la double spécialité premièrement en internet des objets et deuxièmement en cybersécurité. La formation de master est axée sur l'ensemble d'applications de l'Intelligence Artificielle.

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Cette mention propose deux (2) parcours distincts :

- Métiers du Digital (MDi) – disponible aux niveaux Licence et Master. Ce parcours est destiné aux étudiants titulaires du diplôme de Baccalauréat, toutes séries confondues. Il est élaboré pour équiper les étudiants des compétences essentielles dans les domaines du marketing digital, de la communication digitale, et de la gestion de projets digitaux.
- Audit des Systèmes d'Information (ASI) – proposé au niveau Master uniquement. Ce parcours s'adresse aux étudiants titulaires d'une licence (Bac +3) en Mathématiques, en Statistiques ou en Informatique. Il est structuré pour approfondir les connaissances théoriques et pratiques nécessaires à l'audit efficace des systèmes d'information.

Le système d'enseignement adopté pour cette mention est un système en ligne. Ainsi, les cours et ressources pédagogiques sont accessibles en permanence via la plateforme numérique Moodle. Par ailleurs, des regroupements en présentiel ou en ligne seront organisés périodiquement, conformément au planning défini par les responsables pédagogiques.

1.3. Organigramme institutionnel

L'organigramme de l'Ecole est inspiré des dispositions du décret N° 83-185 du 24 Mai 1983. L'ENI est administrée par un Conseil d'Ecole, et dirigée par un directeur nommé par un décret adopté en Conseil des Ministres. Le Collège des enseignants regroupant tous les enseignants-chercheurs permanents de l'Ecole est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil Scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte notamment de l'évolution du marché de travail et de l'adéquation des formations dispensées par rapport aux besoins des entreprises.

La figure 1 représente l'organigramme actuel de l'Ecole Nationale d'Informatique.

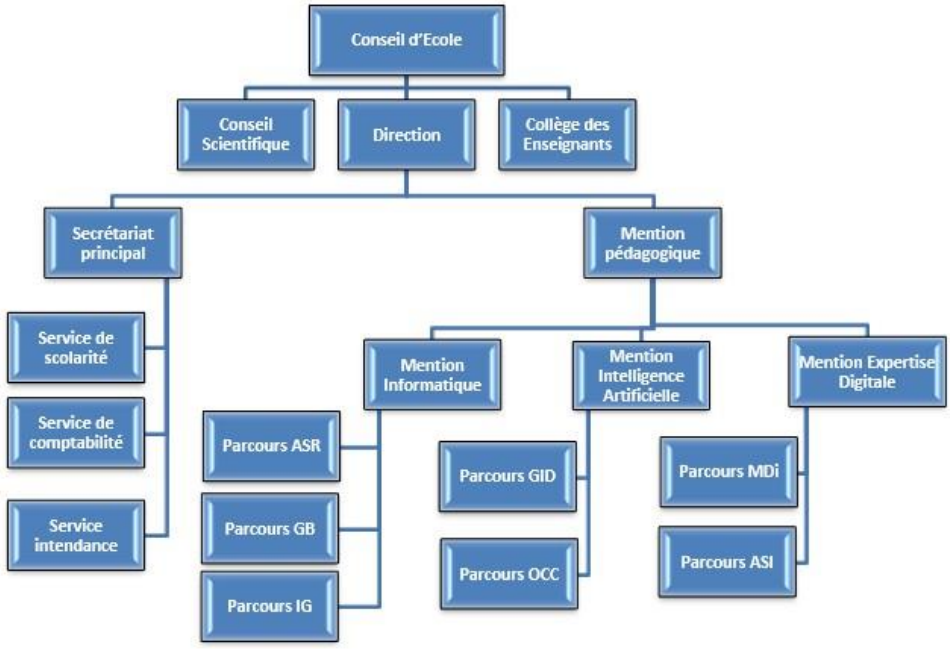


Figure 1 : Organigramme de l'Ecole Nationale d'Informatique

1.4. Domaine de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent sur les domaines suivants :

- Génie logiciel et Base de Données ;
- Administration des Systèmes et Réseaux ;
- Informatique Générale ;
- Modélisation informatique et mathématique des Systèmes complexes ;
- Intelligence artificielle.

Le tableau 1 décrit l'organisation du système de formation pédagogique de l'Ecole.

Tableau 1 : Organisation du système de formation pédagogique de l'Ecole

Formation Théorique	Formation Pratique
➢ Enseignement théorique ➢ Travaux dirigés ➢ Travaux pratiques ➢ Conférences	➢ Etude de cas ➢ Travaux de réalisation ➢ Projets/ Projets tutorés ➢ Voyages d'Etudes ➢ Stages en entreprise

1.5. Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées à l'Ecole ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe deux mentions et cinq parcours.

Le tableau 2 récapitule les mentions et les parcours au sein de l'Ecole :

Tableau 2 : Mention et parcours au sein de l'ENI

Mention	Parcours
Informatique	Génie logiciel et Base de Données (GB)
	Administration des Systèmes et Réseaux (ASR)
	Informatique Générale (IG)
Intelligence Artificielle	Gouvernance et Ingénierie de Données (GID)
	Objets Connectés et Cyber sécurités (OCC)
Expertise Digitale	Métiers du Digital (MDi)
	Audit des Systèmes d'Information (ASI)

La figure 2 représente l'architecture des études correspondant au système LMD.

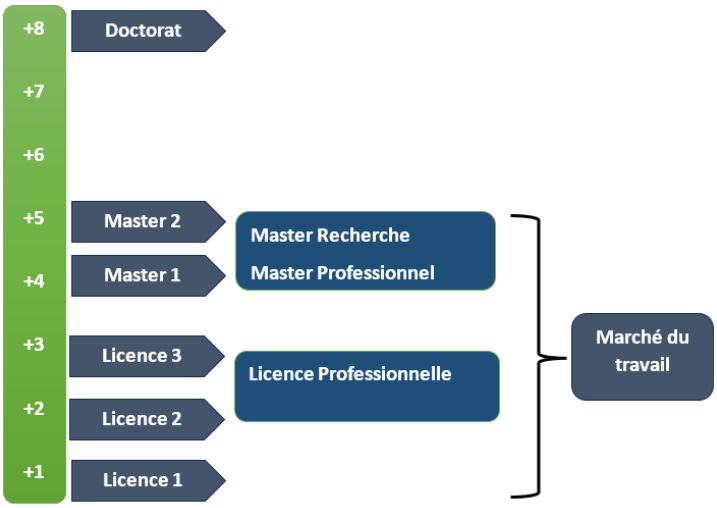


Figure 2 : Architecture des études correspondant au système LMD

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de MASTER se fait

automatiquement pour les étudiants de l'Ecole qui ont obtenu le diplôme de Licence Professionnelle.

Le tableau 3 illustre la liste des formations existantes à l'ENI.

Tableau 3 : Liste des formations existantes à l'ENI

	FORMATION	
	LICENCE PROFESSIONNELLE	MASTER
Condition admission	Par voie de concours	Par voie de concours pour la mention IA
Condition d'accès	Bac de série C, D ou Technique	Être titulaire de licence professionnelle
Durée de Formation	3 ans	2 ans
Diplôme délivré	Diplôme de Licence Professionnelle	Diplôme de Master Professionnel Diplôme de Master Recherche

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire directement dans une Ecole Doctorale.

Les étudiants diplômés de l'Ecole sont plutôt bien accueillis dans les instituts universitaires étrangères (Canada, Suisse, France, …)

1.6. Relation de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'Ecole en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux. L'Ecole dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes publics et privés qui sont des partenaires par l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes par ces derniers. Les compétences que l'Ecole cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, du travail en équipe, le goût de l'expérimentation et l'innovation.

En effet, la vocation de l'ENI est de former des licenciés et des ingénieurs de niveau MASTER avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations à l'Ecole et les besoins évolutifs du marché de l'emploi.

Parmi les sociétés, les entreprises et les organismes partenaires de l'Ecole, on peut citer : ACCENTURE Mauritius, AKATA Goavana, Air Madagascar, Ambre Associates, Airtel,

Agence Universitaire de la Francophonie (AUF), AXIAN, B2B, Banque Centrale, , BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), CEDII-Fianarantsoa, Data Consulting, Central Test, Centre National Antiacridien, CNRE, COLAS, Direction Générale des Douanes, DLC, E-Tech Consulting, , FID, FIHARY Soft, FTM, GNOSYS, GENIUS AT WORK, Hello Tana, IBONIA, INGENOSIA, INSTAT, IOGA, JIRAMA, JOUVE, MADADEV, MAEP, MANAO, MEF, MEN, MESupRES, MFB, , MININTER, Min des Postes/Télécommunications et du Développement Numérique, NEOV MAD, Ny Havana, Madagascar National Parks, OMNITEC, ORANGE, OTME, PRACCESS, QMM Fort-Dauphin, SG Madagasikara SMMC, SMMEC, SNEDADRS Antsirabe, Sénat, Société d'Exploitation du Port de Toamasina (SEPT), SOFTWELL, Strategy Consulting, TELMA, VIVETEC, Société LAZAN'I BETSILEO, WWF, UGD, ARATO, MANAO, MNDPT, NG ACADEMY.NG, Relia, Spoon Consulting …

1.7. Débouchés professionnels et diplômés

Les formations proposées par l'Ecole permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail avec la connaissance d'un métier complet lié à l'informatique aux TIC.

L'Ecole apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Elle a une vocation professionnalisante. Les diplômés en LICENCE et en MASTER issus de l'ENI peuvent faire carrière dans différents secteurs.

L'Ecole bénéficie aujourd'hui de 40 années d'expériences pédagogiques et de reconnaissance auprès des sociétés, des entreprises et des organismes. C'est une Ecole Supérieure de référence en matière informatique.

D'une manière générale, les diplômés de l'ENI n'éprouvent pas de difficultés particulières à être recrutés au terme de leurs études. Cependant, l'ENI recommande à ses diplômés de promouvoir l'entrepreneuriat en TIC et de créer des cybercafés, des SSII ou des bureaux d'études.

Le tableau 4 représente les débouchés éventuels selon la mention.

Tableau 4 : Débouchés éventuels des jeunes diplômés

Mention	Débouchés professionnels
INFORMATIQUE	GB : Développeur logiciel / développeur full-stack, Ingénieur en développement d'applications (desktop, web, mobile), Architecte logiciel, Intégrateur d'applications, Développeur d'API, Administrateur de bases de données (DBA), Ingénieur en conception de bases de données, Développeur SQL / PL-SQL, Spécialiste en migration et optimisation de bases de données, Data engineer (ingénieur en traitement de données), Analyste programmeur, Analyste fonctionnel, Chef de projet informatique, Scrum master / Product owner (en méthodologie Agile), Ingénieur QA (Quality Assurance), Testeur logiciel / testeur automatisé, Spécialiste maintenance applicative, Expert en refactoring et optimisation de code, Analyste de données (Data analyst), Concepteur d'outils décisionnels (BI), Ingénieur en Big Data, Machine Learning Engineer, Consultant fonctionnel ou freelance ASR : Administrateur systèmes et réseaux, Ingénieur systèmes et réseaux, Administrateur systèmes Linux / Windows, Administrateur cloud (AWS, Azure, Google Cloud), Ingénieur virtualisation (VMware, Hyper-V, Proxmox), Administrateur sécurité (Security Administrator), Ingénieur cybersécurité, Analyste SOC (Security Operations Center), Spécialiste firewall / VPN, Responsable sécurité informatique (RSSI – avec expérience), Technicien support informatique (N2/N3), Technicien réseaux et télécoms, Ingénieur exploitation, Administrateur messagerie (Exchange, Microsoft 365), Technicien datacenter, Architecte réseaux, Architecte systèmes, Consultant infrastructure, Chef de projet systèmes et réseaux, Ingénieur DevOps, Administrateur Kubernetes / Docker, Cloud engineer, Spécialiste automatisation (Ansible, Terraform) IG : GB et SR en même temps
INTELLIGENCE ARTIFICIELLE	GID : Développeur IA, Data Scientist, Entrepreneur IA, Consultant IA, Responsable marketing IA, Business Developer IA, Product Owner, Consultant en cybersécurité, Directeur de l'Innovation ; OCC : Analyste en cybersécurité, Ingénieur en cybersécurité, Spécialiste en réponse aux incidents, Architecte de sécurité, Consultant en

	cybersécurité, Auditeur en cybersécurité, Expert en sécurité des données, Responsable de la sécurité des informations, Chercheur en cybersécurité, Formateur en cybersécurité, Ingénieur IoT, Développeur IoT, Architecte IoT, Spécialiste en sécurité IoT, Analyste de données
EXPERTISE DIGITALE	MDi : Spécialiste en Marketing Digital, Gestionnaire de Contenu Web, Analyste UX/UI, Product Owner, Chef de Projet Digital, Stratège en Médias Sociaux, Analyste de Performance Digitale, Chef de Projet Digital, Stratège en Médias Sociaux, Concepteur-Rédacteur Web Analyste de Performance Digitale, Expert en Commerce Électronique, Expert en Commerce Électronique, Spécialiste SEO/SEM, Manager de Communauté (Community Manager), Conseiller en Stratégie Digitale, Responsable CRM (Customer Relationship Management), Spécialiste en Analytique et Intelligence d'Affaires (Business Intelligence), Consultant en Transformation Digitale, Responsable des Partenariats Digitaux, Spécialiste en Cybersécurité Marketing, Directeur de l'Innovation Digitale, Manager de l'E-réputation ASI : Auditeur des Systèmes d'Information, Consultant en Sécurité des Systèmes d'Information, Analyste de Risques IT, Chef de Projet SI, Responsable de la Conformité IT, Spécialiste en Cyber Forensics, Directeur de l'Audit et de la Sécurité des Systèmes d'Information, Expert en Gouvernance IT, Formateur ou Éducateur en Sécurité des Chercheur en Sécurité IT

1.8. Ressources humaines

Les ressources humaines sont citées ci-dessous selon leurs responsabilités :

- Directeur de l'Ecole : Monsieur MAHATODY Thomas, Professeur
- Responsable de la Mention « Informatique » : Monsieur RALAIVAO Jean Christian, Maître de Conférences
- Responsable de la Mention « Intelligence Artificielle » : Monsieur DIMBISOA William Germain, Maître de Conférences

L'ENI compte treize (13) enseignants permanents dont un (01) Professeur Titulaire, deux (02) Professeurs, huit (08) Maîtres de Conférences, deux (02) Assistants d'Enseignement Supérieur et de Recherche.

Chapitre 2. Description du projet

3.1. Formulation

Le projet consiste à concevoir et à réaliser un système de communication sécurisée entre deux machines : un serveur TCP capable de gérer plusieurs clients et un client, reliés par un réseau local. Les messages échangés sont protégés par une couche de chiffrement négociée au moyen d'un handshake sécurisé, avec authentification du serveur et de l'utilisateur. Le projet relève de la sécurité des réseaux et de la cryptographie appliquée.

Le problème à résoudre se formule ainsi :

> **Comment permettre à deux machines d'échanger des messages sur un réseau non fiable, de façon qu'un attaquant capable de lire, de modifier, de rejouer ou d'injecter du trafic ne puisse ni lire les messages, ni les altérer sans être détecté, ni se faire passer pour le serveur ou pour un utilisateur légitime ?**

Répondre à cette question demande de combiner plusieurs briques : un chiffrement des données, un échange de clés sûr au début de la connexion, une authentification du serveur et de l'utilisateur, et des protections contre les attaques classiques (rejeu, force brute, saturation).

3.2. Objectif et besoin de l'utilisateur

L'objectif général est de sécuriser la communication entre un client et un serveur à l'aide du chiffrement symétrique et asymétrique. Les besoins de l'utilisateur se déclinent ainsi :

- échanger des messages confidentiels avec le serveur sur un réseau non fiable ;
- avoir la certitude de dialoguer avec le bon serveur ;
- s'authentifier sans exposer son mot de passe ;
- être protégé contre la modification et le rejeu des messages ;
- disposer d'une interface graphique simple, utilisable pour une démonstration entre deux machines ;
- pour l'administrateur du serveur : créer des comptes, voir les clients connectés et répondre à tous les clients ou à un seul.

3.2.1. Cahier des charges et objectifs complémentaires

Le sujet 8 du projet de fin d'année, intitulé « Chiffrement des communications client-serveur », demande de sécuriser la communication entre deux machines à l'aide du chiffrement symétrique ou asymétrique. Le cahier des charges comprend trois exigences :

- créer un mini serveur TCP et un client ;
- ajouter une couche de chiffrement (RSA ou AES) sur les messages échangés ;
- implémenter une négociation de clé sécurisée (handshake).

Les compétences visées sont la cryptographie appliquée, la sécurité des sockets et l'authentification. Au-delà de ce minimum, des objectifs complémentaires sont fixés :

- utiliser des algorithmes actuels et des bibliothèques éprouvées plutôt que d'inventer des primitives personnelles ;
- garantir la confidentialité **et** l'intégrité des messages, et empêcher leur rejeu ;
- assurer la **confidentialité persistante** (forward secrecy) : la fuite ultérieure d'une clé à long terme ne doit pas révéler les conversations passées ;
- authentifier le serveur (contre l'usurpation) et l'utilisateur, sans jamais transmettre le mot de passe ;
- gérer plusieurs clients en parallèle et permettre au serveur de répondre à tous ou à un seul client ;
- fournir une interface graphique claire, utilisable pour une démonstration entre deux machines ;
- valider le tout par des tests automatiques, des attaques simulées et une analyse du trafic avec Wireshark.

3.3. Moyens nécessaires à la réalisation du projet

3.3.1. Moyens humains

Le tableau 5 présente les personnes impliquées dans le projet et leur rôle.

Tableau 5 : Moyens humains

Fonction	Rôle dans le projet
Étudiant(s) [à compléter]	Analyse, conception, développement, tests et rédaction du mémoire.
Encadreur pédagogique	Suivi académique du projet et validation de la démarche.

Fonction	Rôle dans le projet
Encadreur professionnel [le cas échéant]	Orientation pratique et assistance technique.

3.3.2. Moyens matériels

Le tableau 6 présente le matériel nécessaire à la réalisation et aux tests du projet.

Tableau 6 : Moyens matériels

Désignation	Caractéristiques	Nombre
Ordinateur A (serveur)	[processeur, mémoire, stockage, système d'exploitation : à compléter]	1
Ordinateur B (client)	[processeur, mémoire, stockage, système d'exploitation : à compléter]	1
Équipement réseau	Box, routeur Wi-Fi ou téléphone en partage de connexion	1

3.3.3. Moyens logiciels

Le tableau 7 présente les logiciels nécessaires, tous gratuits et multiplateformes.

Tableau 7 : Moyens logiciels

Logiciel	Rôle
Python 3	Langage de programmation du client, du serveur et des interfaces.
Bibliothèque `cryptography`	Algorithmes cryptographiques (AES-GCM, RSA, X25519, HKDF).
Tkinter	Interfaces graphiques du client et du serveur.
Visual Studio Code	Édition du code et terminal intégré.
Wireshark et Npcap	Capture et analyse du trafic réseau.
Outils du système (`ipconfig`, `ping`, pare-feu)	Adressage, test de connectivité et autorisation des connexions entrantes.

3.4. Résultats attendus

À l'issue du projet, les résultats attendus sont :

- un serveur TCP multi-clients et un client fonctionnels sur deux machines distinctes ;
- des messages chiffrés et authentifiés (AES-256-GCM) avec négociation de clés (X25519 signé par RSA) ;
- une authentification de l'utilisateur sans transmission du mot de passe ;
- des protections démontrées contre l'écoute, la modification, le rejeu, l'usurpation du serveur et la force brute ;
- deux interfaces graphiques (client et serveur) permettant de répondre à tous les clients ou à un client choisi ;
- une analyse du trafic avec Wireshark prouvant la différence entre une communication en clair et une communication chiffrée.

3.5. Chronogramme de travail

Le projet a été réalisé en trois étapes, chacune améliorant la version antérieure :

Le tableau 8 récapitule les trois versions successives du projet et l'apport de chacune.

Tableau 8 : Versions successives du projet

Version	Contenu	Apport principal
Version 1	Serveur et client en terminal ; handshake RSA-OAEP (le client chiffre une clé AES-256 avec la clé publique du serveur) ; messages AES-256-GCM ; mot de passe vérifié avec PBKDF2 ; empreinte du serveur mémorisée.	Solution complète répondant au cahier des charges, avec des démonstrations (version en clair, attaquant « pirate »).
Version 2	Revue de sécurité de la version 1 et corrections : échange X25519 éphémère signé par RSA, deux clés (une par sens), numéro de message, défi/réponse HMAC, limitation des tentatives, taille maximale, délais d'attente.	Forward secrecy, anti-rejeu, mot de passe jamais transmis, résistance à la force brute.
Version 3	Interfaces graphiques Tkinter (client et serveur), choix du destinataire côté serveur, nouveau thème visuel, tests automatiques étendus à 19 tests.	Utilisabilité et démonstration à deux machines.

Le chronogramme indicatif du travail est représenté en semaines. Les durées sont à ajuster selon le calendrier réel du projet [à compléter].

La figure 3 montre le schéma intitulé « Chronogramme indicatif du travail ».

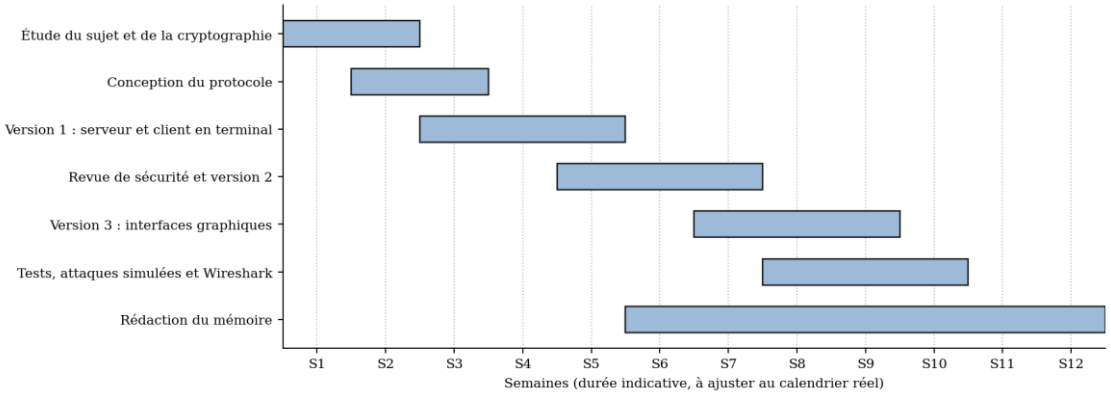


Figure 3 : Chronogramme indicatif du travail

PARTIE II : ANALYSE PREALABLE

Chapitre 4. Analyse de l'existant

4.1. Analyse de l'existant

L'analyse de l'existant porte sur l'environnement réseau dans lequel le projet est réalisé et testé : deux ordinateurs reliés par un réseau local. Les éléments décrits dans cette section sont à adapter à la configuration réelle (mentions [à compléter]).

4.1.1. Topologie réseau

Les deux machines sont raccordées au même réseau local selon une topologie en étoile : un équipement central (box, routeur Wi-Fi, point d'accès ou téléphone en partage de connexion) relie le poste serveur et le poste client et fournit l'accès à Internet. Aucun routeur supplémentaire n'est nécessaire : les deux machines appartiennent au même sous-réseau IP et communiquent directement.

La figure 4 montre le schéma intitulé « Topologie du réseau utilisé pour le projet ».

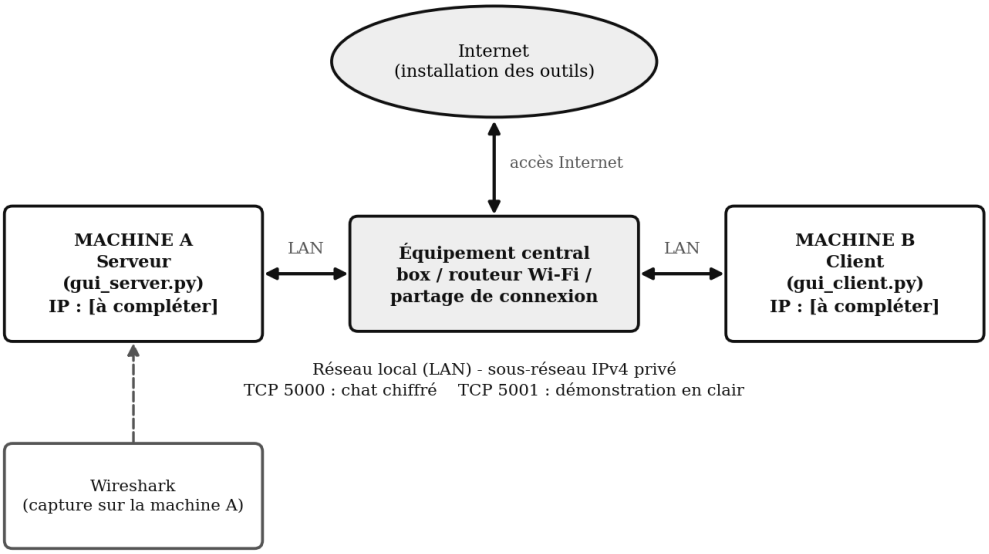


Figure 4 : Topologie du réseau utilisé pour le projet

4.1.2. Types de réseau et connexion Internet

Le réseau est un réseau local (LAN) de type Wi-Fi ou Ethernet, selon les machines, avec un adressage IPv4 privé attribué par DHCP. La connexion à Internet [fournisseur et débit à compléter] sert uniquement à télécharger les outils (Python, Visual Studio Code, Wireshark) : le fonctionnement du projet ne dépend pas d'Internet.

Le tableau 9 résume les caractéristiques du réseau utilisé pour le projet.

Tableau 9 : Caractéristiques du réseau

Caractéristique	Valeur
Type de réseau	Réseau local (LAN)
Support	Wi-Fi (IEEE 802.11) ou Ethernet
Adressage	IPv4 privé attribué par DHCP (plages `192.168.x.x` ou `10.x.x.x`)
Protocole de transport	TCP
Port du chat chiffré	5000
Port de la démonstration en clair	5001
Accès à Internet	Utilisé uniquement pour l'installation des outils

4.1.3. Équipements d'interconnexion

Un seul équipement d'interconnexion intervient : la box ou le routeur Wi-Fi, qui réunit les fonctions de commutateur, de point d'accès et de routeur avec traduction d'adresses. Aucune configuration particulière n'est requise. Certains réseaux d'établissement activent toutefois l'isolation des clients, qui empêche deux machines de se joindre ; dans ce cas, le partage de connexion d'un téléphone est utilisé.

4.1.4. Serveurs

Il n'existe pas de serveur dédié : le rôle de serveur est tenu par un poste de travail (machine A) qui exécute `server.py` ou `gui_server.py`. Ce serveur applicatif écoute sur le port TCP 5000, gère plusieurs clients en parallèle (un thread par connexion) et conserve localement sa clé privée (`server_key.pem`) et les comptes des utilisateurs (`users.json`).

4.1.5. Postes de travail

Deux postes de travail sont utilisés : la machine A (serveur) et la machine B (client). Leur configuration est détaillée au chapitre 6.

4.1.6. Passerelle

La passerelle par défaut est la box ou le routeur Wi-Fi. Elle n'intervient pas dans les échanges entre les deux machines, qui restent internes au réseau local, mais assure l'accès à Internet pour l'installation des outils.

4.1.7. Partage des données

Aucun partage de fichiers en réseau (SMB, FTP) n'est mis en place. Le dossier du projet est copié sur chaque machine (clé USB, stockage en ligne). Les données sensibles (clé privée du serveur, comptes) restent exclusivement sur la machine serveur.

4.1.8. Routage

Le routage se limite à la route par défaut vers la passerelle. Les deux machines étant dans le même sous-réseau, les paquets du projet ne traversent aucun routeur : ils sont livrés directement par l'équipement central, au niveau de la couche liaison.

4.1.9. Surveillance

Aucun outil de surveillance permanent n'est en place. L'observation du trafic est réalisée ponctuellement avec Wireshark, sur la machine A, pour comparer le trafic en clair et le trafic chiffré. Le serveur tient en outre un journal des connexions, des échecs d'authentification et des alertes de sécurité, affiché dans le terminal ou dans la fenêtre du serveur.

4.1.10. Disponibilité

Le service est disponible tant que le serveur est lancé et que le réseau local fonctionne. Aucune redondance n'existe (serveur unique, pas de reprise automatique). Des protections limitées de la disponibilité sont prévues : taille maximale des messages, délais d'attente et blocage des adresses après des échecs répétés.

4.1.11. Protection des échanges existante

Dans une application client-serveur réalisée avec de simples sockets TCP, les données circulent telles quelles : le module `socket` de Python envoie les octets sans aucune transformation. Toute personne située sur le même réseau local peut les capturer avec un outil comme Wireshark et lire les messages et les mots de passe. Ni le serveur ni le client ne peuvent vérifier l'identité de l'autre, et un message modifié ou rejoué est accepté sans contrôle. La

version en clair du projet (`serveur_clair.py` et `client_clair.py`) reproduit cette situation et sert de référence pour l'analyse avec Wireshark.

Des solutions de sécurisation existent et sont largement utilisées :

- **TLS** (HTTPS, SMTPS, etc.) : protocole standard de sécurisation, fondé sur des certificats X.509 et une infrastructure à clés publiques ;
- **SSH** : accès distant sécurisé, qui authentifie le serveur par l'empreinte de sa clé et le client par mot de passe ou par clé publique ;
- **VPN** (IPsec, WireGuard) : tunnel chiffré entre deux machines ou deux réseaux ;
- **messageries chiffrées de bout en bout** : échanges de clés éphémères et renouvellement régulier des clés.

Ces solutions sont matures et recommandées en production. Cependant, elles sont généralement employées comme des boîtes noires : l'utilisateur configure une bibliothèque sans observer ce qui se passe lors du handshake, de la dérivation des clés ou de l'authentification.

4.2. Critique de l'existant

L'analyse de la situation actuelle met en évidence des insuffisances importantes, mais aussi des atouts sur lesquels le projet s'appuie.

4.2.1. Points faibles

- absence de confidentialité : le contenu des messages et les mots de passe sont lisibles sur le réseau avec la version en clair ;
- absence d'intégrité : un message modifié en route est accepté ;
- absence d'authentification du serveur : un faux serveur peut se substituer au vrai ;
- absence de protection contre le rejeu et contre la force brute ;
- pour les solutions toutes faites (TLS, SSH) : fonctionnement difficile à observer dans un cadre pédagogique et mise en place lourde (certificats, configuration) pour un petit projet.

4.2.2. Points forts

- simplicité des sockets TCP : mise en œuvre rapide et compréhension aisée du modèle client-serveur ;
- fiabilité de TCP : les octets arrivent complets et dans l'ordre ;

- maturité des solutions existantes (TLS, SSH) : algorithmes éprouvés et largement audités, dont les principes peuvent être repris ;
- disponibilité de bibliothèques cryptographiques fiables, qui évitent d'écrire soi-même des algorithmes.

4.2.3. Analyse des menaces

Cette section identifie les attaques contre lesquelles le système doit protéger les échanges.

4.2.3.1. Modèle de l'attaquant

Le réseau est supposé **entièrement contrôlé par l'attaquant** : il peut lire tous les paquets, en supprimer, en modifier, en rejouer, en fabriquer, et se placer entre le client et le serveur. Il peut aussi ouvrir lui-même des connexions vers le serveur. En revanche, il est supposé qu'il **ne peut pas casser les algorithmes cryptographiques** eux-mêmes et qu'il n'a pas accès au contenu des machines du client et du serveur (clés privées, fichiers de comptes). Ce modèle est proche du modèle de Dolev-Yao utilisé pour analyser les protocoles.

On distingue l'**attaquant passif**, qui ne fait qu'écouter (par exemple avec Wireshark sur un Wi-Fi partagé), et l'**attaquant actif**, qui interagit avec le trafic. La difficulté d'un protocole est surtout de résister au second.

4.2.3.2. Menaces considérées et contre-mesures

Le tableau 10 dresse la liste des menaces considérées, de la contre-mesure associée et de son emplacement dans le code.

Tableau 10 : Menaces considérées et contre-mesures

Menace	Description	Contre-mesure	Où
Écoute passive	L'attaquant lit les messages et le mot de passe sur le réseau.	Chiffrement AES-256-GCM ; mot de passe jamais envoyé.	`Channel`, `auth_proof`
Homme du milieu	L'attaquant se place entre client et serveur et fait deux échanges de clés.	Signature RSA-PSS des clés éphémères + vérification de l'empreinte du serveur.	`verify_handshake`, `verify_server`
Modification	L'attaquant change des octets d'un message.	Tag GCM : le déchiffrement échoue, connexion coupée.	`Channel.recv`
Rejeu	L'attaquant renvoie un message valide capturé.	Numéro de message dans l'AAD.	`Channel`

Menace	Description	Contre-mesure	Où
Réflexion	Un message du client est renvoyé au client comme s'il venait du serveur.	Deux clés distinctes, une par sens.	`derive_channel`
Force brute	L'attaquant essaie de nombreux mots de passe.	Pause d'1 s par échec, blocage de l'adresse IP (6 échecs en 60 s), PBKDF2 côté client.	`authenticate`, `is_blocked`
Énumération des comptes	Déduire quels noms d'utilisateur existent d'après les réponses.	Faux sel stable et faux hash pour les comptes inconnus.	`user_record`
Saturation	Annoncer un message énorme, ouvrir des connexions muettes.	Taille maximale de 64 Ko, délais d'attente (15 s, 5 min).	`recv_msg`, `handle_client`
Fuite de la clé du serveur	La clé privée RSA est volée plus tard.	Confidentialité persistante : les secrets viennent de clés X25519 éphémères.	`new_ephemeral`
Usurpation d'identité	Se connecter avec le compte d'un autre.	Défi/réponse HMAC lié à la session ; hash PBKDF2 salé.	`auth_proof`

4.2.3.3. Ce qui est hors périmètre

- La compromission d'une des deux machines (programme espion, vol du fichier `server_key.pem`).
- Les attaques par canaux auxiliaires (mesures de temps fines, consommation électrique).
- Les attaques de déni de service massives et distribuées.
- L'anonymat : un observateur voit toujours qui parle à qui (adresses IP) et la taille approximative des messages.
- La confiance initiale dans le serveur (TOFU) : elle repose sur la vérification de l'empreinte par l'utilisateur lors de la première connexion.

4.3. Proposition de solution

4.3.1. Solutions envisagées

Le tableau 11 compare les trois solutions envisagées pour sécuriser la communication.

Tableau 11 : Solutions envisagées

Solution	Principe	Avantages	Limites
Solution 1 : TLS avec le module `ssl`	Sockets enveloppés par TLS, certificats X.509	Standard, très robuste, peu de code	Fonctionnement opaque ; création et gestion de certificats ; peu de mécanismes cryptographiques mis en évidence
Solution 2 : chiffrement symétrique avec clé pré-partagée	Même clé AES écrite à l'avance sur les deux machines	Simple à programmer	Pas de négociation de clé ; distribution manuelle ; pas de confidentialité persistante ; pas d'authentification du serveur
Solution 3 : protocole hybride avec handshake (retenue)	Échange X25519 signé RSA, dérivation HKDF, messages AES-256-GCM, authentification défi/réponse	Répond à toutes les exigences du sujet ; chaque mécanisme est visible et testable	Pas de certificats : confiance par empreinte ; protocole à valider soigneusement

4.3.2. Solution retenue

La solution 3 est retenue. Elle répond à l'exigence de négociation de clé sécurisée (handshake), met en œuvre à la fois le chiffrement symétrique et asymétrique et permet de démontrer chaque protection par des tests. Elle comprend :

- un serveur TCP multi-clients et un client, en terminal et en interface graphique ;
- un handshake X25519 éphémère signé par la clé RSA du serveur, avec dérivation des clés par HKDF ;
- des messages chiffrés et authentifiés avec AES-256-GCM, une clé par sens et un numéro de message ;
- une authentification de l'utilisateur par défi/réponse HMAC, sans transmission du mot de passe ;
- des protections contre la force brute, la saturation et l'usurpation du serveur ;
- des tests automatiques, des attaques simulées et une analyse du trafic avec Wireshark.

Chapitre 5. Études et choix des outils

Ce chapitre rassemble l'étude des notions nécessaires, la comparaison des solutions techniques disponibles et la présentation des outils retenus.

5.1. Étude des notions de sécurité et de cryptographie

Cette section présente les notions nécessaires pour comprendre les choix de conception. Chaque notion est suivie de son usage concret dans le projet.

5.1.1. Les objectifs de la sécurité des communications

On évalue la sécurité d'un canal de communication selon quelques propriétés. Aucune n'est automatique : chacune demande un mécanisme précis.

Le tableau 12 associe chaque propriété de sécurité recherchée au mécanisme qui l'assure dans le projet.

Tableau 12 : Propriétés de sécurité et mécanismes associés

Propriété	Signification	Mécanisme utilisé dans le projet
Confidentialité	Seuls l'émetteur et le destinataire peuvent lire le message.	Chiffrement AES-256-GCM avec une clé de session secrète.
Intégrité	Toute modification du message pendant le transport est détectée.	Tag d'authentification de GCM (16 octets) vérifié à chaque réception.
Authenticité	On sait avec qui on parle : le serveur est le bon serveur, l'utilisateur est le bon utilisateur.	Signature RSA-PSS du serveur + empreinte de clé ; défi/réponse HMAC pour l'utilisateur.
Fraîcheur (anti-rejeu)	Un message valide capturé ne peut pas être renvoyé plus tard.	Numéro de message dans les données authentifiées de GCM.
Confidentialité persistante	Si une clé à long terme fuite, les anciennes sessions restent illisibles.	Clés X25519 éphémères, jetées après chaque session.
Disponibilité	Le service reste utilisable malgré des clients malveillants.	Limite de taille, délais d'attente, blocage après échecs répétés (protection partielle).

5.1.2. Communication par sockets TCP

Un **socket** est le point d'accès d'un programme au réseau. En TCP, le modèle est celui du client et du serveur : le serveur crée un socket, le lie à une adresse et à un port (`bind`), se met à l'écoute (`listen`) puis accepte les connexions entrantes (`accept`) ; le client crée un socket et se connecte (`connect`). Une fois la connexion établie, chaque côté lit et écrit des octets avec `recv` et `send`. Dans ce projet, le serveur écoute sur le port **5000** et sur toutes les interfaces (`0.0.0.0`), ce qui permet à une autre machine du réseau de s'y connecter.

TCP fournit un **flux d'octets** et non une suite de messages : un appel `send` de 100 octets peut être reçu en deux morceaux de 40 et 60 octets, ou fusionné avec le message qui le suit. Il faut donc délimiter soi-même les messages. Un préfixe de longueur est utilisé : chaque message est précédé de 4 octets (entier non signé, gros-boutiste) indiquant sa taille. Le récepteur lit d'abord ces 4 octets, puis exactement le nombre d'octets annoncé. Cette méthode est simple et robuste, à condition de **limiter la taille annoncée** pour qu'un attaquant ne puisse pas demander au serveur de réserver 4 Go de mémoire.

Pour servir plusieurs clients à la fois, le serveur crée un **thread** par connexion. Les structures partagées (liste des clients connectés, compteurs d'échecs) sont protégées par des verrous (`threading.Lock`) afin d'éviter les accès concurrents.

5.1.3. Chiffrement symétrique : AES et le mode GCM

Le chiffrement symétrique utilise **la même clé** pour chiffrer et déchiffrer. L'algorithme **AES** (Advanced Encryption Standard) travaille sur des blocs de 128 bits avec des clés de 128, 192 ou 256 bits. Il est très rapide (souvent accéléré directement par le processeur) et n'a pas d'attaque pratique connue ; des clés de 256 bits sont utilisées dans le projet.

AES ne chiffre qu'un bloc de 16 octets ; pour chiffrer un message complet il faut un **mode opératoire**. Le choix du mode est déterminant :

Le tableau 13 compare les principaux modes opératoires d'AES et indique celui qui est retenu.

Tableau 13 : Comparaison des modes opératoires d'AES

Mode	Principe	Intégrité ?	Verdict
ECB	Chaque bloc chiffré indépendamment.	Non	À proscrire : des blocs identiques donnent des blocs chiffrés identiques (motifs visibles).

Mode	Principe	Intégrité ?	Verdict
CBC	Chaque bloc est mélangé au bloc antérieur avant chiffrement.	Non	Correct pour la confidentialité, mais vulnérable aux modifications et aux attaques par oracle de bourrage sans authentification séparée.
CTR	AES produit un flux pseudo-aléatoire combiné au message par XOR.	Non	Rapide et parallélisable, mais n'authentifie rien.
GCM	CTR + un calcul d'authentification (GHASH) qui produit un tag de 128 bits.	**Oui**	**Choix retenu** : chiffrement authentifié (AEAD) standard, utilisé dans TLS 1.3.

Avec AES-GCM, chaque chiffrement prend en entrée la clé, un **nonce** de 12 octets, le message et éventuellement des **données associées** (AAD) qui sont authentifiées mais non chiffrées. Il produit le texte chiffré suivi d'un **tag** de 16 octets. Au déchiffrement, si le moindre bit du texte chiffré, du nonce, de l'AAD ou du tag a été modifié, le déchiffrement échoue avec une exception (`InvalidTag` dans la bibliothèque Python utilisée). Règle fondamentale : **ne jamais réutiliser le même couple (clé, nonce)**. Ici le nonce est tiré aléatoirement pour chaque message (`os.urandom(12)`) ; avec 96 bits aléatoires et le faible volume de messages d'un chat, la probabilité de collision est négligeable.

Dans ce projet, l'AAD contient le **numéro du message** (compteur sur 8 octets). Il n'est pas transmis : les deux côtés le recalculent. Si un attaquant rejoue, supprime ou réordonne des messages, les compteurs ne correspondent plus et le déchiffrement échoue.

5.1.4. Chiffrement asymétrique : RSA

Le chiffrement asymétrique utilise **deux clés liées** : une clé publique, que tout le monde peut connaître, et une clé privée, gardée secrète. **RSA** repose sur la difficulté de factoriser le produit de deux grands nombres premiers. Une clé RSA de 2048 bits offre environ 112 bits de sécurité, ce qui reste admis aujourd'hui.

RSA sert à deux choses : **chiffrer** avec la clé publique (seul le détenteur de la clé privée peut déchiffrer) et **signer** avec la clé privée (tout le monde peut vérifier avec la clé publique que la signature vient bien du détenteur de la clé privée). Utilisé « brut », RSA est dangereux : il faut un schéma de bourrage. On utilise **OAEP** pour le chiffrement et **PSS** pour la signature, tous deux normalisés dans PKCS #1 (RFC 8017).

RSA est environ mille fois plus lent qu'AES et ne peut chiffrer que des messages plus courts que la clé (moins de 190 octets avec RSA-2048 et OAEP-SHA-256). C'est pourquoi on l'emploie dans un **schéma hybride** : on utilise la cryptographie asymétrique uniquement pour établir ou authentifier une clé, puis la cryptographie symétrique pour les données. Dans la version 1 du projet, RSA chiffrait directement la clé AES. Dans la version finale, RSA **signe** l'échange de clés éphémères (voir la présentation de X25519), ce qui apporte la confidentialité persistante.

5.1.5. Échange de clés Diffie-Hellman et X25519

L'échange de clés **Diffie-Hellman** permet à deux parties de calculer un **secret commun** en n'échangeant que des valeurs publiques, de sorte qu'un espion qui voit tout l'échange ne peut pas retrouver le secret. Chaque partie génère une paire (valeur privée, valeur publique), envoie sa valeur publique, puis combine sa valeur privée avec la valeur publique de l'autre ; les deux calculs aboutissent au même résultat.

X25519 est l'instance de Diffie-Hellman sur la courbe elliptique Curve25519 (RFC 7748). Les clés publiques et privées font 32 octets, les calculs sont rapides et la conception de la courbe évite plusieurs erreurs d'implémentation classiques. Si les paires de clés sont **éphémères** (créées pour une seule session puis détruites), la compromission ultérieure d'une clé à long terme ne permet pas de retrouver les secrets passés : c'est la **confidentialité persistante** (forward secrecy).

> **Limite essentielle :** Diffie-Hellman seul ne dit pas *avec qui* on échange. Un attaquant placé entre le client et le serveur peut faire un échange avec chacun et relayer les messages en les lisant (attaque de l'homme du milieu). Il faut donc **authentifier** l'échange : c'est le rôle de la signature RSA du serveur et de l'empreinte de sa clé.

5.1.6. Hachage, HMAC et dérivation de clés

Une **fonction de hachage** (ici SHA-256) transforme n'importe quelle donnée en une empreinte de 32 octets, de façon irréversible et très sensible aux modifications. Elle sert à identifier une clé publique (empreinte du serveur) et de brique de base aux mécanismes décrits ensuite.

- **HMAC** (RFC 2104) : empreinte à clé secrète qui prouve la connaissance d'une clé sans la révéler. Utilisé dans le défi/réponse d'authentification.

- **HKDF** (RFC 5869) : dérive plusieurs clés indépendantes à partir d'un secret partagé. Ici, le secret X25519 de 32 octets produit 96 octets : une clé client vers serveur, une clé serveur vers client et un identifiant de session.
- **PBKDF2** (RFC 8018) : dérive une clé à partir d'un mot de passe en répétant le calcul 200 000 fois avec un **sel** aléatoire. Cela ralentit volontairement chaque essai d'un attaquant qui voudrait deviner les mots de passe à partir d'un fichier volé, et le sel empêche d'utiliser des tables précalculées.

5.1.7. Authentification : mot de passe, défi/réponse et empreinte de clé

Il y a deux authentifications distinctes dans le projet. **Le serveur doit prouver son identité au client** : il signe l'échange de clés avec sa clé RSA, et le client compare l'empreinte (SHA-256) de la clé publique reçue avec celle qu'il a mémorisée lors de la première connexion. C'est le modèle « trust on first use » (TOFU) utilisé par SSH : à la première connexion on demande à l'utilisateur de vérifier l'empreinte, ensuite toute modification de cette empreinte déclenche une alerte. Un système plus complet utiliserait des certificats X.509 signés par une autorité de confiance (comme dans HTTPS), ce qui sort du périmètre de ce projet.

L'utilisateur doit prouver son identité au serveur. Plutôt que d'envoyer le mot de passe, on utilise un **défi/réponse** : le serveur envoie un nombre aléatoire (le défi) et un sel ; le client calcule la clé PBKDF2 du mot de passe, puis renvoie un HMAC du défi et de l'identifiant de session avec cette clé. Le serveur, qui connaît la même clé dérivée, calcule le HMAC attendu et compare en temps constant. Le défi change à chaque tentative et la preuve est liée à la session : l'enregistrer et la rejouer ailleurs ne sert à rien.

5.1.8. Récapitulatif des algorithmes utilisés

Le tableau 14 récapitule les algorithmes utilisés, leurs paramètres et leur emplacement dans le code.

Tableau 14 : Algorithmes cryptographiques utilisés

Fonction	Algorithme	Paramètres	Où dans le code
Chiffrement des messages	AES-GCM	clé 256 bits, nonce 96 bits, tag 128 bits	`common.py` : `aes_encrypt`, `Channel`
Échange de clés	X25519 (éphémère)	clés de 32 octets	`common.py` : `new_ephemeral`, `exchange`

Fonction	Algorithme	Paramètres	Où dans le code
Authenticité du serveur	RSA-PSS + SHA-256	clé de 2048 bits, sel de 32 octets	`common.py` : `sign_handshake`
Dérivation des clés	HKDF-SHA-256	sortie de 96 octets	`common.py` : `derive_channel`
Preuve du mot de passe	HMAC-SHA-256	défi de 16 octets + id de session	`common.py` : `auth_proof`
Stockage du mot de passe	PBKDF2-HMAC-SHA-256	200 000 itérations, sel de 16 octets	`common.py` : `hash_password`
Empreinte du serveur	SHA-256	empreinte de la clé publique (PEM)	`client.py` : `fingerprint`

5.2. Comparaison et choix des outils

Chaque brique du projet est choisie après comparaison avec les alternatives courantes. Les critères retenus sont la sécurité, la disponibilité dans une bibliothèque reconnue, la simplicité et l'adéquation avec le sujet.

5.2.1. Algorithmes cryptographiques

Le tableau 15 compare les options étudiées pour chaque besoin cryptographique et indique le choix retenu.

Tableau 15 : Comparaison des choix cryptographiques

Besoin	Options comparées	Choix retenu	Raison
Chiffrement des messages	AES-CBC, AES-CTR, AES-GCM, ChaCha20-Poly1305	AES-256-GCM	Chiffrement authentifié (AEAD), accélération matérielle fréquente, utilisé dans TLS 1.3
Échange de clés	Chiffrement de la clé par RSA, Diffie-Hellman classique, X25519	X25519 éphémère	Confidentialité persistante, clés de 32 octets, calcul rapide
Authenticité du serveur	Aucune, mot de passe partagé, signature RSA, signature Ed25519	Signature RSA-PSS (2048 bits)	Standard, bien pris en charge, signature des clés éphémères
Dérivation des clés	Hachage direct, HKDF	HKDF-SHA-256	Produit plusieurs clés indépendantes liées au contexte

Besoin	Options comparées	Choix retenu	Raison
Stockage des mots de passe	SHA-256 simple, PBKDF2, bcrypt, scrypt, Argon2	PBKDF2-HMAC-SHA-256 (200 000 itérations)	Fourni par la bibliothèque standard, ralentit les attaques par dictionnaire
Preuve du mot de passe	Envoi du mot de passe, défi/réponse HMAC	Défi/réponse HMAC-SHA-256	Le mot de passe ne circule jamais

5.2.2. Langage de programmation et bibliothèques

Le tableau 16 compare les langages et bibliothèques étudiés pour réaliser le projet.

Tableau 16 : Comparaison des langages et des bibliothèques

Option	Avantages	Inconvénients	Décision
Python 3 + `cryptography`	Lisible ; modules `socket` et `threading` standard ; bibliothèque cryptographique reconnue ; Tkinter inclus	Moins rapide qu'un langage compilé	**Retenu**
Java + JCA	Bibliothèques cryptographiques intégrées ; multiplateforme	Code plus long ; interface graphique plus lourde	Écarté
C + OpenSSL	Performant ; proche du système	Gestion de la mémoire et API difficiles ; risque d'erreurs de sécurité	Écarté
Python + PyCryptodome	Large choix d'algorithmes	Moins de garde-fous sur l'usage des primitives	Écarté

5.2.3. Éditeur de code

Le tableau 17 compare les éditeurs de code étudiés pour le développement en Python.

Tableau 17 : Comparaison des éditeurs de code

Option	Avantages	Inconvénients	Décision
Visual Studio Code	Gratuit, léger, terminal intégré, extension Python, plusieurs terminaux en parallèle	Configuration initiale de l'extension	**Retenu**
PyCharm	Très complet pour Python	Plus lourd à installer et à lancer	Écarté

Option	Avantages	Inconvénients	Décision
IDLE	Fourni avec Python	Fonctions limitées, pas de terminal multiple	Écarté

5.2.4. Interface graphique

Le tableau 18 compare les solutions étudiées pour réaliser les interfaces graphiques.

Tableau 18 : Comparaison des solutions d'interface graphique

Option	Avantages	Inconvénients	Décision
Tkinter / ttk	Inclus avec Python ; aucune installation ; suffisant pour une application de messagerie	Apparence à soigner	**Retenu**
PyQt / PySide	Interface très riche	Dépendance externe, installation plus lourde	Écarté
Interface web (Flask et navigateur)	Accessible depuis tout navigateur	Ajoute un serveur web et du JavaScript hors du sujet	Écarté

5.2.5. Analyseur de trafic

Le tableau 19 compare les analyseurs de trafic étudiés pour observer les échanges réseau.

Tableau 19 : Comparaison des analyseurs de trafic

Option	Avantages	Inconvénients	Décision
Wireshark	Interface graphique, filtres, suivi de flux TCP, affichage hexadécimal	Installation de Npcap sous Windows	**Retenu**
tcpdump	Léger, en ligne de commande	Moins adapté pour présenter un flux lisible à un jury	Écarté

5.3. Présentation des outils retenus

5.3.1. Python 3

Python permet d'écrire rapidement des programmes réseau lisibles : le module standard `socket` donne accès à TCP en quelques lignes, `threading` gère la concurrence, et la

bibliothèque `cryptography` offre des implémentations fiables des algorithmes. La clarté du langage est un atout pour un projet pédagogique où l'on doit pouvoir expliquer chaque ligne.

5.3.2. La bibliothèque cryptography et pip

pip installe les bibliothèques externes. Le projet n'en a besoin que d'une, `cryptography`, listée dans le fichier `requirements.txt`. Cette bibliothèque est maintenue activement et s'appuie sur OpenSSL ; elle fournit les algorithmes utilisés par le projet : `AESGCM`, `rsa`, `x25519`, `HKDF`, ainsi que les exceptions `InvalidTag` et `InvalidSignature` qui permettent de détecter les attaques.

5.3.3. Tkinter

Tkinter est la bibliothèque graphique standard de Python, basée sur la boîte à outils Tk. Elle est livrée avec Python sous Windows et macOS ; sous Linux il faut installer le paquet `python3-tk`. Le projet utilise le sous-module **ttk** (widgets thématiques) avec le thème « clam », qui accepte des couleurs personnalisées, afin d'obtenir une interface moderne sans dépendance supplémentaire. Le fichier `ui_theme.py` regroupe les couleurs, les polices, les styles de boutons et la zone de discussion en bulles.

5.3.4. Visual Studio Code

Visual Studio Code (VS Code) est un éditeur de code gratuit de Microsoft. Il offre la coloration syntaxique, l'exploration des fichiers et un **terminal intégré** qui s'ouvre directement dans le dossier du projet, ce qui évite d'avoir à naviguer dans les dossiers en ligne de commande.

5.3.5. Wireshark et Npcap

Wireshark est l'analyseur de protocoles réseau de référence. Il capture les paquets qui passent sur une interface réseau (Wi-Fi, Ethernet…) et les décode. Dans ce projet, il sert à **prouver visuellement** la différence entre une communication en clair et une communication chiffrée. Sous Windows, Wireshark s'appuie sur le pilote de capture **Npcap**, installé en même temps.

5.3.6. Outils réseau du système

Les commandes `ipconfig` (ou `hostname -I` sous Linux), `ping` et le pare-feu du système permettent de trouver l'adresse IP du serveur, de tester la connectivité entre les deux machines et d'autoriser les connexions entrantes. Leur utilisation est détaillée à la section 7.5.

PARTIE III : REALISATION

Chapitre 6. Présentation de l'environnement

Ce chapitre décrit l'environnement matériel, réseau et logiciel dans lequel le projet est développé et testé, ainsi que l'architecture générale de la solution.

6.1. Environnement matériel

Les tests et la démonstration sont réalisés avec deux machines reliées au même réseau local (configuration à compléter).

Le tableau 20 décrit la configuration de la machine serveur et de la machine cliente.

Tableau 20 : Configuration des deux machines de test

	Machine A (serveur)	**Machine B (client)**
Système d'exploitation	Windows 11	Windows 11
Version de Python	3.12.0	3.14.0
Adresse IP sur le réseau	192.168.43.162	192.168.42.129
Réseau	Partage de connexion	Partage de connexion
Programme lancé	`gui_server.py` (ou `server.py`)	`gui_client.py` (ou `client.py`)
Fichiers propres à la machine	`server_key.pem`, `users.json`	`known_<IP>_5000.txt`

6.2. Environnement réseau

Les deux machines communiquent par des connexions TCP sur le réseau local. Chaque programme utilise un port précis, qui doit être autorisé dans le pare-feu de la machine serveur.

Le tableau 21 présente les ports et adresses utilisés par les programmes du projet.

Tableau 21 : Ports et adresses utilisés

Programme	**Port TCP**	**Adresse d'écoute ou de connexion**
`server.py` / `gui_server.py`	5000	Écoute sur toutes les interfaces (`0.0.0.0`)
`client.py` / `gui_client.py`	5000	Connexion à l'adresse IP de la machine A
`serveur_clair.py` / `client_clair.py`	5001	Démonstration en clair, mêmes règles que le chat chiffré

Programme	Port TCP	Adresse d'écoute ou de connexion
`pirate.py`	5002	Relais d'attaque simulée, sur la machine locale uniquement

6.3. Environnement logiciel

Le projet repose uniquement sur des logiciels gratuits et multiplateformes. Chacun est présenté à la section 5.3 et son installation est décrite au chapitre 7.

Le tableau 22 recense les logiciels utilisés, leur rôle et les versions à renseigner.

Tableau 22 : Environnement logiciel du projet

Outil	Rôle dans le projet	Version minimale	Version utilisée
Python 3	Langage de programmation du client, du serveur et des interfaces.	3.9	3.14
pip	Gestionnaire de paquets Python (installation de `cryptography`).	inclus avec Python	21.0
cryptography	Bibliothèque de primitives cryptographiques (AES-GCM, RSA, X25519, HKDF).	41	41.0.0
Tkinter / ttk	Interface graphique (fenêtres, boutons, zone de discussion).	inclus avec Python	8.6
Visual Studio Code	Éditeur de code et terminal intégré.	récente	1.2
Wireshark + Npcap	Analyse du trafic réseau (comparaison clair / chiffré).	4.0	Wireshark 4.0 / Npcap 1.3
Outils réseau du système	`ipconfig`, `ping`, pare-feu : trouver l'adresse IP et tester la connectivité.	—	Windows 11

6.4. Architecture de la solution

6.4.1. Architecture générale

Le système suit le modèle client-serveur. Les deux machines exécutent le **même dossier de projet** ; le module `common.py` contient toutes les fonctions cryptographiques utilisées des deux côtés, ce qui garantit qu'ils parlent exactement le même protocole. Les interfaces (terminal

ou graphique) ne contiennent aucune cryptographie : elles appellent `common.py` directement ou via `chat_core.py`.

La figure 5 montre le schéma intitulé « Architecture générale : client, réseau, serveur ».

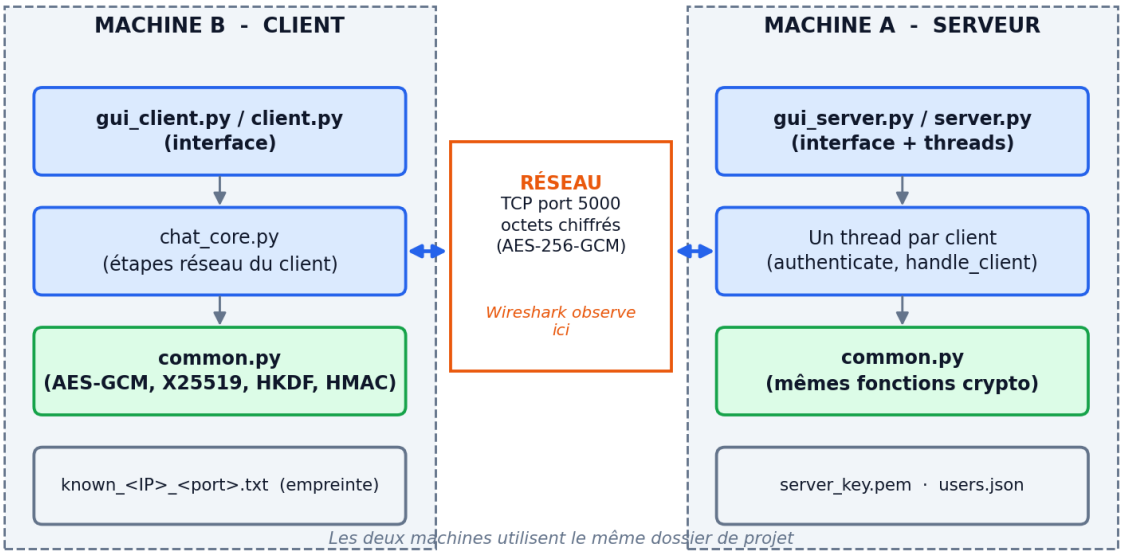


Figure 5 : Architecture générale : client, réseau, serveur

Côté serveur, un thread par client exécute le handshake, l'authentification puis la boucle de réception. Une liste partagée (`clients`, protégée par un verrou) contient les clients authentifiés et permet d'envoyer un message à tous ou à un seul. Côté client, l'interface graphique utilise un thread réseau pour ne jamais bloquer la fenêtre (section 7.8).

6.4.2. Structure du projet

Le tableau 23 présente les fichiers du projet et le rôle de chacun.

Tableau 23 : Fichiers du projet et rôle de chacun

Fichier	Rôle
`common.py`	Cœur cryptographique et réseau : envoi/réception avec préfixe de taille, AES-GCM, X25519, signature RSA-PSS, HKDF, classe `Channel`, hachage des mots de passe, preuve d'authentification.
`server.py`	Serveur multi-clients (terminal) : clé RSA persistante, handshake, authentification, limitation des tentatives, réception, envoi à tous ou à un client (`@nom message`).
`client.py`	Client en terminal : handshake, vérification de l'empreinte du serveur, authentification, envoi et réception en parallèle.

Fichier	Rôle
`chat_core.py`	Étapes du client découpées pour l'interface graphique : connexion, handshake, confiance, login, envoi, réception.
`gui_server.py`, `gui_client.py`	Interfaces graphiques Tkinter du serveur et du client.
`ui_theme.py`	Thème visuel commun : couleurs, styles, bulles de discussion, fenêtres de dialogue.
`create_user.py`	Création d'un compte utilisateur en ligne de commande.
`pirate.py`	Démonstration d'attaque : relais malveillant qui modifie ou rejoue un message.
`serveur_clair.py`, `client_clair.py`	Version volontairement non chiffrée, pour la comparaison avec Wireshark.
`test_securite.py`	19 tests automatiques des protections de sécurité.
`README.md`, `requirements.txt`	Documentation d'utilisation et dépendances.
Fichiers créés à l'exécution	`server_key.pem` (clé privée RSA du serveur), `users.json` (comptes), `known_<IP>_<port>.txt` (empreintes mémorisées par le client).

Chapitre 7. Installation et configuration

Ce chapitre décrit l'installation et l'utilisation des outils (Python, Visual Studio Code, bibliothèques, Wireshark, outils réseau), puis la conception, le développement et le déploiement de la solution sur deux machines.

7.1. Installation de Python

7.1.1. Installation sous Windows

1. Ouvrir le site officiel **python.org**, rubrique Downloads, et télécharger l'installateur de la dernière version stable de Python 3.
2. Lancer l'installateur. Sur le premier écran, **cocher la case « Add python.exe to PATH »** (indispensable pour utiliser `python` dans le terminal), puis cliquer sur « Install Now ».
3. Attendre la fin de l'installation, puis fermer la fenêtre (« Disable path length limit » est facultatif).
4. Ouvrir un nouveau terminal (PowerShell ou Invite de commandes) et vérifier l'installation.

La figure 6 montre la capture d'écran intitulée « Installateur de Python avec la case « Add python.exe to PATH » cochée ».

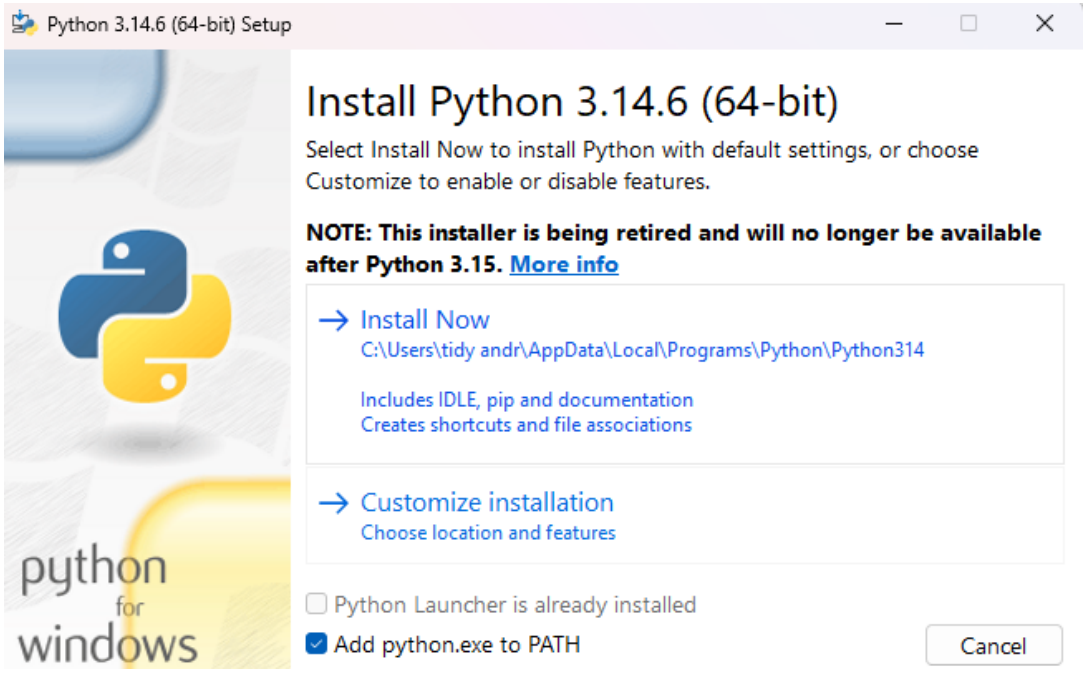


Figure 6 : Installateur de Python avec la case « Add python.exe to PATH » cochée

Sous **Linux** (Debian, Ubuntu), Python est généralement déjà installé ; on ajoute si besoin `sudo apt install python3 python3-pip python3-tk`. Sous **macOS**, on installe Python depuis python.org ou avec Homebrew (`brew install python`). Sur ces systèmes, la commande s'appelle `python3` au lieu de `python`.

7.1.2. Vérification

```
python --version
pip --version
```

La figure 7 montre la capture d'écran intitulée « Vérification des versions de Python et de pip dans le terminal ».

```
E:\Projet_chiffrement\Projet_chiffrement_final>python --version
Python 3.12.0

E:\Projet_chiffrement\Projet_chiffrement_final>pip --version
pip 23.2.1 from C:\Users\11rav\AppData\Local\Programs\Python\Python312\Lib\site-packages\p
ip (python 3.12)
```

Figure 7 : Vérification des versions de Python et de pip dans le terminal

7.2. Installation et utilisation de Visual Studio Code

Visual Studio Code sert d'environnement de travail pour écrire, lire et exécuter le code du projet. Son installation et son utilisation sont décrites dans cette section.

7.2.1. Installation

1. Télécharger VS Code sur **code.visualstudio.com** et lancer l'installateur (options par défaut).
2. Au premier lancement, accepter si VS Code propose d'installer l'**extension Python** (Microsoft) : elle apporte la coloration, l'autocomplétion et l'exécution des fichiers.
3. Ouvrir le projet : menu **Fichier > Ouvrir le dossier…**, puis choisir le dossier extrait du projet. Il faut ouvrir le dossier lui-même, pas un seul fichier.
4. Ouvrir le terminal : menu **Terminal > Nouveau terminal**. Il s'ouvre déjà dans le dossier du projet.

La figure 8 présente le projet ouvert dans Visual Studio Code.

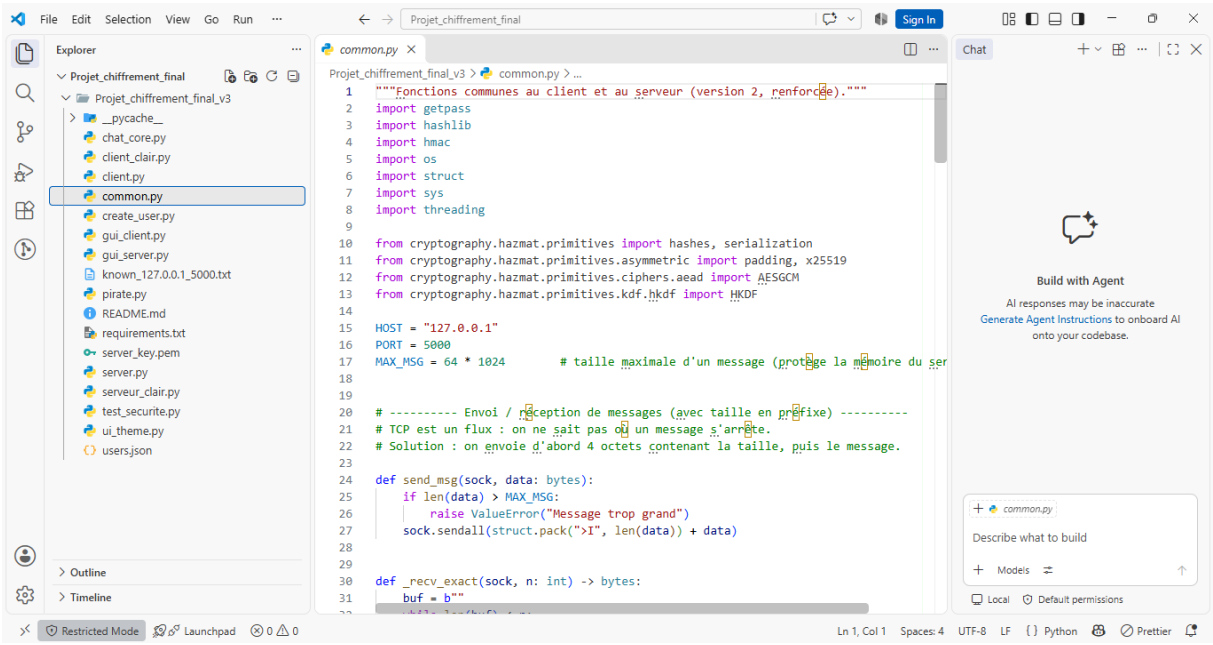


Figure 8 : Le projet ouvert dans Visual Studio Code

La figure 9 présente le terminal intégré de VS Code.



Figure 9 : Le terminal intégré de VS Code

Dans le terminal intégré, il est possible d'ouvrir plusieurs terminaux en parallèle (bouton « + » ou « Fractionner le terminal ») : un pour le serveur, un pour le client, ce qui est pratique pour tester sur une seule machine.

7.2.2. Utilisation de Visual Studio Code pour le projet

Visual Studio Code est utilisé pour consulter le code, exécuter les programmes et réaliser les captures de code du mémoire. La démarche d'utilisation comprend plusieurs étapes :

1. ouvrir le dossier du projet avec **Fichier > Ouvrir le dossier…** ;
2. ouvrir un terminal avec **Terminal > Nouveau terminal** (le terminal s'ouvre dans le dossier du projet) ;
3. lancer un programme avec une commande, par exemple `python gui_server.py` ou `python gui_client.py` ;
4. fractionner le terminal (bouton « Fractionner le terminal ») pour lancer le serveur et le client côte à côte sur une même machine ;
5. consulter dans l'éditeur le code d'un fichier, dont les numéros de ligne sont affichés dans la marge, pour localiser un passage précis.

La figure 10 montre la capture d'écran intitulée « Lancement du serveur depuis le terminal de Visual Studio Code ».

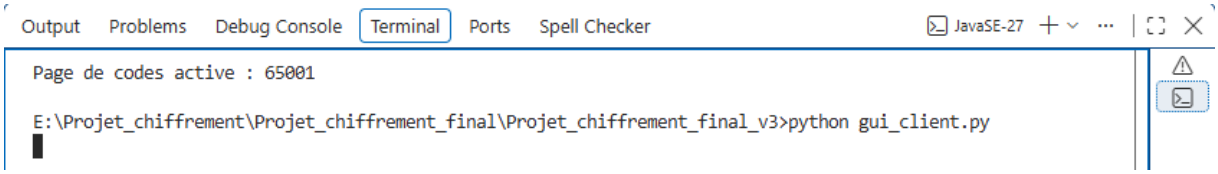


Figure 10 : Lancement du serveur depuis le terminal de Visual Studio Code

7.3. Installation de la bibliothèque cryptography et de Tkinter

Pour l'installer, on se place dans le dossier du projet et on exécute :

```
pip install cryptographie
```

La figure 11 montre la capture d'écran intitulée « Installation des dépendances avec pip ».

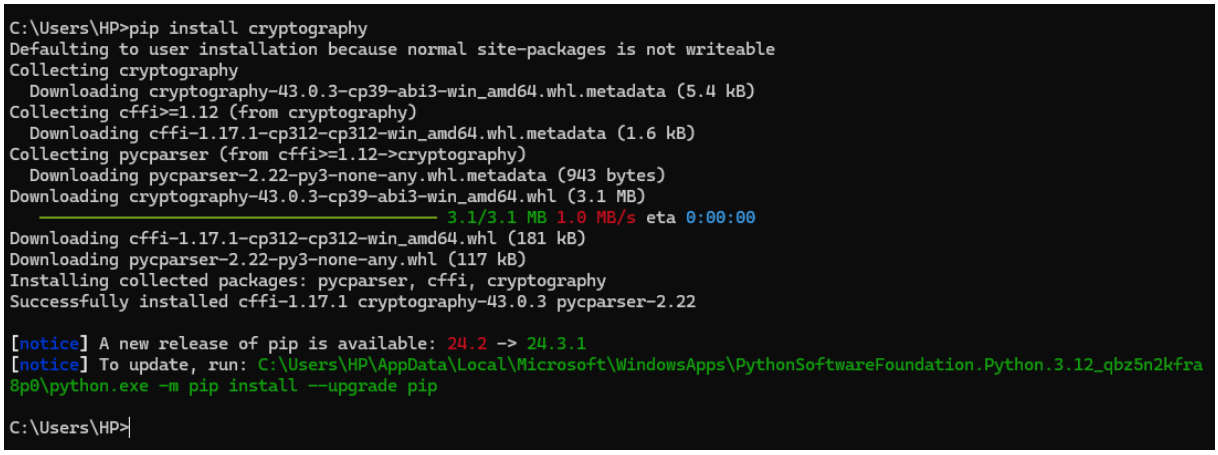


Figure 11 : Installation des dépendances avec pip

On vérifie ensuite que les deux bibliothèques nécessaires sont disponibles (la deuxième, Tkinter, est présentée à la section 5.3) :

```
python -c "import tkinter, cryptography; print('OK')"
```

La figure 12 montre la capture d'écran intitulée « Vérification des bibliothèques ».

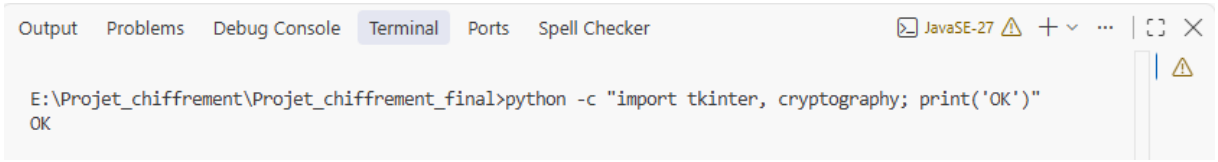


Figure 12 : Vérification des bibliothèques

> **Remarque :** si `pip` n'est pas reconnu, utiliser `python -m pip install -r requirements.txt`. Sous Windows, la commande `py` est aussi disponible à la place de `python`.

Tkinter est inclus avec Python sous Windows et macOS. Sous Linux, son installation se fait avec la commande `sudo apt install python3-tk`. La commande de vérification confirme la disponibilité des deux bibliothèques.

7.4. Installation et utilisation de Wireshark

7.4.1. Installation

1. Télécharger Wireshark sur **wireshark.org** (rubrique Download) et lancer l'installateur.
2. Accepter l'installation de **Npcap** lorsqu'elle est proposée (nécessaire pour capturer). Garder les options par défaut.
3. Terminer l'installation. Si Windows demande un redémarrage, l'effectuer.
4. Lancer Wireshark. L'écran d'accueil liste les **interfaces** disponibles (Wi-Fi, Ethernet, « Adapter for loopback traffic capture » pour le trafic local) avec un petit graphique d'activité.

La figure 13 montre la capture d'écran intitulée « Installation de Wireshark (étape Npcap) ».

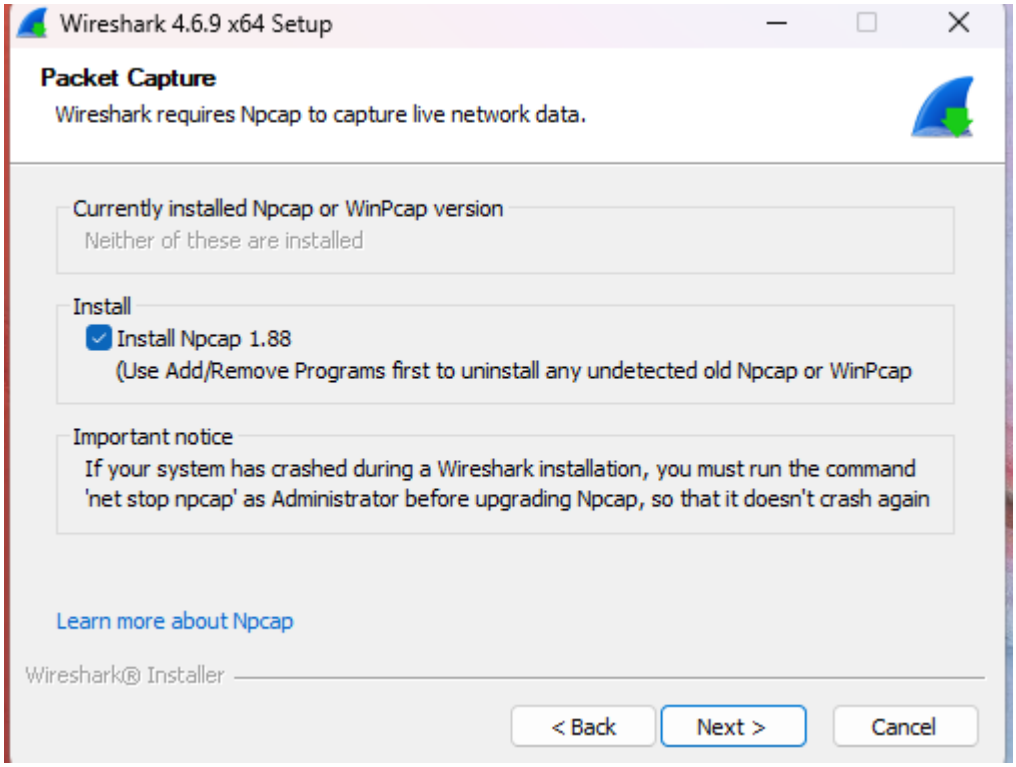


Figure 13 : Installation de Wireshark (étape Npcap)

La figure 14 montre la capture d'écran intitulée « Écran d'accueil de Wireshark et liste des interfaces ».

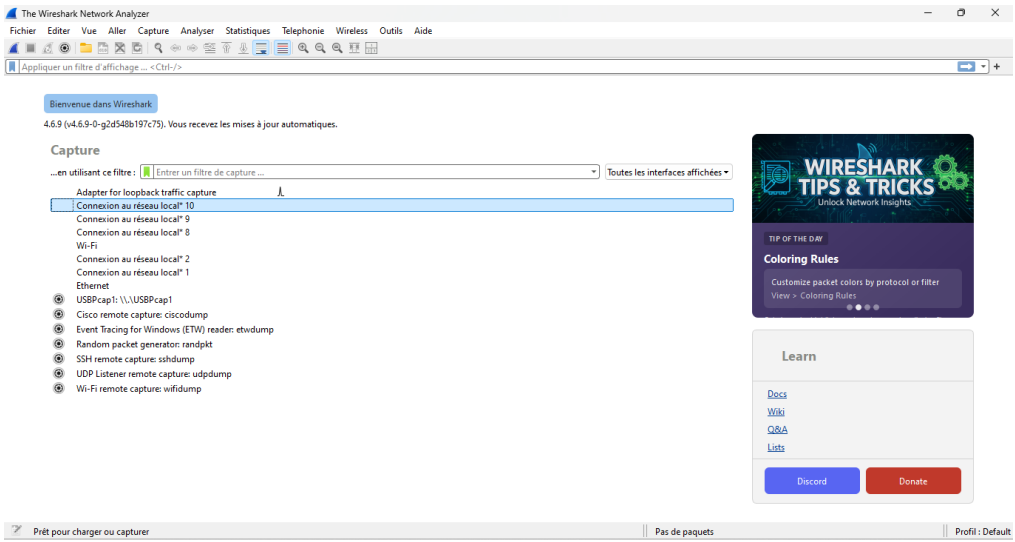


Figure 14 : Écran d'accueil de Wireshark et liste des interfaces

7.4.2. Utilisation pour le projet

1. **Choisir l'interface** : double-cliquer sur celle qui porte le trafic entre les deux machines (Wi-Fi ou Ethernet). Pour un test sur une seule machine, choisir l'interface de loopback.
2. **Filtrer l'affichage** : dans la barre de filtre en haut, saisir `tcp.port == 5000` (chat chiffré) ou `tcp.port == 5001` (démonstration en clair), puis valider avec Entrée. La barre devient verte lorsque le filtre est valide.
3. **Lancer la capture** : bouton bleu en forme d'aileron de requin. Utiliser ensuite l'application normalement.
4. **Arrêter la capture** : bouton carré rouge.
5. **Suivre un flux** : clic droit sur un paquet du dialogue, **Suivre > Flux TCP** (ou `Ctrl+Alt+Maj+T`). Wireshark reconstitue la conversation complète ; en rouge ce qui part du client, en bleu ce qui part du serveur.
6. **Examiner les octets** : le panneau du bas montre le contenu du paquet en hexadécimal et en ASCII.

7. **Enregistrer** : Fichier > Enregistrer sous, au format `.pcapng`, pour conserver la capture comme preuve.

Le tableau 24 présente les filtres d'affichage Wireshark employés pour isoler le trafic du projet.

Tableau 24 : Filtres d'affichage Wireshark utiles

Filtre d'affichage	Effet
`tcp.port == 5000`	Tout le trafic du chat chiffré.
`tcp.port == 5001`	Tout le trafic de la démonstration en clair.
`ip.addr == 192.168.1.20`	Tout le trafic échangé avec une machine précise (remplacer par l'adresse réelle).
`tcp.len > 0`	Uniquement les paquets qui contiennent des données (masque les accusés de réception).
`frame contains "motdepasse123"`	Les paquets qui contiennent cette chaîne : utile pour prouver qu'un mot de passe est visible en clair.

7.5. Outils réseau du système

Pour une démonstration entre **deux machines**, il faut connaître l'adresse IP de la machine serveur et s'assurer que le pare-feu laisse passer les connexions entrantes.

7.5.1. Trouver l'adresse IP

Sous Windows, ouvrir un terminal et exécuter `ipconfig` ; l'adresse à relever est la ligne **Adresse IPv4** de l'interface connectée (par exemple `192.168.1.20` ou `10.x.x.x`). Sous Linux : `hostname -I`. Sous macOS : `ipconfig getifaddr en0`. L'interface graphique du serveur affiche aussi cette adresse en haut à droite.

> **Attention :** l'adresse `127.0.0.1` désigne la machine elle-même (« localhost »). Elle ne fonctionne que si le client et le serveur sont sur la **même** machine. Pour une démonstration à deux machines, il faut donner au client l'adresse IP réseau du serveur. Si la fenêtre du serveur affiche `127.0.0.1`, c'est que la machine n'est pas connectée à un réseau : se connecter au Wi-Fi puis relancer le serveur.

```
ipconfig
```

La figure 15 montre la capture d'écran intitulée « Adresse IPv4 de la machine serveur avec ipconfig ».

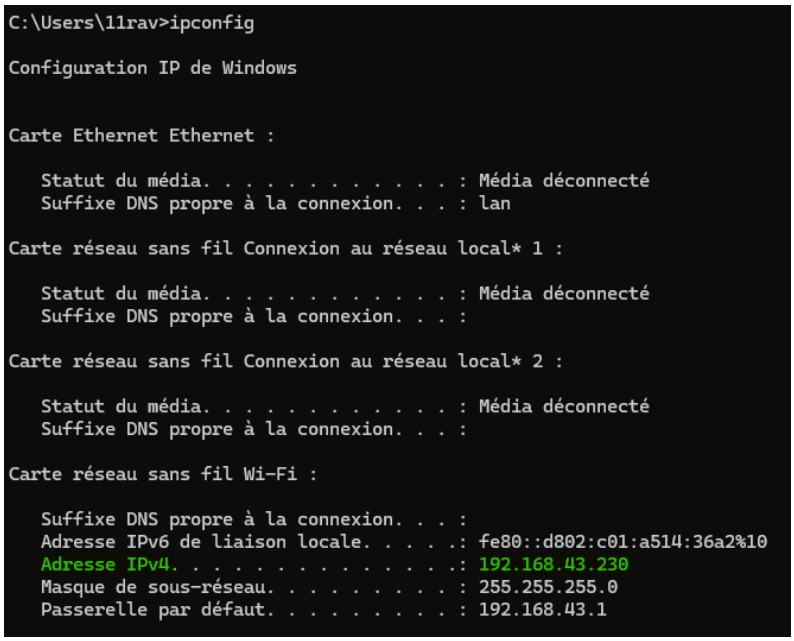


Figure 15 : Adresse IPv4 de la machine serveur avec ipconfig

7.5.2. Tester la connectivité avec ping

Depuis la machine client, `ping <adresse du serveur>` vérifie que les deux machines se voient sur le réseau. Si les réponses (« Réponse de… ») s'affichent, le réseau est correct ; sinon il faut revoir le réseau (même Wi-Fi ?) ou le pare-feu. Certains réseaux d'établissement isolent les appareils entre eux : le partage de connexion d'un téléphone est alors une solution simple.

7.5.3. Pare-feu Windows

Au premier lancement du serveur, Windows affiche une fenêtre « Pare-feu Windows Defender a bloqué certaines fonctionnalités de cette application ». Il faut **cocher « Réseaux privés »** puis cliquer sur **Autoriser l'accès**, sans quoi la deuxième machine ne pourra pas se connecter. Si cette fenêtre n'apparaît pas, on crée une règle manuellement : Pare-feu Windows Defender > Paramètres avancés > Règles de trafic entrant > Nouvelle règle > Port > TCP, port **5000** (et **5001** pour la démonstration en clair) > Autoriser la connexion.

La figure 16 montre la capture d'écran intitulée « Autorisation du pare-feu Windows pour le serveur ».

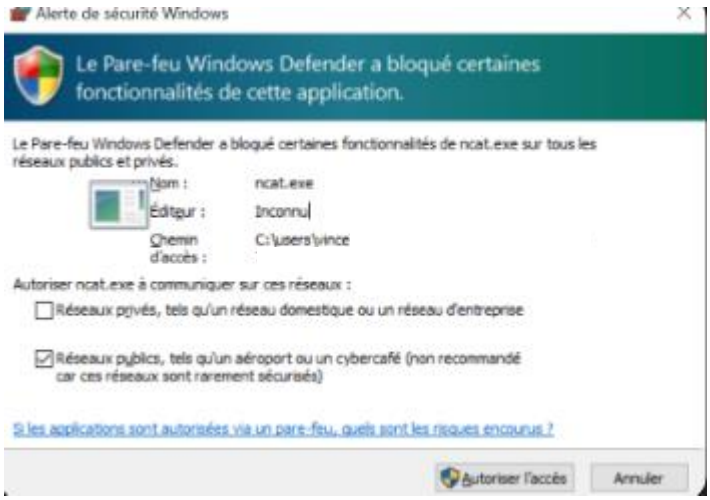


Figure 16 : Autorisation du pare-feu Windows pour le serveur

7.6. Conception du protocole de communication

7.6.1. Le protocole de communication

Le protocole se déroule en trois phases : le **handshake** (établissement des clés), l'**authentification** de l'utilisateur, puis l'**échange de messages**. Un schéma résume l'ensemble des échanges.

La figure 17 montre le schéma intitulé « Diagramme de séquence du protocole (handshake, authentification, messages) ».

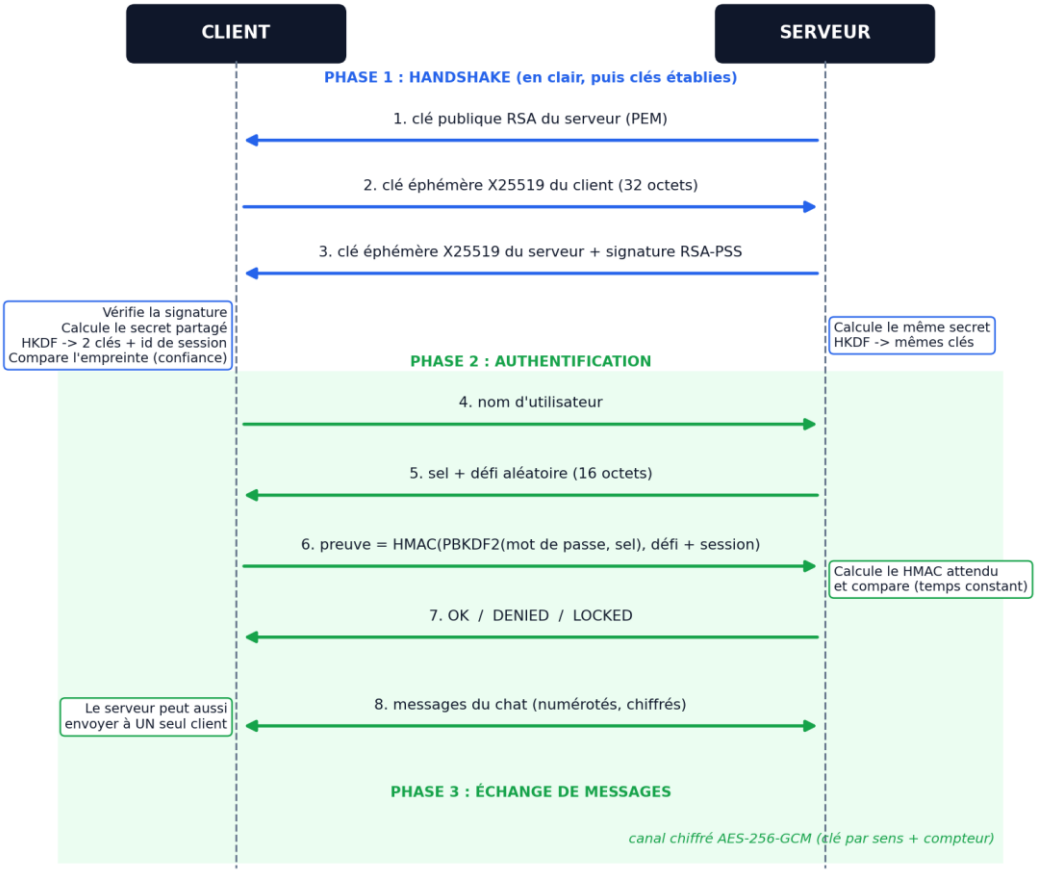


Figure 17 : Diagramme de séquence du protocole (handshake, authentification, messages)

7.6.1.1. Phase 1 : handshake

1. Le serveur envoie sa **clé publique RSA** (format PEM). Elle sert à vérifier sa signature et à calculer son empreinte.
2. Le client génère une paire de clés **X25519 éphémère** et envoie sa clé publique (32 octets).
3. Le serveur génère à son tour une paire éphémère, **signe** (RSA-PSS) la concaténation des deux clés éphémères avec sa clé privée RSA et envoie sa clé publique éphémère suivie de la signature.
4. Le client **vérifie la signature** avec la clé publique reçue à l'étape 1. Les deux côtés calculent le **secret partagé** X25519 puis dérivent les clés avec HKDF.

5. Le client **compare l'empreinte** du serveur à celle mémorisée (ou demande confirmation à la première connexion). Cette vérification a lieu **avant** que le moindre identifiant ne soit envoyé.

La signature porte sur les deux clés éphémères : un attaquant ne peut pas remplacer la clé éphémère du serveur par la sienne sans invalider la signature, et il ne peut pas signer sans la clé privée RSA du serveur. S'il remplace aussi la clé publique RSA par la sienne, l'empreinte ne correspond plus et le client le détecte.

La figure 18 montre le schéma intitulé « Dérivation des clés de session à partir du secret X25519 ».

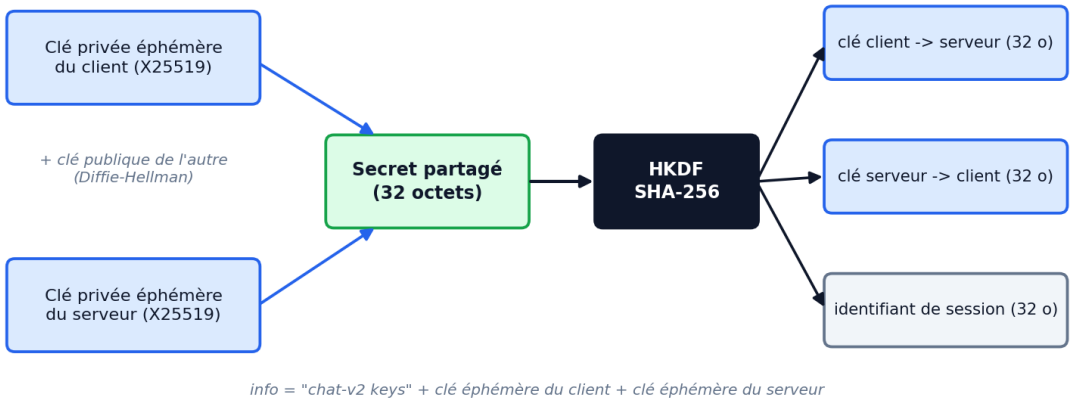


Figure 18 : Dérivation des clés de session à partir du secret X25519

7.6.1.2. Phase 2 : authentification

1. Le client envoie son nom d'utilisateur, déjà **chiffré** par le canal.
2. Le serveur répond avec le **sel** du compte et un **défi** aléatoire de 16 octets. Si le compte n'existe pas, il envoie un faux sel stable : la réponse a la même forme.
3. Le client calcule la clé PBKDF2 du mot de passe avec ce sel, puis envoie la **preuve** `HMAC(clé, « auth » + défi + identifiant de session)`.
4. Le serveur calcule la preuve attendue à partir du hash stocké et compare en temps constant. Il répond `OK`, `DENIED` ou `LOCKED`. Après chaque échec, une pause d'une seconde est imposée ; après trois échecs sur la connexion, ou six échecs en une minute depuis la même adresse IP, l'accès est refusé.

7.6.1.3. Phase 3 : échange de messages

Chaque message est chiffré avec AES-256-GCM avec la clé du sens correspondant, un nonce aléatoire et le numéro du message comme donnée authentifiée. Le serveur peut envoyer un message à tous les clients connectés ou à un seul ; dans ce dernier cas, le message est chiffré avec la clé de session de ce client uniquement.

7.6.2. Format des messages sur le réseau

Sur le fil, chaque message est constitué de la longueur sur 4 octets, suivie du nonce de 12 octets, du texte chiffré et du tag d'authentification de 16 octets. Le texte chiffré a la même taille que le message en clair ; le tag est ajouté à la fin par GCM.

La figure 19 montre le schéma intitulé « Structure d'un message chiffré sur le réseau ».

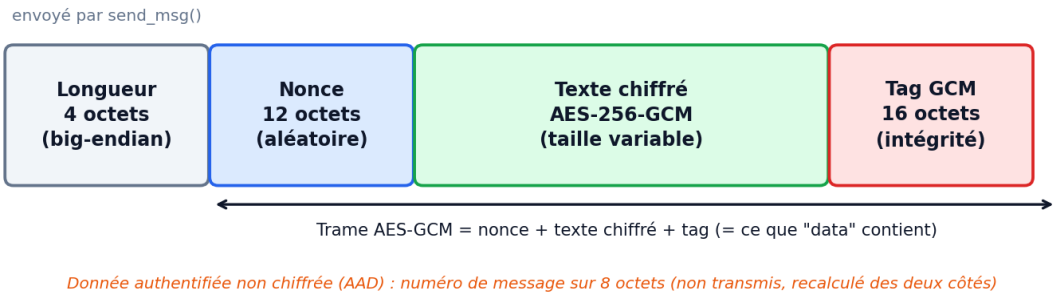


Figure 19 : Structure d'un message chiffré sur le réseau

Le tableau 25 détaille les champs d'un message chiffré transmis sur le réseau.

Tableau 25 : Format d'un message chiffré

Champ	Taille	Contenu
Longueur	4 octets	Taille de ce qui suit, en octets (gros-boutiste). Limitée à 64 Ko.
Nonce	12 octets	Valeur aléatoire, différente pour chaque message ; transmise en clair.
Texte chiffré	Variable	Message UTF-8 chiffré par AES-256-GCM.
Tag GCM	16 octets	Authentifie le texte chiffré, le nonce et le numéro de message.
Numéro de message (AAD)	8 octets	Compteur, non transmis : recalculé par chaque côté.

Une conséquence visible dans Wireshark : un même message envoyé deux fois produit **deux textes chiffrés différents** (nonce et numéro de message différents), et la taille d'un message chiffré est celle du message en clair plus 28 octets (12 de nonce et 16 de tag) plus les 4 octets de longueur.

7.6.3. Comptes, empreintes et fichiers

- `server_key.pem` : clé privée RSA-2048 du serveur (format PKCS #8). Créée au premier lancement, puis réutilisée, afin que l'empreinte du serveur reste stable.
- `users.json` : un objet par utilisateur, avec un **sel** aléatoire de 16 octets et la clé dérivée par PBKDF2 (`hash`). Aucun mot de passe en clair.
- `known_<IP>_<port>.txt` : sur la machine cliente, l'empreinte SHA-256 de la clé publique du serveur, mémorisée à la première connexion. Le nom contient l'adresse et le port pour distinguer plusieurs serveurs.

> **Sécurité des fichiers :** `server_key.pem` et `users.json` ne doivent exister que sur la machine du serveur. C'est pourquoi l'archive du projet ne les contient pas et pourquoi il ne faut jamais les copier sur la machine cliente.

7.6.4. Gestion de la concurrence

Le serveur crée un thread par connexion (`accept_loop`). Trois points demandent de l'attention. D'abord, la table des clients connectés est protégée par `clients_lock`. Ensuite, les compteurs d'échecs par adresse IP sont protégés par `failures_lock`. Enfin, l'envoi sur un canal est protégé par un verrou **interne** à la classe `Channel` : le chiffrement et l'envoi d'un message doivent former une opération indivisible, sinon deux threads qui envoient en même temps pourraient numéroter leurs messages dans un ordre différent de l'ordre d'envoi, et le récepteur rejetterait le second à tort. Un test automatique (`test_envois_simultanes_gardent_l_ordre`) vérifie ce comportement avec cinq threads.

7.6.5. Choix de conception justifiés

Le tableau 26 récapitule les principaux choix de conception et leur justification.

Tableau 26 : Choix de conception justifiés

Question	Choix	Justification
Chiffrer la clé avec RSA, ou faire un Diffie-Hellman ?	X25519 éphémère signé par RSA	Apporte la confidentialité persistante, que le chiffrement de la clé par RSA ne donne pas (version 1). RSA reste utilisé pour l'identité du serveur.
Pourquoi AES-GCM ?	AEAD	Un seul mécanisme assure confidentialité et intégrité ; c'est le mode recommandé et utilisé par TLS 1.3.
Une ou deux clés de session ?	Deux (HKDF)	Empêche la réflexion d'un message du client vers le client.
Nonce aléatoire ou compteur ?	Aléatoire + compteur en AAD	Le nonce aléatoire évite toute collision de couple (clé, nonce) ; le compteur en AAD apporte l'anti-rejeu.
Envoyer le mot de passe dans le canal chiffré ?	Non : défi/réponse HMAC	Même si le canal est chiffré, un faux serveur accepté par erreur aurait pu récupérer le mot de passe.
Comment authentifier le serveur sans autorité de certification ?	Empreinte + TOFU	Principe de SSH : simple, compréhensible, adapté à un projet sans infrastructure à clés publiques.
Nombre d'itérations PBKDF2	200 000	Compromis entre ralentissement des attaques et temps de connexion acceptable (environ 0,1 à 0,3 s).

7.7. Développement du code

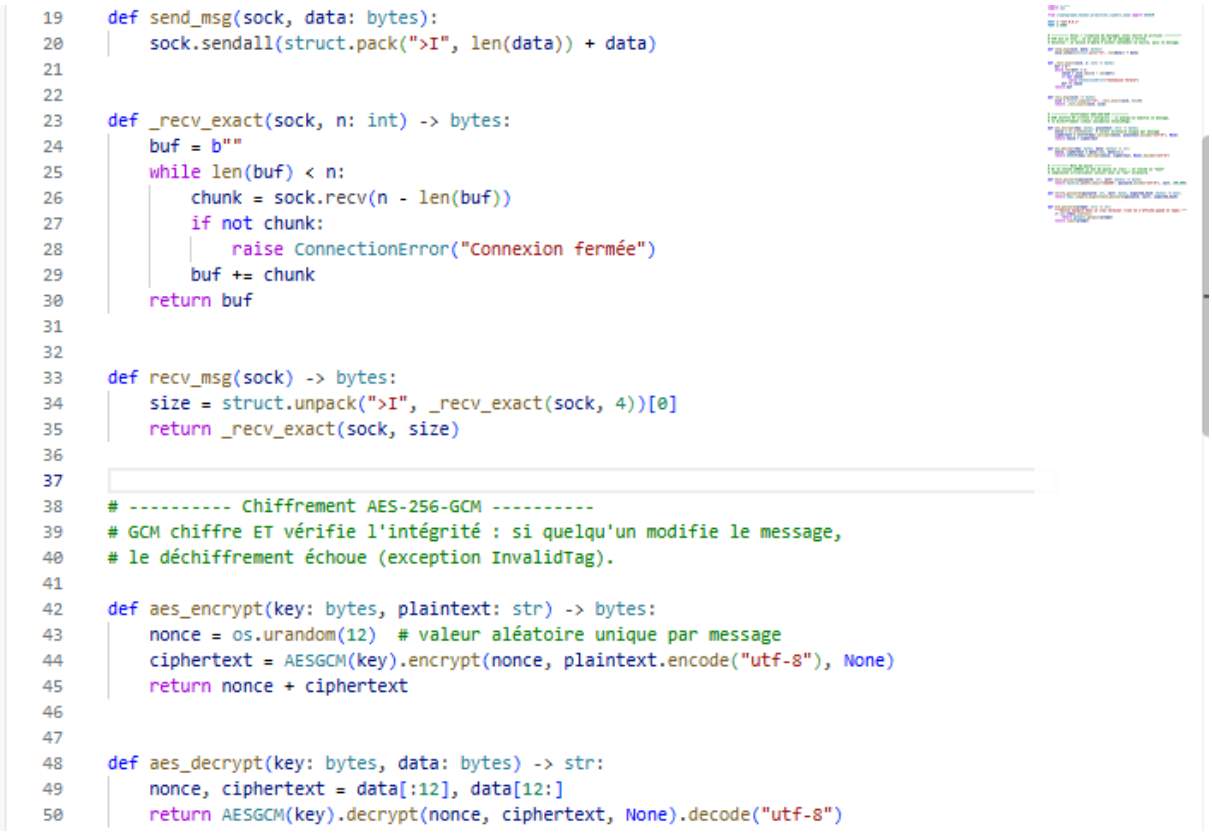
Cette section décrit le code module par module. Pour chaque extrait important, une capture d'écran du code (prise dans VS Code) illustre le passage commenté ; les numéros de lignes indiqués dans les légendes correspondent aux listings de l'annexe B. Le code complet est fourni en annexe.

7.7.1. Le module common.py

Ce module est partagé par le client et le serveur. Il regroupe tout ce qui est cryptographique, de sorte que les deux côtés utilisent exactement les mêmes fonctions.

7.7.1.1. Envoi et réception de messages délimités

La figure 20 montre le code de `common.py` : fonctions `send_msg`, `_recv_exact` et `recv_msg`.



```python
def send_msg(sock, data: bytes):
    sock.sendall(struct.pack(">I", len(data)) + data)


def _recv_exact(sock, n: int) -> bytes:
    buf = b""
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("Connexion fermée")
        buf += chunk
    return buf


def recv_msg(sock) -> bytes:
    size = struct.unpack(">I", _recv_exact(sock, 4))[0]
    return _recv_exact(sock, size)

# ---------- Chiffrement AES-256-GCM ----------
# GCM chiffre ET vérifie l'intégrité : si quelqu'un modifie le message,
# le déchiffrement échoue (exception InvalidTag).

def aes_encrypt(key: bytes, plaintext: str) -> bytes:
    nonce = os.urandom(12)  # valeur aléatoire unique par message
    ciphertext = AESGCM(key).encrypt(nonce, plaintext.encode("utf-8"), None)
    return nonce + ciphertext


def aes_decrypt(key: bytes, data: bytes) -> str:
    nonce, ciphertext = data[:12], data[12:]
    return AESGCM(key).decrypt(nonce, ciphertext, None).decode("utf-8")
```

Figure 20 : `common.py` : fonctions `send_msg`, `_recv_exact` et `recv_msg`

- `send_msg` refuse tout message de plus de `MAX_MSG` (64 Ko), puis envoie `struct.pack(">I", len(data)) + data` : 4 octets de longueur suivis des données.
- `_recv_exact` boucle sur `sock.recv` jusqu'à avoir lu exactement `n` octets, car TCP peut livrer les données en plusieurs morceaux. Si la connexion se ferme avant la fin, elle lève `ConnectionError`.
- `recv_msg` lit d'abord les 4 octets de longueur et **refuse** toute taille annoncée supérieure à `MAX_MSG`, ce qui empêche un attaquant de faire allouer des gigaoctets au serveur.

7.7.1.2. Chiffrement AES-GCM

La figure 21 montre le code de `common.py` : fonctions `aes_encrypt` et `aes_decrypt` .



```
48  def aes_decrypt(key: bytes, data: bytes) -> str:
49      nonce, ciphertext = data[:12], data[12:]
50      return AESGCM(key).decrypt(nonce, ciphertext, None).decode("utf-8")
51
52
53  # ---------- Mots de passe ----------
54  # On ne stocke JAMAIS un mot de passe en clair : on stocke un "hash"
55  # (empreinte irréversible) calculé avec un "sel" aléatoire.
56
57  def hash_password(password: str, salt: bytes) -> bytes:
58      return hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, 200_000)
59
60
61  def verify_password(password: str, salt: bytes, expected_hash: bytes) -> bool:
62      return hmac.compare_digest(hash_password(password, salt), expected_hash)
63
64
65  def ask_password(prompt: str) -> str:
66      """Saisie masquée dans un vrai terminal (rien ne s'affiche quand on tape)."""
67      if sys.stdin.isatty():
68          return getpass.getpass(prompt)
69      return input(prompt)
70
```

Figure 21 : `common.py` : fonctions `aes_encrypt` et `aes_decrypt` (lignes 47 à 60)

- `aes_encrypt` tire un nonce de 12 octets avec `os.urandom`, chiffre avec `AESGCM(key).encrypt(nonce, texte, aad)` et renvoie `nonce + texte chiffré + tag` (le tag est ajouté automatiquement par la bibliothèque).
- `aes_decrypt` sépare le nonce du reste et déchiffre ; si le message, le nonce, l'AAD ou le tag ont été modifiés, la bibliothèque lève `InvalidTag`.
- Le paramètre `aad` (données authentifiées) reçoit le numéro de message depuis la classe `Channel`.

7.7.1.3. Handshake : X25519 éphémère et signature RSA-PSS

La figure 22 montre le code de `common.py` : fonctions du handshake .

```python
def new_ephemeral():
    """Retourne (clé privée jetable, clé publique en 32 octets)."""
    private = x25519.X25519PrivateKey.generate()
    public = private.public_key().public_bytes(serialization.Encoding.Raw,
                                               serialization.PublicFormat.Raw)
    return private, public


def exchange(private, peer_public: bytes) -> bytes:
    """Secret partagé Diffie-Hellman (lève ValueError si la clé reçue est invalide)."""
    if len(peer_public) != 32:
        raise ValueError("clé éphémère invalide")
    return private.exchange(x25519.X25519PublicKey.from_public_bytes(peer_public))


def _transcript(client_eph: bytes, server_eph: bytes) -> bytes:
    return b"chat-v2 handshake|" + client_eph + server_eph


def sign_handshake(rsa_private_key, client_eph: bytes, server_eph: bytes) -> bytes:
    return rsa_private_key.sign(_transcript(client_eph, server_eph), _PSS, hashes.SHA256())


def verify_handshake(server_pem: bytes, client_eph: bytes, server_eph: bytes, signature: bytes):
    """Lève cryptography.exceptions.InvalidSignature si la signature est fausse."""
    serialization.load_pem_public_key(server_pem).verify(
        signature, _transcript(client_eph, server_eph), _PSS, hashes.SHA256())
```

Figure 22 : `common.py` : fonctions du handshake

- `new_ephemeral` crée une clé privée X25519 jetable et renvoie aussi sa clé publique brute de 32 octets.
- `exchange` calcule le secret partagé Diffie-Hellman ; elle vérifie que la clé reçue fait bien 32 octets, et la bibliothèque rejette les valeurs dangereuses (`ValueError`).
- `sign_handshake` signe, avec la clé privée RSA, la chaîne `chat-v2 handshake|` suivie des deux clés éphémères. Le schéma utilisé est RSA-PSS avec SHA-256.
- `verify_handshake` fait l'opération inverse côté client avec la clé publique du serveur et lève `InvalidSignature` si la signature est fausse.

7.7.1.4. Le canal chiffré : la classe Channel

La figure 23 montre le code de `common.py` : classe `Channel`.

```python
class Channel:
    """Canal AES-GCM d'une connexion.

    - Deux clés différentes (client->serveur et serveur->client) : un message
      du client ne peut pas être renvoyé au client comme s'il venait du serveur.
    - Un compteur dans les données authentifiées : un message rejoué, supprimé
      ou réordonné fait échouer le déchiffrement (InvalidTag).
    - Le verrou garantit que l'ordre des compteurs = l'ordre d'envoi, même si
      plusieurs threads envoient en même temps.
    """

    def __init__(self, send_key: bytes, recv_key: bytes, session_id: bytes):
        self._send_key = send_key
        self._recv_key = recv_key
        self.session_id = session_id      # identifiant unique de CETTE session
        self._tx = 0
        self._rx = 0
        self._lock = threading.Lock()

    def send(self, sock, text: str) -> bytes:
        """Chiffre et envoie ; retourne les octets chiffrés (pour l'affichage)."""
        with self._lock:
            data = aes_encrypt(self._send_key, text, struct.pack(">Q", self._tx))
            send_msg(sock, data)
            self._tx += 1
        return data

    def recv(self, sock):
        """Reçoit et déchiffre ; retourne (texte, octets chiffrés)."""
        data = recv_msg(sock)
        text = aes_decrypt(self._recv_key, data, struct.pack(">Q", self._rx))
        self._rx += 1
        return text, data
```

Figure 23 : `common.py` : classe `Channel`

La classe `Channel` représente une connexion chiffrée. Elle garde deux clés (envoi et réception), deux compteurs, un identifiant de session et un verrou. Dans `send`, le numéro du message (`self._tx`) est converti en 8 octets avec `struct.pack(">Q", …)` et passé en AAD ; le compteur n'est incrémenté qu'après l'envoi, le tout sous verrou. Dans `recv`, le compteur attendu (`self._rx`) est utilisé pour déchiffrer : un message rejoué ou hors de l'ordre échoue donc avec `InvalidTag`.

La figure 24 montre le code de `common.py` : fonction `derive_channel`.

```python
def derive_channel(shared: bytes, client_eph: bytes, server_eph: bytes, role: str) -> Channel:
    """Dérive les clés de session avec HKDF. role = "client" ou "server"."""
    okm = HKDF(algorithm=hashes.SHA256(), length=96, salt=None,
               info=b"chat-v2 keys|" + client_eph + server_eph).derive(shared)
    c2s, s2c, session_id = okm[:32], okm[32:64], okm[64:96]
    if role == "client":
        return Channel(send_key=c2s, recv_key=s2c, session_id=session_id)
    return Channel(send_key=s2c, recv_key=c2s, session_id=session_id)
```

Figure 24 : `common.py` : fonction `derive_channel`

`derive_channel` utilise HKDF-SHA-256 pour produire 96 octets à partir du secret partagé, en liant le résultat aux deux clés éphémères (paramètre `info`). Les 32 premiers octets

forment la clé client vers serveur, les 32 octets centraux la clé serveur vers client, et les 32 derniers l'identifiant de session. Selon son rôle, chaque côté inverse l'usage des deux clés.

7.7.1.5. Mots de passe et preuve d'authentification

La figure 25 montre le code de `common.py` : `hash_password` et `auth_proof`.

```python
def hash_password(password: str, salt: bytes) -> bytes:
    return hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, 200_000)


def verify_password(password: str, salt: bytes, expected_hash: bytes) -> bool:
    return hmac.compare_digest(hash_password(password, salt), expected_hash)


def auth_proof(password_hash: bytes, challenge: bytes, session_id: bytes) -> bytes:
    """Preuve de connaissance du mot de passe (défi/réponse HMAC).

    Le défi est aléatoire (change à chaque tentative) et la preuve est liée à la
    session : elle ne sert à rien si un attaquant la rejoue ailleurs.
    """
    return hmac.new(password_hash, b"auth|" + challenge + session_id, "sha256").digest()
```

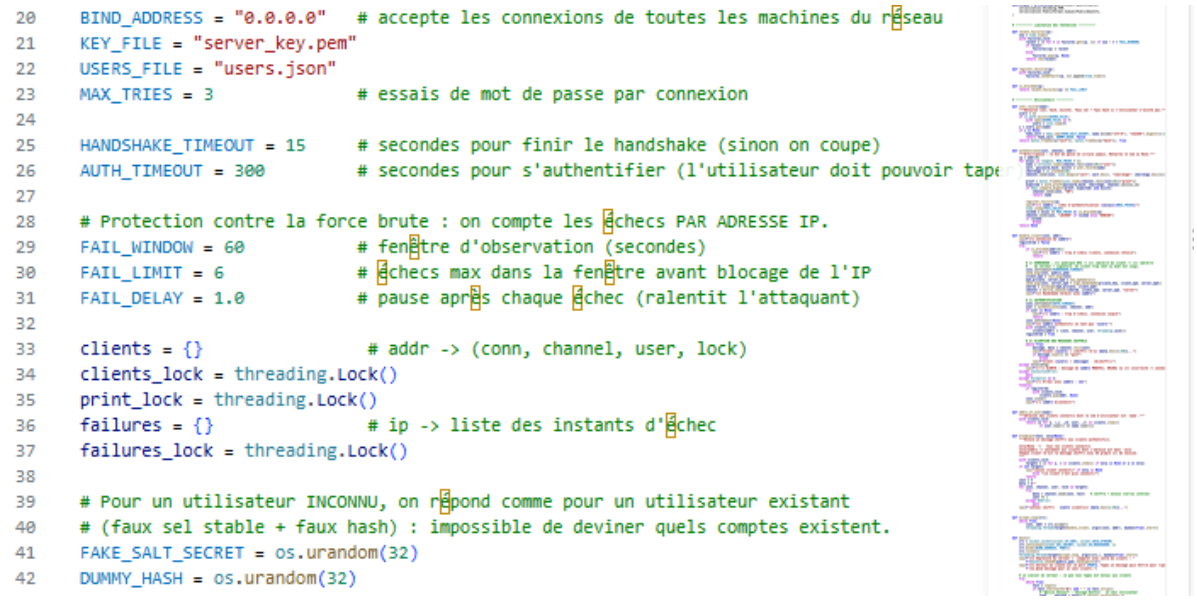
Figure 25 : `common.py` : `hash_password` et `auth_proof`

- `hash_password` applique PBKDF2-HMAC-SHA-256 avec 200 000 itérations et le sel du compte.
- `auth_proof` calcule `HMAC-SHA-256(hash, "auth|" + défi + identifiant de session)`. La preuve dépend du mot de passe, du défi du moment et de la session en cours.

7.7.2. Le serveur : server.py

7.7.2.1. Paramètres de sécurité et clé du serveur

La figure 26 montre le code de `server.py` : constantes de configuration.



```
20  BIND_ADDRESS = "0.0.0.0"   # accepte les connexions de toutes les machines du réseau
21  KEY_FILE = "server_key.pem"
22  USERS_FILE = "users.json"
23  MAX_TRIES = 3              # essais de mot de passe par connexion
24
25  HANDSHAKE_TIMEOUT = 15     # secondes pour finir le handshake (sinon on coupe)
26  AUTH_TIMEOUT = 300         # secondes pour s'authentifier (l'utilisateur doit pouvoir tape
27
28  # Protection contre la force brute : on compte les échecs PAR ADRESSE IP.
29  FAIL_WINDOW = 60           # fenêtre d'observation (secondes)
30  FAIL_LIMIT = 6             # échecs max dans la fenêtre avant blocage de l'IP
31  FAIL_DELAY = 1.0           # pause après chaque échec (ralentit l'attaquant)
32
33  clients = {}               # addr -> (conn, channel, user, lock)
34  clients_lock = threading.Lock()
35  print_lock = threading.Lock()
36  failures = {}              # ip -> liste des instants d'échec
37  failures_lock = threading.Lock()
38
39  # Pour un utilisateur INCONNU, on répond comme pour un utilisateur existant
40  # (faux sel stable + faux hash) : impossible de deviner quels comptes existent.
41  FAKE_SALT_SECRET = os.urandom(32)
42  DUMMY_HASH = os.urandom(32)
```

Figure 26 : `server.py` : constantes de configuration

La figure 27 montre le code de `server.py` : fonction `load_or_create_key`.

```
52  def load_or_create_key():
53      """La clé RSA est sauvegardée pour que l'empreinte du serveur reste la même."""
54      if os.path.exists(KEY_FILE):
55          with open(KEY_FILE, "rb") as f:
56              return serialization.load_pem_private_key(f.read(), password=None)
57      key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
58      with open(KEY_FILE, "wb") as f:
59          f.write(key.private_bytes(
60              serialization.Encoding.PEM,
61              serialization.PrivateFormat.PKCS8,
62              serialization.NoEncryption(),
63          ))
64      return key
65
66
67  private_key = load_or_create_key()
68  public_pem = privat
69      serialization.
70      serialization.PublicFormat.SubjectPublicKeyInfo,
71  )
```

(function) PublicFormat: Any

Figure 27 : `server.py` : fonction `load_or_create_key`

Le serveur écoute sur `0.0.0.0` (toutes les interfaces). Les constantes regroupent les paramètres de sécurité : trois essais de mot de passe par connexion, 15 secondes pour le

handshake, 5 minutes pour s'authentifier, et au plus 6 échecs en 60 secondes par adresse IP. `load_or_create_key` charge `server_key.pem` s'il existe, sinon génère une clé RSA de 2048 bits et l'enregistre, afin que l'empreinte du serveur reste la même d'un lancement à l'autre.

7.7.2.2. Limitation des tentatives et comptes

La figure 28 montre le code de `server.py` : limitation des échecs et recherche d'un utilisateur.

```
76  def recent_failures(ip):
77      now = time.time()
78      with failures_lock:
79          recent = [t for t in failures.get(ip, []) if now - t < FAIL_WINDOW]
80          if recent:
81              failures[ip] = recent
82          else:
83              failures.pop(ip, None)
84          return len(recent)
85
86
87  def register_failure(ip):
88      with failures_lock:
89          failures.setdefault(ip, []).append(time.time())
90
91
92  def is_blocked(ip):
93      return recent_failures(ip) >= FAIL_LIMIT
94
95
96  # ---------- Utilisateurs ----------
97
98  def user_record(name):
99      """Retourne (sel, hash, existe). Faux sel + faux hash si l'utilisateur n'existe pas."""
100     users = {}
101     if os.path.exists(USERS_FILE):
102         with open(USERS_FILE) as f:
103             users = json.load(f)
104     u = users.get(name)
105     if u is None:
106         fake_salt = hmac.new(FAKE_SALT_SECRET, name.encode("utf-8"), "sha256").digest()[:16]
107         return fake_salt, DUMMY_HASH, False
108     return bytes.fromhex(u["salt"]), bytes.fromhex(u["hash"]), True
109
```

Figure 28 : `server.py` : limitation des échecs et recherche d'un utilisateur

- `register_failure` mémorise l'instant de chaque échec par adresse IP ; `recent_failures` ne compte que ceux des 60 dernières secondes ; `is_blocked` renvoie vrai à partir de 6 échecs. Le blocage expire donc tout seul.
- `user_record` renvoie le sel et le hash du compte. Pour un compte **inconnu**, elle renvoie un **faux sel** calculé par `HMAC(secret du serveur, nom)` (donc identique à chaque demande) et un faux hash : un attaquant ne peut pas deviner quels comptes existent.

7.7.2.3. Authentification défi/réponse

La figure 29 montre le code de `server.py` : fonction `authenticate`.

```
111  def authenticate(conn, channel, addr):
112      """Défi/réponse : le mot de passe ne circule jamais. Retourne le nom ou None."""
113      ip = addr[0]
114      for essai in range(1, MAX_TRIES + 1):
115          name = str(json.loads(channel.recv(conn)[0])["user"])
116          salt, password_hash, exists = user_record(name)
117          challenge = os.urandom(16)
118          channel.send(conn, json.dumps({"salt": salt.hex(), "challenge": challenge.hex()}))
119
120          proof = bytes.fromhex(json.loads(channel.recv(conn)[0])["proof"])
121          expected = auth_proof(password_hash, challenge, channel.session_id)
122          if hmac.compare_digest(proof, expected) and exists:
123              channel.send(conn, "OK")
124              return name
125
126          register_failure(ip)
127          say(f"[!] {addr} : échec d'authentification ({essai}/{MAX_TRIES})")
128          time.sleep(FAIL_DELAY)
129          locked = essai == MAX_TRIES or is_blocked(ip)
130          channel.send(conn, "LOCKED" if locked else "DENIED")
131          if locked:
132              break
133      return None
134
```

Figure 29 : `server.py` : fonction `authenticate`

Pour chaque tentative, le serveur lit le nom d'utilisateur, envoie le sel et un défi de 16 octets tirés avec `os.urandom`, lit la preuve, calcule la preuve attendue avec `auth_proof` et compare avec `hmac.compare_digest` (comparaison en temps constant). En cas d'échec, il enregistre l'échec pour l'adresse IP, attend une seconde (`FAIL_DELAY`) et répond `DENIED` ou `LOCKED` (dernier essai ou adresse bloquée).

7.7.2.4. Gestion d'un client

La figure 30 montre le code de `server.py` : fonction `handle_client`.



```
136  def handle_client(conn, addr):
137      say(f"[+] Connexion de {addr}")
138      registered = False
139      try:
140          if is_blocked(addr[0]):
141              say(f"[!] {addr} : trop d'échecs récents, connexion refusée")
142              return
143
144          # 1) HANDSHAKE : clé publique RSA -> clé éphémère du client -> clé éphémère
145          #    du serveur + signature. Un client trop lent ou muet est coupé.
146          conn.settimeout(HANDSHAKE_TIMEOUT)
147          send_msg(conn, public_pem)
148          client_eph = recv_msg(conn)
149          eph_private, server_eph = new_ephemeral()
150          send_msg(conn, server_eph + sign_handshake(private_key, client_eph, server_eph))
151          shared = exchange(eph_private, client_eph)
152          channel = derive_channel(shared, client_eph, server_eph, "server")
153          say(f"[*] Handshake terminé avec {addr}")
154
155          # 2) AUTHENTIFICATION
156          conn.settimeout(AUTH_TIMEOUT)
157          user = authenticate(conn, channel, addr)
158          if user is None:
159              say(f"[!] {addr} : trop d'échecs, connexion coupée")
160              return
161          conn.settimeout(None)
162          say(f"[*] {addr} authentifié en tant que '{user}'")
163          with clients_lock:
164              clients[addr] = (conn, channel, user, threading.Lock())
165          registered = True
166
167          # 3) RÉCEPTION DES MESSAGES CHIFFRÉS
168          while True:
169              message, data = channel.recv(conn)
170              say(f"Client ({user}) > [chiffré reçu] {data.hex()[:48]}...")
171              if message.lower() == "quit":
172                  break
173              say(f"Client ({user}) > {message}   (déchiffré)")
```

Figure 30 : `server.py` : fonction `handle_client`

`handle_client` est exécutée dans un thread par connexion. Elle refuse d'abord les adresses bloquées, puis exécute le handshake sous un délai de 15 secondes : envoi de la clé publique RSA, réception de la clé éphémère du client, génération de la clé éphémère du serveur, envoi de celle-ci avec la signature, calcul du secret partagé et création du `Channel`. Elle authentifie ensuite le client (délai de 5 minutes), l'enregistre dans le dictionnaire `clients`, puis boucle sur `channel.recv`. Les exceptions sont traitées séparément : `InvalidTag` produit l'alerte « message MODIFIÉ, REJOUÉ ou clé incorrecte » et coupe la connexion ; `ConnectionError` est une déconnexion normale. Le bloc `finally` retire toujours le client de la liste et ferme le socket.

7.7.2.5. Envoi à tous ou à un seul client

La figure 31 montre le code de `server.py` : fonctions `addrs_of_user` et `broadcast`.

```
188  def addrs_of_user(name):
189      """Adresses des clients connectés dont le nom d'utilisateur est `name`."""
190      with clients_lock:
191          return {a for a, (_c, _ch, user, _l) in clients.items()
192                  if user.lower() == name.lower()}
193
194
195  def broadcast(text, only=None):
196      """Envoie un message chiffré aux clients authentifiés.
197
198      only=None  -> à tous les clients connectés
199      only={addr} -> seulement aux clients dont l'adresse est dans `only`
200      Chaque client reçoit le message chiffré avec SA propre clé de session.
201      """
202      with clients_lock:
203          targets = [v for a, v in clients.items() if only is None or a in only]
204      if not targets:
205          say("(aucun client connecté)" if only is None
206              else "(ce client n'est plus connecté)")
207          return
208      sent = 0
209      data = b""
210      for conn, channel, user, lock in targets:
211          try:
212              data = channel.send(conn, text)   # chiffre + envoie (verrou interne)
213              sent += 1
214          except OSError:
215              pass
216      say(f"[envoyé chiffré à {sent} client(s)] {data.hex()[:48]}...")
```

Figure 31 : `server.py` : fonctions `addrs_of_user` et `broadcast`

`broadcast (text, only=None)` envoie le message à tous les clients authentifiés ; si `only` contient un ensemble d'adresses, seuls les clients correspondants le reçoivent. Chaque message est chiffré avec le canal de **chaque destinataire**, donc avec une clé différente. En version terminal, la saisie `@alice bonjour` appelle `addrs_of_user` puis `broadcast` avec l'ensemble d'adresses d'Alice ; l'interface graphique fait de même à partir de la ligne sélectionnée dans la liste.

7.7.3. Le client : client.py

La figure 32 montre le code de `client.py` : fonction `verify_server`.

```python
def verify_server(pem: bytes, host: str, port: int) -> bool:
    """Authentification du serveur : on mémorise son empreinte à la 1re connexion
    et on la compare aux suivantes (même principe que SSH).
    Le fichier dépend de l'adresse ET du port : known_<adresse>_<port>.txt"""
    known_file = f"known_{host}_{port}.txt"
    fp = fingerprint(pem)
    print(f"[*] Empreinte du serveur : {fp}")
    if os.path.exists(known_file):
        with open(known_file) as f:
            if f.read().strip() == fp:
                print("[*] Serveur reconnu (empreinte identique)")
                return True
        print("[!!!] ALERTE : l'empreinte du serveur a CHANGÉ ! Possible attaque (homme du milieu).")
        return False
    print("    Comparez cette empreinte avec celle affichée sur le serveur.")
    answer = input("Première connexion. Faire confiance à ce serveur ? (o/n) ")
    if answer.lower().startswith("o"):
        with open(known_file, "w") as f:
            f.write(fp)
        return True
    return False
```

Figure 32 : `client.py` : fonction `verify_server`

`verify_server` calcule l'empreinte SHA-256 de la clé publique reçue et la compare au contenu de `known_<adresse>_<port>.txt`. Si le fichier n'existe pas, elle affiche l'empreinte et demande à l'utilisateur de faire confiance ou non ; si l'empreinte a changé, elle affiche l'alerte « possible attaque de l'homme du milieu » et refuse la connexion.

La figure 33 montre le code de `client.py` : fonction `handshake`.

```python
def handshake(sock, host: str, port: int):
    """Retourne le canal chiffré, ou None si le serveur n'est pas de confiance."""
    pem = recv_msg(sock)

    # Échange de clés éphémères X25519 ; le serveur signe avec sa clé RSA.
    eph_private, client_eph = new_ephemeral()
    send_msg(sock, client_eph)
    reply = recv_msg(sock)
    server_eph, signature = reply[:32], reply[32:]
    verify_handshake(pem, client_eph, server_eph, signature)   # InvalidSignature si faux
    channel = derive_channel(exchange(eph_private, server_eph), client_eph, server_eph, "client")

    # On vérifie QUI est le serveur avant d'envoyer le moindre identifiant.
    if not verify_server(pem, host, port):
        return None
    return channel
```

Figure 33 : `client.py` : fonction `handshake`

Le client reçoit la clé publique RSA, génère sa paire éphémère, envoie sa clé publique, reçoit la clé éphémère du serveur accompagnée de la signature, **vérifie la signature**, dérive le canal, et seulement ensuite vérifie l'empreinte. L'ordre est voulu : aucun identifiant n'est envoyé avant que le serveur soit reconnu.

La figure 34 montre le code de `client.py` : fonction `login` .

```python
def login(sock, channel) -> bool:
    """Le mot de passe n'est jamais envoyé : on répond à un défi aléatoire du serveur."""
    for _ in range(3):
        user = input("Utilisateur : ")
        password = ask_password("Mot de passe : ")
        channel.send(sock, json.dumps({"user": user}))
        info = json.loads(channel.recv(sock)[0])          # sel + défi
        key = hash_password(password, bytes.fromhex(info["salt"]))
        proof = auth_proof(key, bytes.fromhex(info["challenge"]), channel.session_id)
        channel.send(sock, json.dumps({"proof": proof.hex()}))
        reply = channel.recv(sock)[0]
        if reply == "OK":
            print("[*] Authentification réussie\n")
            return True
        if reply == "LOCKED":
            print("[!] Trop d'échecs : accès refusé.")
            return False
        print("[!] Identifiants incorrects, réessayez.")
    return False
```

Figure 34 : `client.py` : fonction `login`

`login` demande le nom et le mot de passe (saisie masquée), envoie le nom, reçoit le sel et le défi, calcule la clé PBKDF2 puis la preuve HMAC, et l'envoie. Trois essais sont permis ; le message `LOCKED` met fin à la tentative. En parallèle de la boucle d'envoi, la fonction `receiver` s'exécute dans un thread séparé pour afficher les messages du serveur dès leur arrivée.

7.7.4. Le module chat_core.py

La figure 35 montre le code de `chat_core.py` : classe `ClientCore`.

```python
class ClientCore:
    """Une connexion client : connect -> handshake -> (confiance) -> login -> send/receive."""

    def __init__(self):
        self.sock = None
        self.channel = None

    def connect(self, host: str, port: int, timeout: float = 8) -> bytes:
        """Ouvre la connexion et retourne la clé publique PEM du serveur."""
        self.sock = socket.create_connection((host, port), timeout=timeout)
        return recv_msg(self.sock)

    def handshake(self, pem: bytes) -> None:
        """Échange de clés éphémères X25519 ; vérifie la signature RSA du serveur.

        Lève InvalidSignature si la signature est fausse. L'empreinte du
        serveur doit ensuite être contrôlée (trust_status) AVANT le login.
        """
        eph_private, client_eph = new_ephemeral()
        send_msg(self.sock, client_eph)
        reply = recv_msg(self.sock)
        server_eph, signature = reply[:32], reply[32:]
        verify_handshake(pem, client_eph, server_eph, signature)
        self.channel = derive_channel(exchange(eph_private, server_eph),
                                      client_eph, server_eph, "client")

    def login(self, user: str, password: str) -> str:
        """Défi/réponse (le mot de passe n'est jamais envoyé).
        Retourne la réponse du serveur : "OK", "DENIED" ou "LOCKED"."""
        self.channel.send(self.sock, json.dumps({"user": user}))
        info = json.loads(self.channel.recv(self.sock)[0])
        key = hash_password(password, bytes.fromhex(info["salt"]))
        proof = auth_proof(key, bytes.fromhex(info["challenge"]), self.channel.session_id)
        self.channel.send(self.sock, json.dumps({"proof": proof.hex()}))
        return self.channel.recv(self.sock)[0]

    def set_timeout(self, seconds) -> None:
```

Figure 35 : `chat_core.py` : classe `ClientCore`

`ClientCore` reprend les étapes de `client.py` sous forme de méthodes indépendantes : `connect`, `handshake`, `login`, `send`, `receive`, `close`. Contrairement à `client.py`, elle ne contient aucune saisie au clavier (`input`) : l'interface graphique appelle chaque étape au bon moment et affiche elle-même le dialogue de confiance. La logique cryptographique, elle, reste celle de `common.py`.

7.8. Interfaces graphiques

7.8.1. Objectifs et choix

Les versions en terminal suffisent pour valider le protocole, mais une interface graphique rend la démonstration plus claire, en particulier entre deux machines : on voit les clients connectés, on choisit le destinataire d'un message, et on peut afficher côte à côte le message en clair et sa forme chiffrée. **Tkinter** est retenu car il est fourni avec Python et ne demande aucune installation supplémentaire. L'apparence est soignée grâce au thème « clam »

de ttk, à une palette cohérente (bleu nuit, bleu d'accentuation, vert pour ce qui est sécurisé, rouge pour les alertes) et à une zone de discussion en bulles.

7.8.2. Principe : un thread réseau et une file d'événements

Une interface graphique ne doit jamais attendre le réseau : si la fenêtre exécute un `recv` bloquant, elle se fige. De plus, Tkinter n'est pas sûr pour les accès depuis plusieurs threads. La règle appliquée est donc simple : **le thread principal est le seul à toucher aux widgets**. Les opérations bloquantes (connexion, handshake, attente des messages) s'exécutent dans un thread réseau qui dépose des événements dans une `queue.Queue` ; le thread principal interroge cette file toutes les 100 ms avec `root.after` et met à jour l'affichage.

La figure 36 montre le schéma intitulé « Échanges entre le thread réseau et le thread principal de Tkinter ».

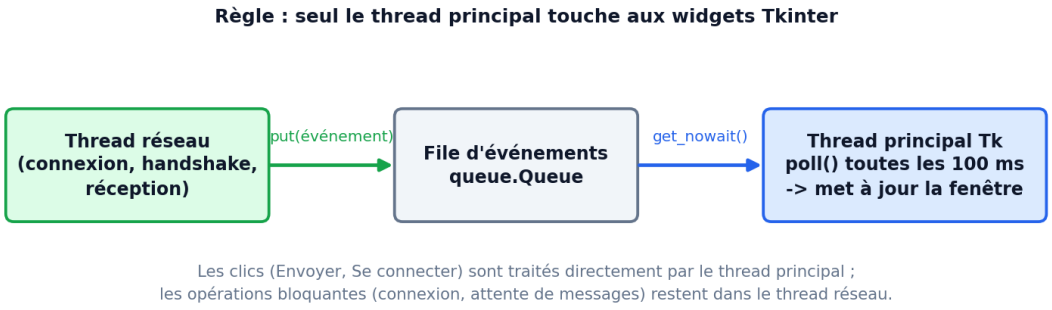


Figure 36 : Échanges entre le thread réseau et le thread principal de Tkinter

La figure 37 montre le code de `gui_client.py` : fonction `network_thread`.



```
113  def main():
114      host, port = parse_args()
115      try:
116          with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
117              sock.settimeout(15)
118              sock.connect((host, port))
119              print(f"[*] Connecté à {host}:{port}")
120
121              # 1) HANDSHAKE
122              channel = handshake(sock, host, port)
123              if channel is None:
124                  print("Connexion annulée.")
125                  return
126              print("[*] Handshake terminé : canal chiffré établi (X25519 + AES-256-GCM)")
127
128              # 2) AUTHENTIFICATION
129              sock.settimeout(300)
130              if not login(sock, channel):
131                  return
132              sock.settimeout(None)
133
134              # 3) DIALOGUE : un thread écoute le serveur, la boucle ci-dessous envoie
135              closing = threading.Event()
136              threading.Thread(target=receiver, args=(sock, channel, closing), daemon=True).start()
137              print("Tapez vos messages ('quit' pour quitter).")
138              while True:
139                  try:
140                      message = input("Vous > ")
141                  except EOFError:
142                      message = "quit"
143                  if not message:
144                      continue
145                  channel.send(sock, message)
146                  if message.lower() == "quit":
147                      closing.set()
148                      break
149      except InvalidSignature:
150          print("[!!!] Signature du serveur INVALIDE : le serveur est peut-être usurpé. Connexion coupée.")
```

Figure 37 : `gui_client.py` : fonction `network_thread`

Lorsque l'interface doit poser une question à l'utilisateur depuis le thread réseau (faire confiance à une nouvelle empreinte ?), le thread poste l'événement `ask_trust` avec un objet de synchronisation, puis attend sur un `threading.Event` ; le thread principal affiche la fenêtre de dialogue, enregistre la réponse et libère le thread réseau.

7.8.3. Le client graphique

Le client comporte deux écrans. L'**écran de connexion** demande l'adresse IP du serveur, le port, le nom d'utilisateur et le mot de passe (masqué). Au clic sur « Se connecter », le thread réseau effectue le handshake ; à la première connexion à ce serveur, une **fenêtre de confiance** affiche l'empreinte du serveur en deux lignes lisibles et demande de la comparer avec celle affichée sur le serveur. Après l'authentification, l'**écran de discussion** affiche un bandeau (utilisateur, adresse, empreinte, mention « Chiffré AES-256-GCM »), la zone de messages en bulles et la zone de saisie. Une case permet d'afficher, sous chaque bulle, le début du texte chiffré en hexadécimal, ce qui est très utile pour la démonstration.

La figure 38 montre la capture d'écran intitulée « Client graphique : écran de connexion ».

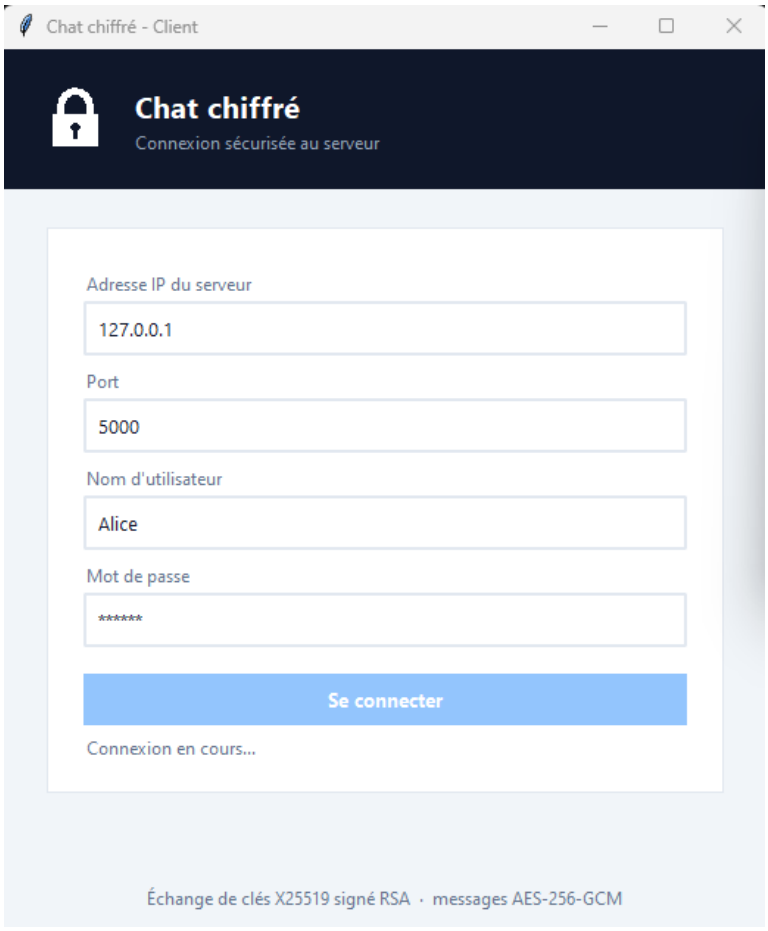


Figure 38 : Client graphique : écran de connexion

La figure 39 montre la capture d'écran intitulée « Client graphique : fenêtre de confiance avec l'empreinte du serveur ».

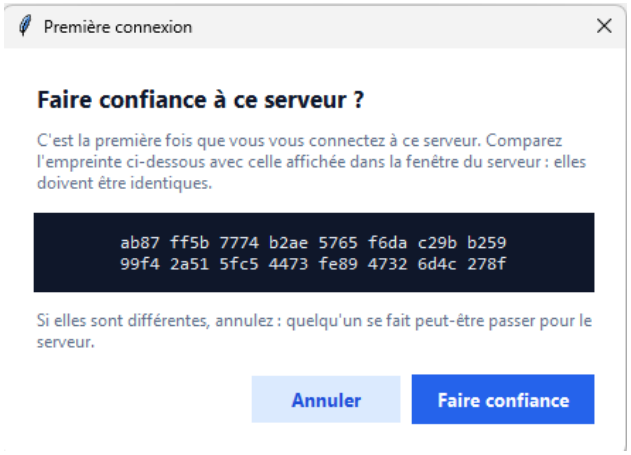


Figure 39 : Client graphique : fenêtre de confiance avec l'empreinte du serveur

La figure 40 montre la capture d'écran intitulée « Client graphique : discussion avec affichage des messages chiffrés ».

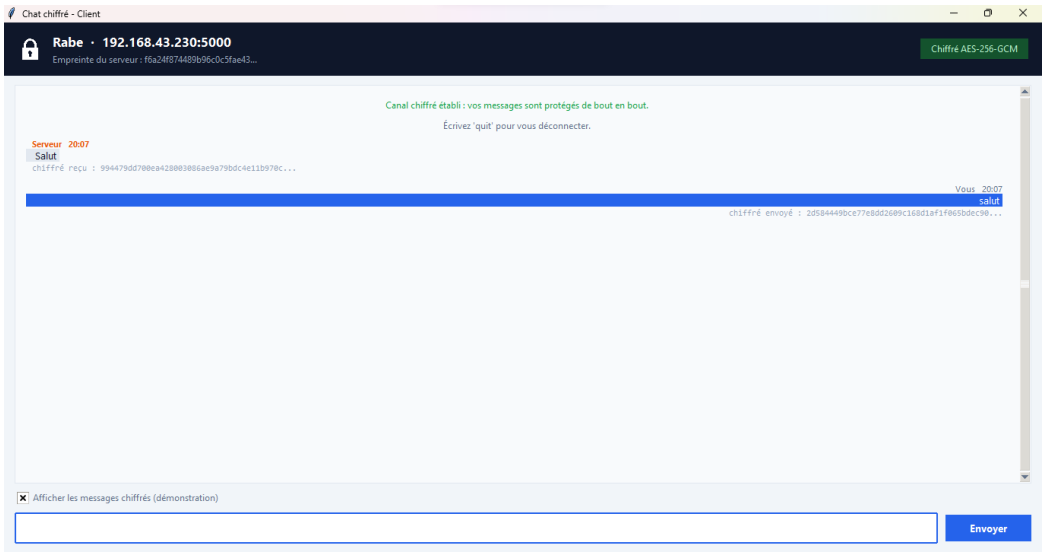


Figure 40 : Client graphique : discussion avec affichage des messages chiffrés

7.8.4. Le serveur graphique

La fenêtre du serveur comprend un **bandeau** avec l'adresse IP, le port et l'état ; une **barre d'empreinte** affichant l'empreinte SHA-256 de la clé du serveur avec un bouton « Copier » ; à gauche, la liste des **clients connectés** et le bouton « Ajouter un utilisateur » ; à droite, le journal des échanges en bulles, avec le nom et l'heure de chaque message reçu ; en bas, la zone de saisie et le bouton d'envoi.

Le serveur graphique **réutilise** `server.py` **sans le modifier** : `gui_server.py` importe le module, remplace seulement la fonction `say` (qui écrivait dans le terminal) par une fonction qui dépose les lignes dans une file, et affiche ces lignes dans la fenêtre. Les expressions régulières de `gui_server.py` reconnaissent les formes de lignes produites par `server.py` (message reçu, message envoyé, événements) pour les afficher sous forme de bulles ou de messages système colorés. Ainsi, la logique de sécurité n'est écrite qu'une seule fois.

La figure 41 montre la capture d'écran intitulée « Serveur graphique au démarrage ».

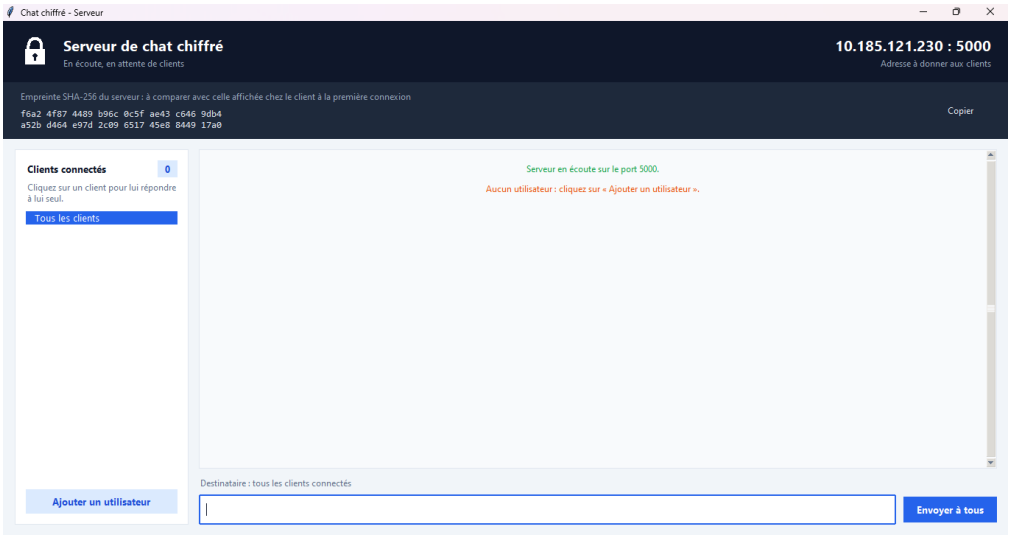


Figure 41 : Serveur graphique au démarrage

La figure 42 montre la capture d'écran intitulée « Ajout d'un utilisateur ».

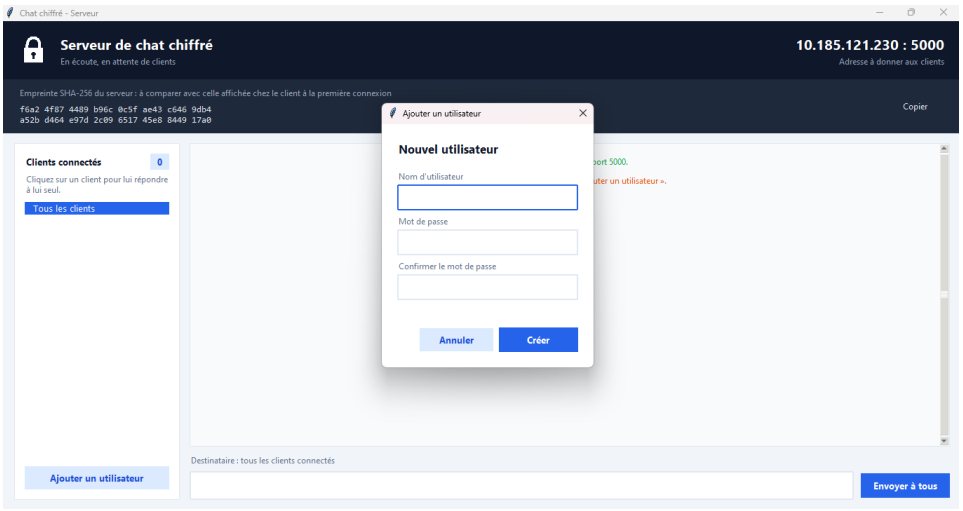


Figure 42 : Ajout d'un utilisateur

La figure 43 montre la capture d'écran intitulée « Serveur graphique avec deux clients connectés ».

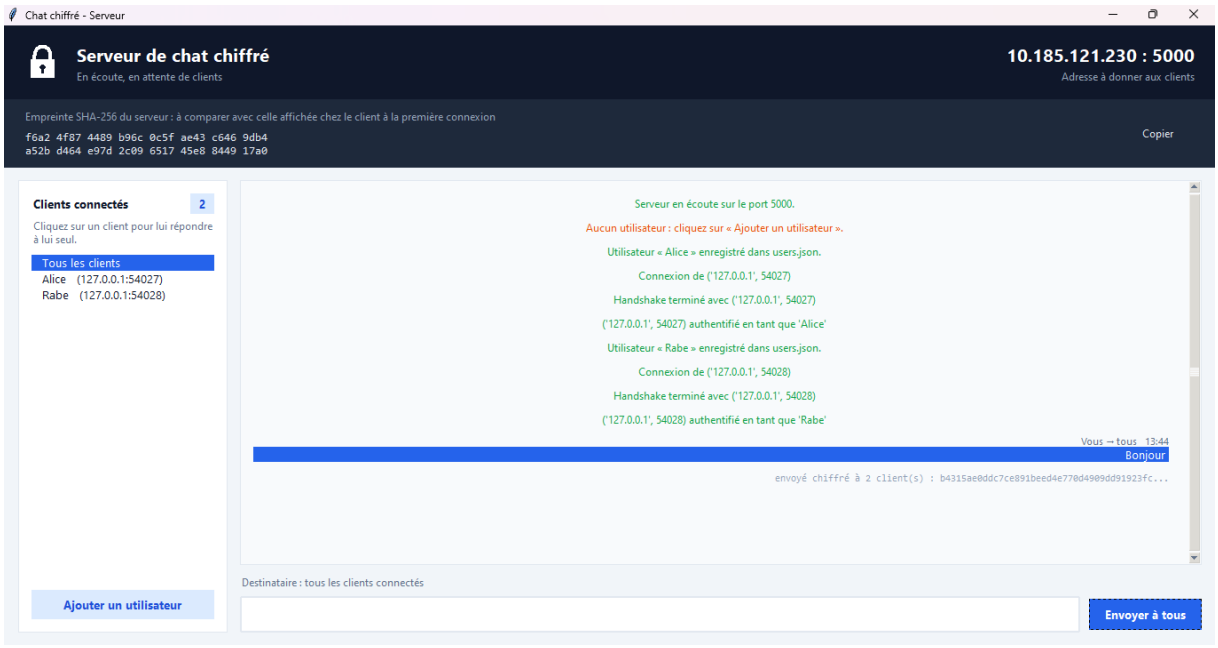


Figure 43 : Serveur graphique avec deux clients connectés

7.8.5. Répondre à tous ou à un seul client

La liste des clients commence par la ligne « **Tous les clients** », sélectionnée par défaut. Tant qu'elle est sélectionnée, le bouton affiche « Envoyer à tous » et le message part à tous les clients connectés. Si l'on clique sur un client, le bouton devient « Envoyer à Alice » (par exemple) et une phrase sous la zone de saisie rappelle « Destinataire : Alice uniquement » ; le message est alors chiffré avec la clé de session d'Alice et envoyé à elle seule. La sélection est conservée lorsque d'autres clients arrivent ou partent ; si le client sélectionné se déconnecte, l'interface revient automatiquement sur « Tous les clients ».

Techniquement, la liste conserve en parallèle une table `rows` associant chaque ligne à l'adresse du client (la première valeur, `None`, représente « tous »). Au clic sur le bouton d'envoi, `gui_server.py` appelle `server.broadcast(texte, only={adresse})` dans un thread séparé, pour ne jamais figer la fenêtre si un client est lent.

La figure 44 montre la capture d'écran intitulée « Sélection d'un client : le bouton devient « Envoyer à … » ».

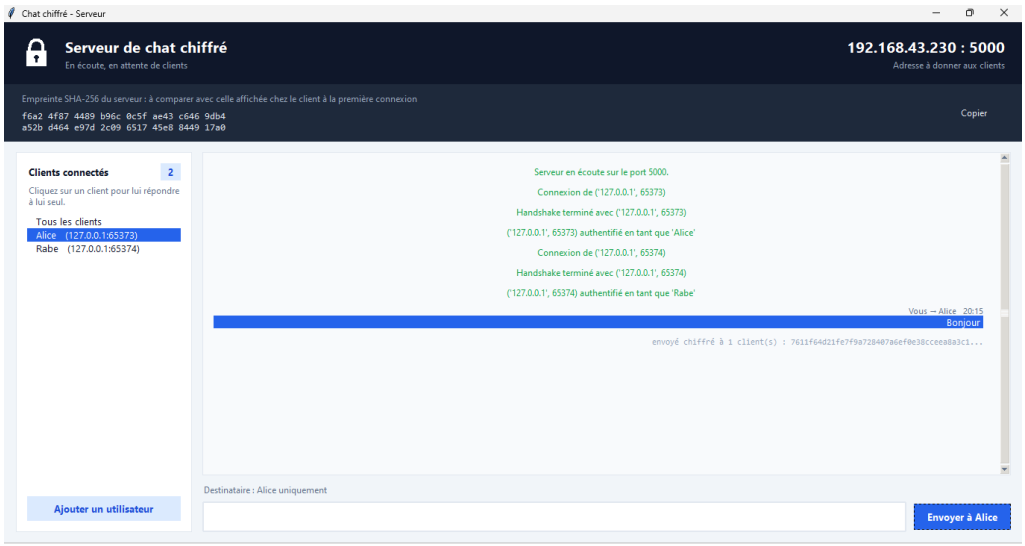


Figure 44 : Sélection d'un client : le bouton devient « Envoyer à … »

La figure 45 montre la capture d'écran intitulée « Message ciblé : reçu uniquement par le client choisi ».

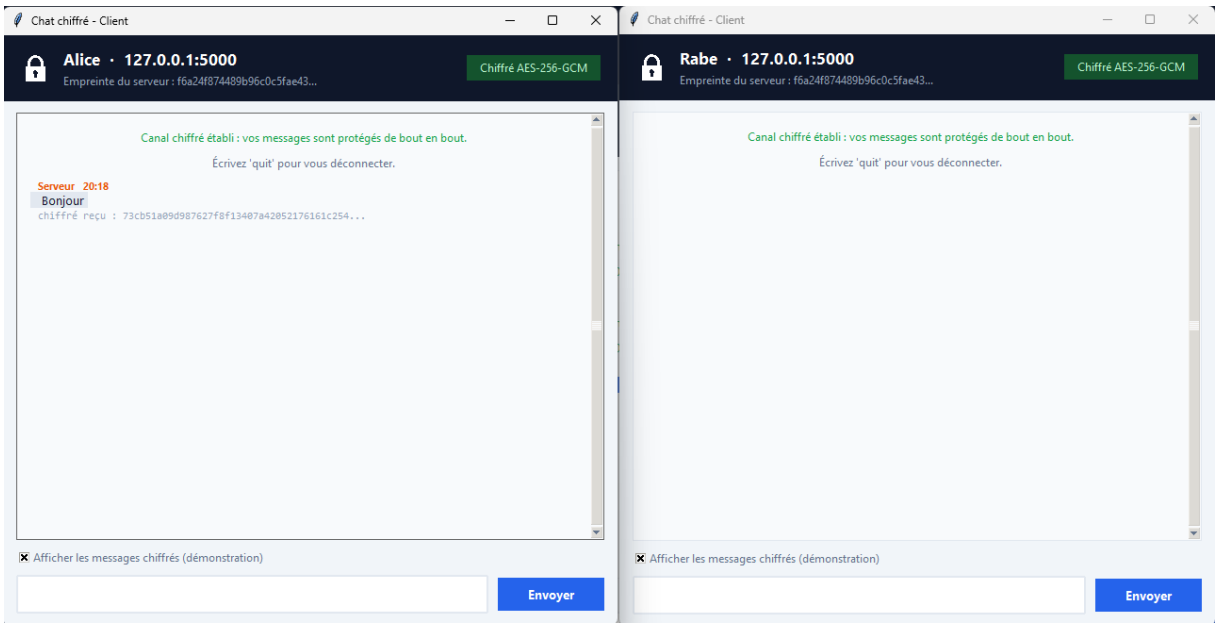


Figure 45 : Message ciblé : reçu uniquement par le client choisi

La figure 46 montre la capture d'écran intitulée « Code de la sélection du destinataire ».

```
    def selected_index(self) -> int:
        sel = self.clients_box.curselection()
        return sel[0] if sel else 0

    def render_clients(self, clients):
        """clients = liste de (addr, pseudo). Garde la sélection si le client est toujours là."""
        previous = self.rows[self.selected_index()] if self.rows else None
        self.rows = [None] + [a for a, _n in clients]
        self.row_names = [ALL_LABEL] + [n for _a, n in clients]
        self.clients_box.delete(0, "end")
        self.clients_box.insert("end", "    " + ALL_LABEL)
        for addr, name in clients:
            self.clients_box.insert("end", f"    {name}    ({addr[0]}:{addr[1]})")
        index = self.rows.index(previous) if previous in self.rows else 0
        self.clients_box.selection_clear(0, "end")
        self.clients_box.selection_set(index)
        self.clients_box.see(index)
        self.count_label.config(text=str(len(clients)))
        self.update_target()

    def refresh_clients(self):
        with server.clients_lock:
            current = sorted(((addr, user) for addr, (_c, _ch, user, _l)
                              in server.clients.items()), key=lambda x: (x[1].lower(), x[0]))
        if current != self.last_clients:
            self.last_clients = current
            self.render_clients(current)
        self.root.after(500, self.refresh_clients)

    def update_target(self):
        index = self.selected_index()
        if index == 0:
            self.send_btn.config(text="Envoyer à tous")
            self.target_label.config(text="Destinataire : tous les clients connectés")
        else:
            name = self.row_names[index]
            self.send_btn.config(text=f"Envoyer à {name}")
```

Figure 46 : Code de la sélection du destinataire

La figure 47 montre la capture d'écran intitulée « Code de la zone de discussion en bulles ».

```
def setup_style(root):
    """Applique le thème à toute la fenêtre."""
    style = ttk.Style(root)
    try:
        style.theme_use("clam")          # thème qui accepte les couleurs personnalisées
    except tk.TclError:
        pass
    root.configure(bg=BG)

    style.configure(".", font=F_BASE, background=BG, foreground=TEXT)
    style.configure("TFrame", background=BG)
    style.configure("Card.TFrame", background=CARD)
    style.configure("Header.TFrame", background=NAVY)
    style.configure("Header2.TFrame", background=NAVY_2)

    style.configure("TLabel", background=BG, foreground=TEXT, font=F_BASE)
    style.configure("Card.TLabel", background=CARD)
    style.configure("Muted.TLabel", background=BG, foreground=MUTED)
    style.configure("CardMuted.TLabel", background=CARD, foreground=MUTED, font=F_SMALL)
    style.configure("CardBold.TLabel", background=CARD, font=F_BOLD)
    style.configure("Error.TLabel", background=CARD, foreground=RED, font=F_SMALL)
    style.configure("Header.TLabel", background=NAVY, foreground="white", font=F_HEAD)
    style.configure("HeaderBig.TLabel", background=NAVY, foreground="white", font=F_BIG)
    style.configure("HeaderMuted.TLabel", background=NAVY, foreground=LIGHT, font=F_SMALL)
    style.configure("Header2.TLabel", background=NAVY_2, foreground=LIGHT, font=F_SMALL)
    style.configure("HeaderMono.TLabel", background=NAVY_2, foreground="#e2e8f0", font=F_MONO)

```

Figure 47 : Code de la zone de discussion en bulles

7.9. Déploiement sur deux machines

7.9.1. Test sur une seule machine

Avant tout test à deux machines, il est conseillé de vérifier le fonctionnement sur une seule, avec deux terminaux dans VS Code. L'adresse à utiliser est `127.0.0.1`.

1. Terminal 1 : `python gui_server.py`, puis « Ajouter un utilisateur » (par exemple `alice` avec un mot de passe).
2. Terminal 2 : `python gui_client.py`, adresse `127.0.0.1`, port `5000`, identifiants d'Alice, « Se connecter ».
3. À la première connexion, comparer l'empreinte (identique sur le serveur et le client) et accepter.
4. Envoyer un message depuis le client, vérifier qu'il apparaît sur le serveur ; répondre depuis le serveur et vérifier qu'il apparaît sur le client.

7.9.2. Version en terminal

Les versions sans interface sont utilisables de la même façon et sont **compatibles** avec les interfaces graphiques (même protocole) : on peut par exemple utiliser `server.py` avec `gui_client.py`.

```
python create_user.py          (créer un compte)
python server.py               (serveur ; affiche l'empreinte au
démarrage)
python client.py <IP du serveur> (client)
```

Dans `server.py`, on tape un message puis Entrée pour répondre à **tous** les clients, ou `@alice bonjour` pour répondre à Alice seulement.

La figure 48 montre la capture d'écran intitulée « Version terminal : serveur et client ».

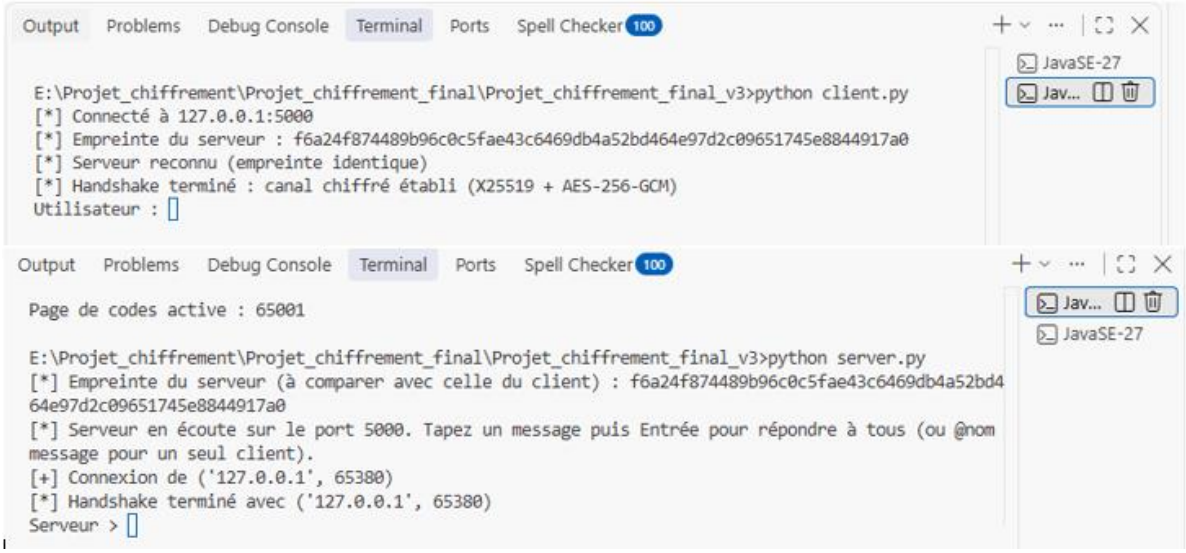


Figure 48 : Version terminal : serveur et client

7.9.3. Démonstration entre deux machines

7.9.3.1. Préparation et fichiers à transmettre

Les deux machines doivent être sur le **même réseau** (même Wi-Fi, même box, ou partage de connexion d'un téléphone). Le dossier du projet est copié sur la machine cliente par clé USB, stockage en ligne ou e-mail. Il faut copier le dossier **tel qu'il est dans l'archive d'origine**, c'est-à-dire avant d'avoir lancé le serveur dessus.

Le tableau 27 distingue les fichiers à copier sur la machine cliente de ceux qui ne doivent jamais l'être.

Tableau 27 : Fichiers à copier ou non sur la machine cliente

À copier sur la machine cliente	**À ne JAMAIS copier**
`common.py`, `client.py`, `chat_core.py`, `gui_client.py`, `ui_theme.py`, `requirements.txt` (et le reste du projet si l'on préfère copier tout le dossier d'origine)	`server_key.pem` : clé privée du serveur ; la machine cliente ne doit jamais la posséder. `users.json` : comptes et hashes des mots de passe. `known_*.txt` : empreintes mémorisées, qui fausseraient la démonstration de la première connexion. `__pycache__` : fichiers temporaires.

7.9.3.2. Étapes sur la machine A (serveur)

1. Installer Python et exécuter `pip install -r requirements.txt` dans le dossier du projet.
2. Lancer `python gui_server.py`. Sous Windows, autoriser Python dans le pare-feu (réseaux privés).
3. Relever l'**adresse IP** et l'**empreinte** affichées dans la fenêtre. Si l'adresse est `127.0.0.1`, la machine n'est pas connectée à un réseau : se connecter, fermer et relancer le serveur.
4. Cliquer sur « Ajouter un utilisateur » et créer le compte (nom, mot de passe).

La figure 49 montre la capture d'écran intitulée « Machine A : fenêtre du serveur avec son adresse IP réseau ».

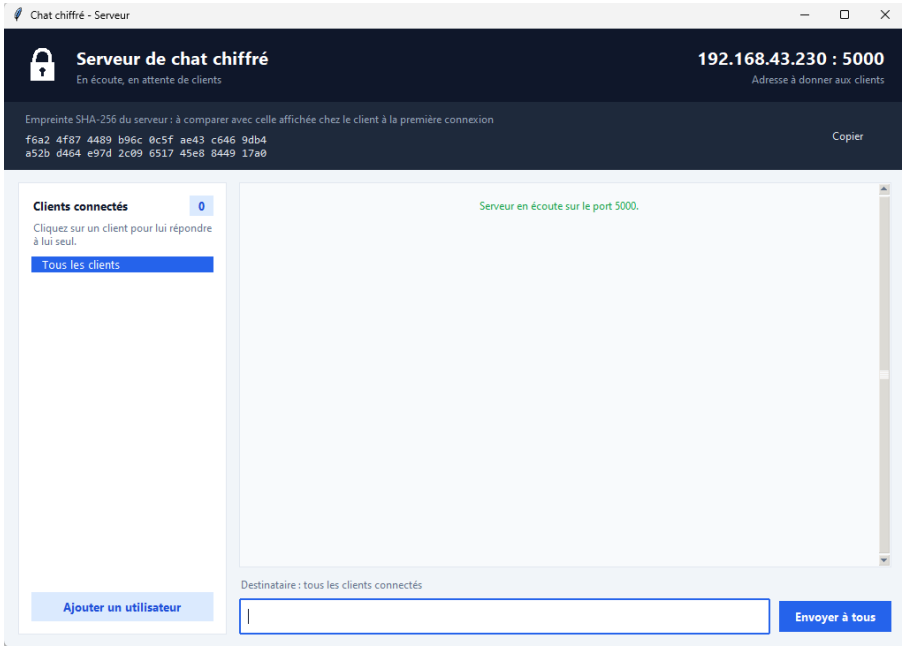


Figure 49 : Machine A : fenêtre du serveur avec son adresse IP réseau

7.9.3.3. Étapes sur la machine B (client)

1. Installer Python et exécuter `pip install -r requirements.txt`.
2. Vérifier la connectivité : `ping <IP de la machine A>` doit obtenir des réponses.
3. Lancer `python gui_client.py`, saisir l'**adresse IP de la machine A**, le port `5000`, le nom d'utilisateur et le mot de passe, puis « Se connecter ».

4. Dans la fenêtre de confiance, **comparer l'empreinte** avec celle affichée sur la machine A (lue à voix haute ou envoyée par un autre canal) ; si elles sont identiques, cliquer sur « Faire confiance ».
5. Échanger des messages dans les deux sens ; essayer l'envoi ciblé en sélectionnant le client dans la liste du serveur.

La figure 50 montre la capture d'écran intitulée « Machine B : test de connectivité avec ping ».

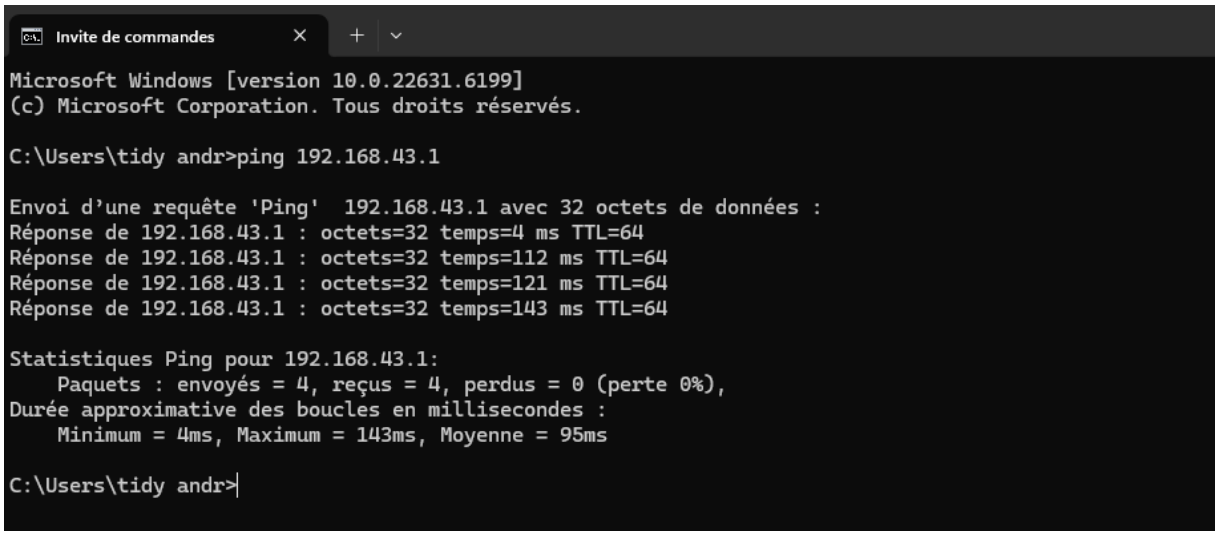


Figure 50 : Machine B : test de connectivité avec ping

La figure 51 montre la capture d'écran intitulée « Machine B : saisie de l'adresse du serveur ».

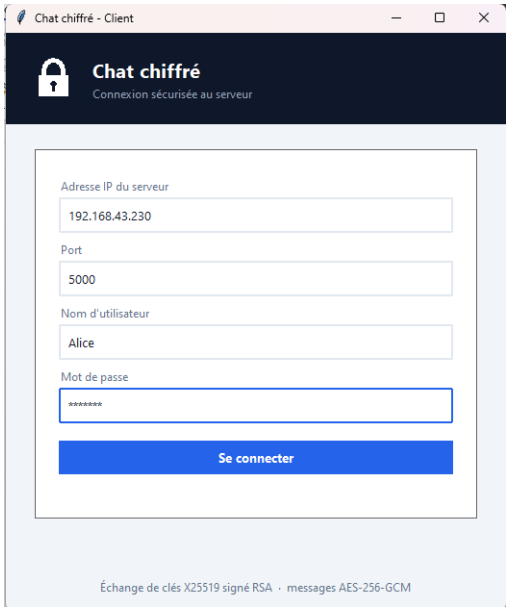


Figure 51 : Machine B : saisie de l'adresse du serveur

La figure 52 montre la capture d'écran intitulée « Comparaison des empreintes entre les deux machines ».



Figure 52 : Comparaison des empreintes entre les deux machines

La figure 53 montre la capture d'écran intitulée « Échange de messages entre les deux machines ».

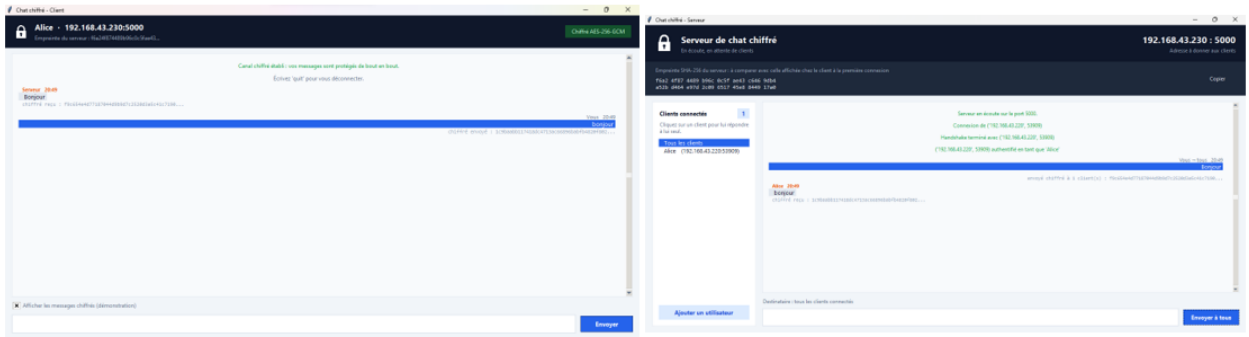


Figure 53 : Échange de messages entre les deux machines

7.9.3.4. Problèmes fréquents

Le tableau 28 recense les problèmes fréquents rencontrés lors de la démonstration entre deux machines, avec leur cause probable et leur solution.

Tableau 28 : Problèmes fréquents lors de la démonstration

Symptôme	Cause probable	Solution
« Impossible de joindre le serveur » / délai dépassé	Pare-feu, mauvaise adresse IP, machines sur des réseaux différents, serveur non lancé.	Vérifier l'IP, autoriser le port 5000, mettre les deux machines sur le même réseau, `ping`.
« Connection refused »	Le serveur n'est pas lancé ou le port est différent.	Lancer `gui_server.py` ; vérifier le port.
Le serveur affiche `127.0.0.1`	La machine n'est pas connectée à un réseau.	Se connecter au Wi-Fi, fermer et relancer le serveur.
« ALERTE : l'empreinte a changé »	Le serveur a une nouvelle clé (`server_key.pem` supprimé) ou une attaque est en cours.	Si le changement est normal, supprimer `known_<IP>_<port>.txt` côté client.
« Identifiants incorrects »	Nom ou mot de passe faux, compte inexistant.	Recréer le compte avec « Ajouter un utilisateur ».
« Connexion fermée… trop d'erreurs de mot de passe »	Six échecs en une minute depuis la même adresse.	Attendre une minute.
`ModuleNotFoundError: cryptography`	La bibliothèque n'est pas installée.	`pip install -r requirements.txt`.
`ModuleNotFoundError: tkinter`	Tkinter absent (Linux).	`sudo apt install python3-tk`.
« Port indisponible »	Un autre serveur utilise déjà le port.	Fermer l'autre serveur, ou `python gui_server.py 6000`.

Chapitre 8. Test et validation

Ce chapitre présente les tests réalisés pour vérifier le fonctionnement et la sécurité du système, puis la validation du projet : bilan, limites et perspectives.

8.1. Tests

8.1.1. Environnement de test

Les tests sont réalisés sur les deux machines décrites à la section 6.1 (machine A : serveur ; machine B : client), reliées au même réseau local, ainsi que sur une seule machine avec les adresses locales. Les tests automatiques s'exécutent avec le module `unittest` de Python.

8.1.2. Stratégie de test

La validation combine quatre approches complémentaires : des **tests unitaires automatiques** qui vérifient chaque protection isolément ; des **tests fonctionnels** qui exercent le programme comme un utilisateur ; des **attaques simulées** qui confirment que les attaques sont bien détectées ; et une **analyse du trafic avec Wireshark** qui montre ce qu'un espion voit réellement sur le réseau.

8.1.3. Tests unitaires automatiques

Le fichier `test_securite.py` contient **19 tests** qui n'ont besoin d'aucune interface graphique. Ils utilisent des paires de sockets locales et, pour le serveur, un serveur réel démarré dans un dossier temporaire. Ils se lancent avec la commande :

```
python -m unittest -v test_securite
```

Le tableau 29 énumère les 19 tests automatiques et la protection que chacun vérifie.

Tableau 29 : Tests unitaires automatiques

Groupe	Test	Vérifie que…
Handshake	`test_signature_valide`	une signature honnête est acceptée.
	`test_faux_serveur_signature_refusee`	un serveur qui signe avec une autre clé RSA est rejeté (`InvalidSignature`).

Groupe	Test	Vérifie que…
	`test_signature_liee_aux_cles_ephemeres`	remplacer une clé éphémère invalide la signature.
	`test_memes_cles_des_deux_cotes_et_forward_secrecy`	client et serveur obtiennent le même identifiant de session ; deux sessions donnent des clés différentes.
Canal	`test_aller_retour_dans_les_deux_sens`	les messages (accents compris) passent dans les deux sens.
	`test_reflexion_refusee`	un message du client renvoyé au client est rejeté.
	`test_rejeu_refuse`	un message envoyé deux fois est refusé la deuxième fois.
	`test_message_supprime_detecte`	la suppression d'un message est détectée (compteur décalé).
	`test_modification_detectee`	un bit modifié fait échouer le déchiffrement.
	`test_envois_simultanes_gardent_l_ordre`	cinq threads envoient 100 messages sans erreur de compteur.
Limites	`test_message_trop_grand_refuse`	une taille annoncée de 4 Go est refusée à la réception.
	`test_envoi_trop_grand_refuse`	l'envoi d'un message de plus de 64 Ko est refusé.
Authentification	`test_preuve_correcte_et_liee_au_defi_et_a_la_session`	la preuve change avec le défi, la session et le mot de passe.
Serveur réel	`test_bon_mot_de_passe`	une authentification correcte renvoie `OK`.
	`test_mauvais_mot_de_passe`	un mot de passe faux renvoie `DENIED`.
	`test_utilisateur_inconnu_meme_reponse_et_meme_sel_stable`	un compte inconnu reçoit la même réponse et un faux sel stable.
	`test_blocage_apres_trop_d_echecs`	6 échecs bloquent l'adresse, et le blocage expire.
	`test_message_serveur_vers_client`	un message du serveur arrive au client.

Groupe	Test	Vérifie que…
	`test_envoi_a_un_seul_client`	avec `only=`, seul le client visé reçoit le message.

La figure 54 montre la capture d'écran intitulée « Exécution des 19 tests automatiques ».

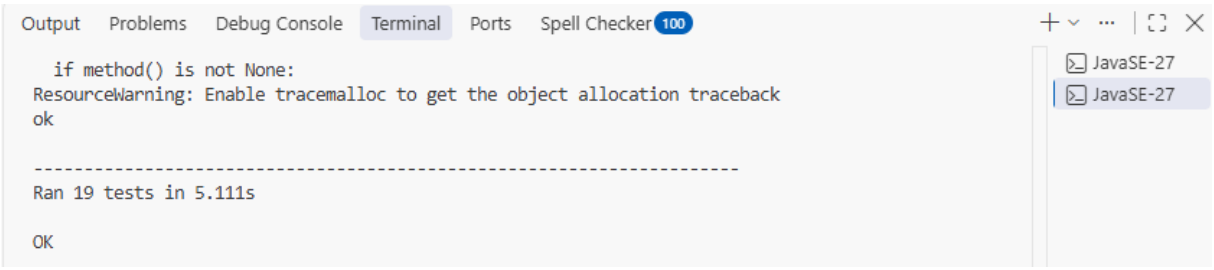


Figure 54 : Exécution des 19 tests automatiques

8.1.4. Tests fonctionnels

Le tableau 30 présente les scénarios de tests fonctionnels, le résultat attendu et le résultat observé.

Tableau 30 : Scénarios de tests fonctionnels

Scénario	Résultat attendu	Observé
Connexion avec les bons identifiants	Authentification réussie, accès à la discussion.	Authentification réussie et accès à l'interface de discussion autorisé.
Mot de passe incorrect	Message « Identifiants incorrects » ; alerte d'échec dans le journal du serveur.	Le message « Identifiants incorrects » s'affiche et l'échec de connexion est enregistré dans le journal du serveur.
Nom d'utilisateur inexistant	Même message que pour un mot de passe incorrect.	Le message « Identifiants incorrects » s'affiche, sans révéler l'existence ou non du compte.
Six échecs en moins d'une minute	L'adresse est bloquée : connexion fermée par le serveur.	Après six tentatives échouées en moins d'une minute, l'adresse IP est bloquée et la connexion est fermée par le serveur.

Scénario	Résultat attendu	Observé
Deux clients connectés en même temps	Les deux apparaissent dans la liste du serveur ; leurs messages sont distingués par leur nom.	Les deux clients sont affichés dans la liste des utilisateurs connectés et leurs messages sont correctement identifiés par leurs noms respectifs.
Envoi à tous	Les deux clients reçoivent le message.	Le message envoyé est reçu correctement par les deux clients connectés.
Envoi à un seul client sélectionné	Seul ce client reçoit le message.	Le message est transmis uniquement au client sélectionné ; l'autre client ne le reçoit pas.
Déconnexion d'un client (`quit` ou fermeture)	Le client disparaît de la liste ; s'il était sélectionné, retour sur « Tous ».	Après la déconnexion ou la fermeture du client, celui-ci disparaît automatiquement de la liste des utilisateurs et la sélection revient sur « Tous » si nécessaire.

La figure 55 montre la capture d'écran intitulée « Mot de passe incorrect : message du client et journal du serveur ».

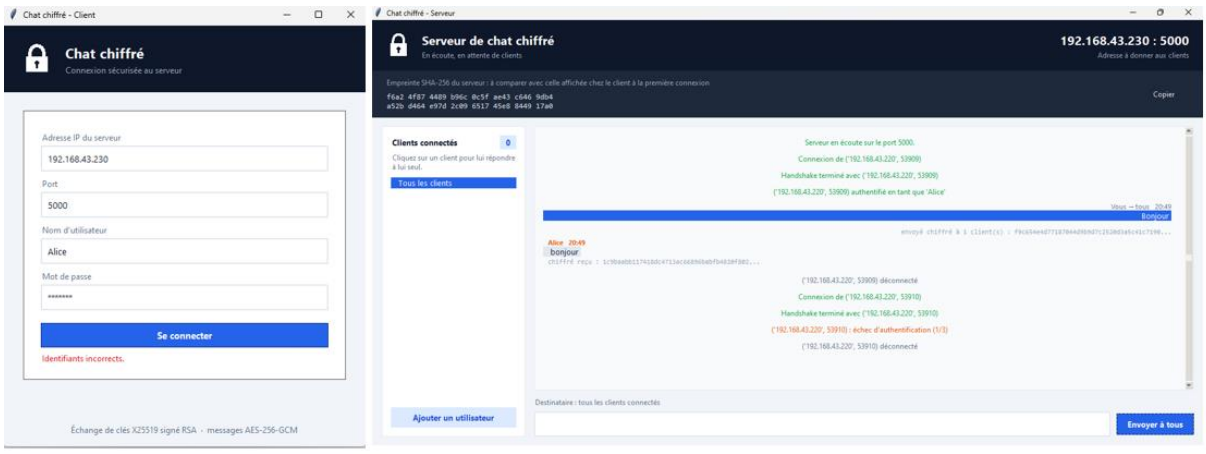


Figure 55 : Mot de passe incorrect : message du client et journal du serveur

La figure 56 montre la capture d'écran intitulée « Blocage de l'adresse après trop d'échecs ».

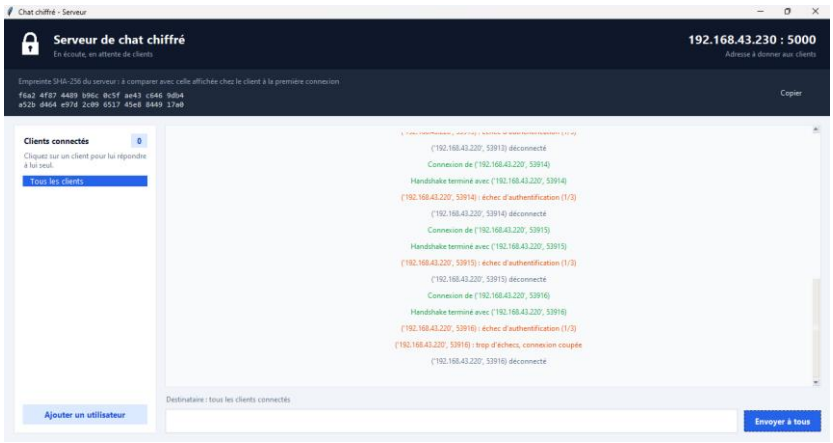


Figure 56 : Blocage de l'adresse après trop d'échecs

8.1.5. 8.1.6. Analyse du trafic avec Wireshark

Cette partie répond à la question : **que voit un espion sur le réseau ?** Deux programmes sont comparés : la version **en clair** (`serveur_clair.py` et `client_clair.py`, port 5001), conçue uniquement pour la démonstration, et le **chat chiffré** (port 5000).

8.1.6.1. Protocole de la capture (deux machines)

1. **Machine A** : lancer Wireshark, ouvrir l'interface Wi-Fi, saisir le filtre `tcp.port == 5001`, puis démarrer la capture.
2. **Machine A** : lancer `python serveur_clair.py` (autoriser le port 5001 dans le pare-feu).
3. **Machine B** : lancer `python client_clair.py <IP de A>`, saisir `alice` et `motdepasse123`, envoyer deux ou trois messages, puis `quit`.
4. **Machine A** : arrêter la capture, faire un clic droit sur un paquet de données, **Suivre > Flux TCP**.
5. Recommencer avec le chat chiffré : filtre `tcp.port == 5000`, `python gui_server.py` sur A, `python gui_client.py` sur B, quelques messages, puis **Suivre > Flux TCP**.

8.1.6.2. Version en clair

Le serveur et le client en clair envoient les octets tels quels. Le flux TCP reconstitué par Wireshark affiche, lisibles, la ligne d'authentification `alice:motdepasse123`, la réponse `OK`

et chaque message avec la réponse du serveur. N'importe quel utilisateur du même réseau qui lance Wireshark obtient donc **le mot de passe et toute la conversation**. Le filtre `frame contains "motdepasse123"` isole immédiatement le paquet fautif.

La figure 57 montre la capture d'écran intitulée « Capture Wireshark de la version en clair : liste des paquets ».

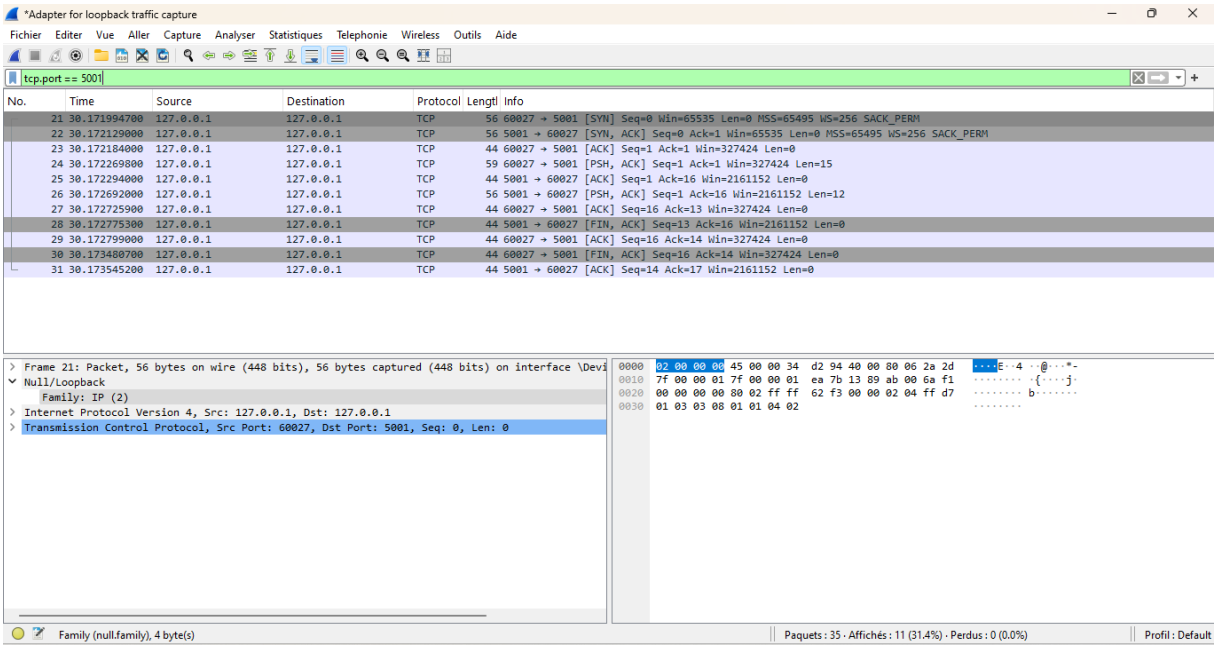


Figure 57 : Capture Wireshark de la version en clair : liste des paquets

8.1.6.3. Version chiffrée

Dans le chat chiffré, la capture montre le même déroulement TCP, mais le contenu est tout autre. Le flux commence par la **clé publique RSA du serveur** en clair (`-----BEGIN PUBLIC KEY-----`) : c'est normal, une clé publique n'est pas secrète. Viennent ensuite deux clés éphémères X25519 de 32 octets, la signature de 256 octets, puis uniquement des octets **illisibles**. Le nom d'utilisateur, le défi, la preuve et chaque message sont chiffrés ; **aucune trace du mot de passe ni des messages** n'apparaît, quel que soit le filtre utilisé. Les trames ont toutes une taille de message + 32 octets (4 octets de longueur, 12 de nonce, 16 de tag).

La figure 58 montre la capture d'écran intitulée « Capture Wireshark du chat chiffré : liste des paquets ».

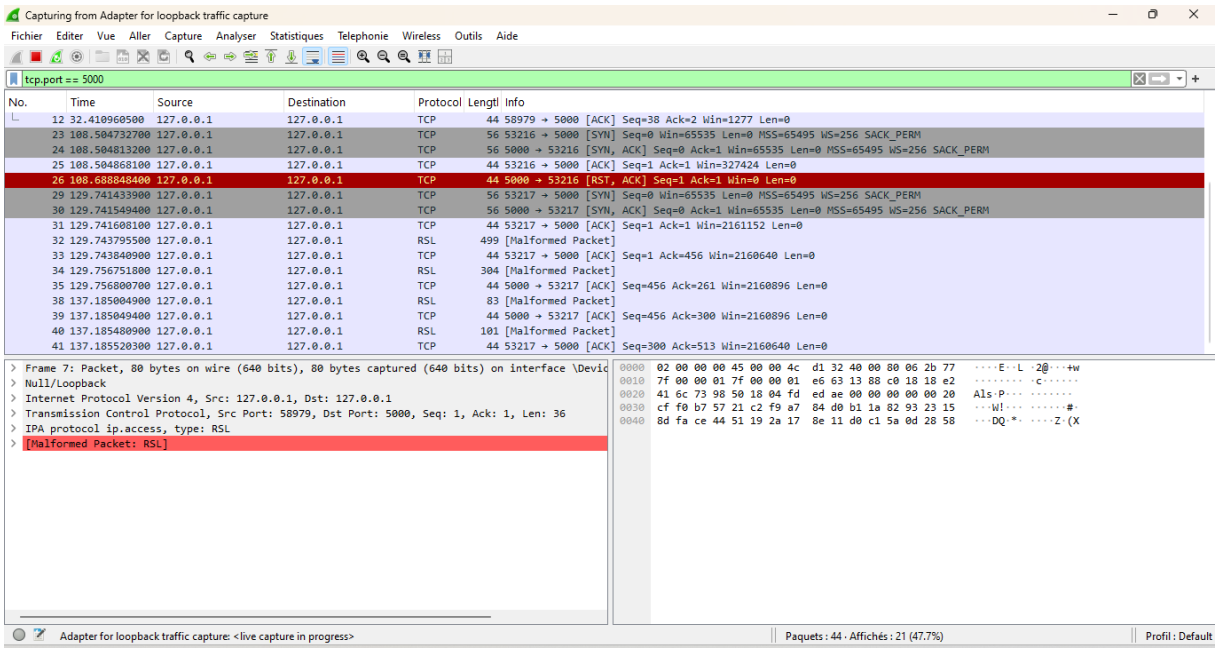


Figure 58 : Capture Wireshark du chat chiffré : liste des paquets

La figure 59 montre la capture d'écran intitulée « Flux TCP chiffré : seuls des octets illisibles ».

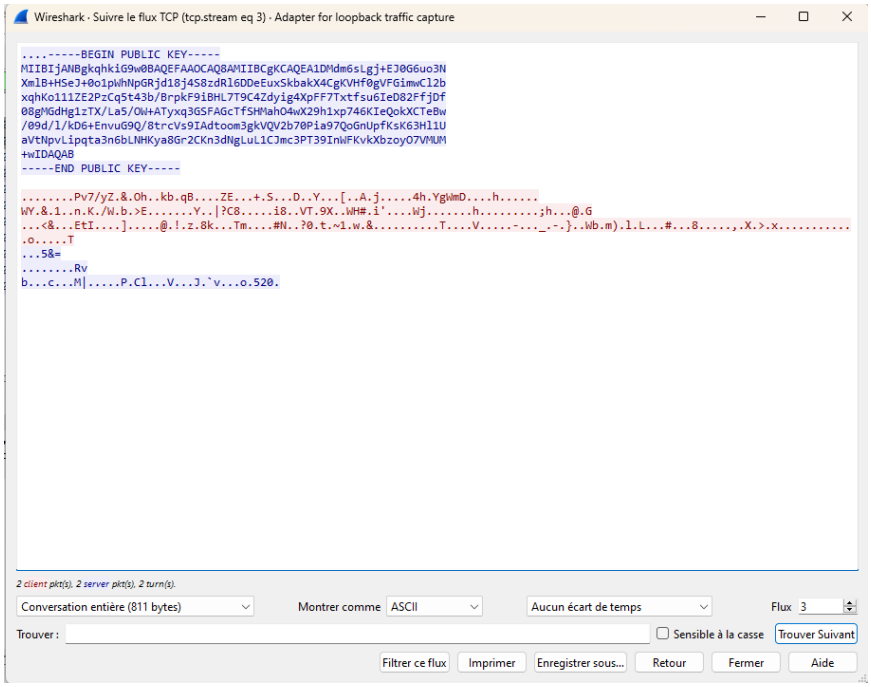


Figure 59 : Flux TCP chiffré : seuls des octets illisibles

8.1.6.4. Comparaison

Le tableau 31 compare ce qu'un espion observe sur le réseau dans la version en clair et dans le chat chiffré.

Tableau 31 : Comparaison entre la version en clair et le chat chiffré

Critère	Version en clair (port 5001)	Chat chiffré (port 5000)
Mot de passe visible sur le réseau	**Oui**, en clair	**Non**, jamais envoyé (défi/réponse)
Contenu des messages lisible	**Oui**	**Non**, AES-256-GCM
Détection d'une modification en route	Non	**Oui**, tag GCM
Détection du rejeu	Non	**Oui**, numéro de message
Authentification du serveur	Aucune	Signature RSA-PSS + empreinte
Même message envoyé deux fois	Octets identiques	Octets **différents** (nonce aléatoire)
Ce que voit l'espion	Tout	Adresses IP, ports, tailles et moments des échanges

> **Ce que Wireshark ne peut pas faire :** retrouver les messages du chat chiffré. Il n'a pas les clés, qui n'ont jamais circulé sur le réseau (seules des clés publiques éphémères ont été échangées), et les clés éphémères sont détruites en fin de session : même avec l'enregistrement complet de la conversation et la clé privée RSA du serveur, on ne pourrait pas déchiffrer cette session. C'est la confidentialité persistante.

8.1.7. Résultats obtenus

Le tableau 32 synthétise le résultat obtenu pour chaque menace et la preuve correspondante.

Tableau 32 : Résultats des tests par menace

Menace	Résultat du test	Preuve
Écoute passive	Neutralisée : contenu illisible, mot de passe absent.	Wireshark (section 8.1.6), test d'envoi chiffré.
Modification	Détectée et connexion coupée.	`test_modification_detectee`, attaque du pirate.
Rejeu	Détecté.	`test_rejeu_refuse`, `pirate.py rejeu`.

Menace	Résultat du test	Preuve
Réflexion	Détectée.	`test_reflexion_refusee`.
Usurpation du serveur	Détectée par la signature et l'empreinte.	`test_faux_serveur_signature_refusee`, alerte d'empreinte.
Force brute	Ralentie puis bloquée.	`test_blocage_apres_trop_d_echecs`.
Saturation (message géant)	Refusée.	`test_message_trop_grand_refuse`.

8.1.8. Interprétation des résultats

Les résultats confirment que chaque protection fonctionne comme prévu. Un espion placé sur le réseau n'obtient que des octets illisibles et ne voit jamais le mot de passe. Une modification d'un seul bit, un rejeu ou une réflexion de message provoquent un échec du déchiffrement et la coupure de la connexion. Un faux serveur est refusé par la vérification de la signature et de l'empreinte. Les tentatives répétées de connexion sont ralenties puis bloquées. Ces résultats sont conformes aux objectifs fixés à la section 3.2.

8.1.9. Conclusion de la phase de test

L'ensemble des tests automatiques, des attaques simulées et des analyses Wireshark valide le comportement attendu du système sur une et sur deux machines. Les limites restantes sont exposées dans la section de validation.

8.2. Validation

8.2.1. Bilan du projet

Les objectifs fixés à la section 3.2 sont atteints : serveur et client sur deux machines, chiffrement authentifié, négociation de clé sécurisée, authentification sans transmission du mot de passe, interfaces graphiques et validation par des tests.

8.2.1.1. Évolution de la version 1 à la version finale

Après la première version fonctionnelle, le code a été relu avec un œil d'attaquant. Cette revue a révélé plusieurs faiblesses, toutes corrigées dans les versions ultérieures :

Le tableau 33 présente les faiblesses relevées lors de la revue de la version 1, leur conséquence et la correction apportée.

Tableau 33 : Faiblesses de la version 1 et corrections apportées

Faiblesse de la version 1	Conséquence	Correction
Même clé AES dans les deux sens, sans lien avec la direction.	Un message du client pouvait être renvoyé au client comme venant du serveur (réflexion).	Deux clés dérivées par HKDF.
Aucun numéro de message authentifié.	Un message capturé pouvait être rejoué.	Compteur en données authentifiées GCM.
Mot de passe envoyé au serveur (dans le canal chiffré).	Un faux serveur accepté à la première connexion aurait récupéré le mot de passe.	Défi/réponse HMAC lié à la session.
Trois essais par connexion, mais reconnexion illimitée.	Force brute possible.	Pause d'1 s et blocage de l'adresse (6 échecs / 60 s).
Taille de message annoncée non limitée ; pas de délai d'attente.	Un client peut faire allouer 4 Go ou garder une connexion muette.	Taille maximale de 64 Ko ; délais d'attente.
Réponse plus rapide pour un compte inconnu ; empreinte indépendante du port.	Énumération des comptes ; confusion entre serveurs.	Faux sel stable ; fichier `known_<IP>_<port>.txt`.
RSA chiffrait la clé de session.	Pas de confidentialité persistante.	X25519 éphémère signé par RSA.

8.2.1.2. Difficultés rencontrées et solutions

Plusieurs difficultés techniques ont été rencontrées durant le développement et la démonstration. Elles sont récapitulées avec la solution retenue pour chacune.

Le tableau 34 récapitule les principales difficultés rencontrées et la solution retenue pour chacune.

Tableau 34 : Difficultés rencontrées et solutions retenues

Difficulté	Solution retenue
TCP ne conserve pas les frontières de messages : les messages arrivaient parfois collés ou coupés.	Préfixe de longueur de 4 octets et lecture exacte avec `_recv_exact`.
Ordre des compteurs avec plusieurs threads : deux envois simultanés pouvaient numéroter les messages dans un ordre différent de l'ordre d'envoi, provoquant un rejet à tort.	Verrou interne à `Channel` qui rend « chiffrer + envoyer + incrémenter » indivisible ; test à cinq threads.

Difficulté	Solution retenue
Interface graphique figée par les appels réseau bloquants, et accès à Tkinter depuis plusieurs threads.	Thread réseau + file d'événements, avec une seule règle : seul le thread principal touche aux widgets.
Temps humain et délais d'attente : à la première connexion, l'utilisateur met du temps à comparer l'empreinte alors que le serveur coupe les connexions lentes.	Handshake limité à 15 s ; vérification de l'empreinte placée après le handshake et avant le login ; 5 minutes de délai pour l'authentification.
Adresse `127.0.0.1` **affichée** sur le serveur pendant un test à deux machines.	Explication de la différence avec l'adresse réseau ; l'interface du serveur affiche maintenant un avertissement lorsqu'aucune adresse réseau n'est détectée.
Pare-feu Windows bloquant les connexions entrantes.	Autorisation de Python (réseaux privés) ou règle de trafic entrant sur les ports 5000 et 5001.
Démonstration en clair limitée à `127.0.0.1`, donc impossible à capturer entre deux machines.	`serveur_clair.py` écoute sur toutes les interfaces et `client_clair.py` accepte l'adresse du serveur.
Fichiers sensibles dans l'archive (clé privée, comptes) copiés par erreur sur la machine cliente.	Archive livrée sans `server_key.pem`, `users.json` ni `known_*.txt` ; liste explicite des fichiers à ne jamais copier.

8.2.2. Limites restantes

- **Pas de certificats** : la confiance dans le serveur repose sur la vérification de l'empreinte à la première connexion. Un utilisateur qui accepte sans comparer s'expose à une usurpation.
- `users.json` **est sensible** : le hash PBKDF2 stocké sert aussi de secret pour le défi/réponse. Quelqu'un qui volerait ce fichier pourrait s'authentifier sans connaître le mot de passe. Un protocole comme SCRAM ou OPAQUE corrigerait ce point.
- `server_key.pem` **n'est pas chiffrée** par un mot de passe : toute personne ayant accès au fichier peut se faire passer pour le serveur (sans toutefois pouvoir déchiffrer les sessions passées).
- **Un attaquant actif peut toujours couper la connexion** (déni de service) ou bloquer une adresse en multipliant les échecs depuis elle.
- **Métadonnées visibles** : adresses, ports, tailles et horaires des messages ne sont pas protégés.
- **Pas de rotation de clés** pendant une longue session et pas de journalisation persistante.

8.2.3. Pistes d'amélioration

- Remplacer la confiance à la première connexion par des **certificats X.509**, ou utiliser directement le module `ssl` de Python (TLS) pour comparer avec l'implémentation réalisée.
- Adopter un protocole d'**authentification par mot de passe sans secret équivalent** (SCRAM, OPAQUE).
- Protéger `server_key.pem` par un mot de passe ou un coffre de clés du système.
- Ajouter un **chat de groupe** : le serveur retransmet les messages d'un client aux autres, rechiffrés avec la clé de chaque destinataire.
- Ajouter le **transfert de fichiers**, l'historique chiffré, la rotation périodique des clés de session.
- Empaqueter l'application (exécutable) et fournir un **journal d'audit** des connexions.

CONCLUSION GENERALE

Ce projet a permis de réaliser un système de communication client-serveur sécurisé, de la conception du protocole à sa validation. Les trois exigences du sujet sont remplies : un serveur TCP et un client fonctionnent sur deux machines distinctes ; les messages sont protégés par du chiffrement symétrique (AES-256-GCM) ; et une négociation de clé sécurisée (X25519 signé par RSA, dérivation HKDF) établit les clés de session, avec authentification du serveur et de l'utilisateur.

Au-delà du cahier des charges, la démarche « construire, attaquer, corriger » s'est révélée instructive : la revue de la première version a mis en évidence des failles subtiles (réflexion, rejeu, mot de passe transmis) qui ne se voient pas à l'usage normal mais qu'un attaquant actif exploiterait. Les tests automatiques, les attaques simulées et l'analyse Wireshark apportent des preuves concrètes que les protections fonctionnent : un espion ne voit que des octets illisibles, une modification ou un rejeu est détecté, et un faux serveur est refusé.

Sur le plan des compétences, le projet a mobilisé la cryptographie appliquée (choix et composition d'algorithmes standards plutôt que création de primitives), la programmation réseau (sockets, délimitation des messages, concurrence) et l'analyse de sécurité (modèle de menaces, contre-mesures, limites). Il a aussi montré qu'un système sûr dépend autant de l'ingénierie (délais d'attente, limites de taille, gestion des fichiers sensibles, ergonomie des vérifications d'empreinte) que des algorithmes eux-mêmes.

Les limites identifiées (absence de certificats, fichier de comptes sensible, clé du serveur non protégée) constituent autant de pistes pour la suite, jusqu'à comparer le protocole réalisé avec TLS réel. Ce travail a donné une compréhension concrète de ce qui se passe, en petit, derrière chaque connexion HTTPS.

REFERENCES BIBLIOGRAPHIQUES

[1] N. Ferguson, B. Schneier, T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*, 2010, 1re édition (Wiley Publishing, Indianapolis), 384 pages.

[2] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 2020, 3e édition (Wiley, Indianapolis), [nombre de pages à vérifier].

REFERENCES WEBOGRAPHIQUES

[3] https://docs.python.org/3/library/socket.html, Python 3 : module socket, consulté le [date de consultation]

[4] https://docs.python.org/3/library/threading.html, Python 3 : module threading, consulté le [date de consultation]

[5] https://docs.python.org/3/library/tkinter.html, Python 3 : module tkinter, consulté le [date de consultation]

[6] https://docs.python.org/3/library/unittest.html, Python 3 : module unittest, consulté le [date de consultation]

[7] https://cryptography.io/en/latest/, Documentation de la bibliothèque cryptography, consulté le [date de consultation]

[8] https://csrc.nist.gov/pubs/sp/800/38/d/final, NIST SP 800-38D : mode GCM, consulté le [date de consultation]

[9] https://www.rfc-editor.org/rfc/rfc7748, RFC 7748 : Elliptic Curves for Security (X25519), consulté le [date de consultation]

[10] https://www.rfc-editor.org/rfc/rfc8017, RFC 8017 : PKCS #1, RSA (OAEP, PSS), consulté le [date de consultation]

[11] https://www.rfc-editor.org/rfc/rfc5869, RFC 5869 : HKDF, consulté le [date de consultation]

[12] https://www.rfc-editor.org/rfc/rfc2104, RFC 2104 : HMAC, consulté le [date de consultation]

[13] https://www.rfc-editor.org/rfc/rfc8018, RFC 8018 : PKCS #5, PBKDF2, consulté le [date de consultation]

[14] https://www.rfc-editor.org/rfc/rfc8446, RFC 8446 : TLS 1.3, consulté le [date de consultation]

[15] https://www.rfc-editor.org/rfc/rfc9293, RFC 9293 : Transmission Control Protocol, consulté le [date de consultation]

[16] https://www.wireshark.org/docs/wsug_html_chunked/, Wireshark User's Guide, consulté le [date de consultation]

[17] https://code.visualstudio.com/docs, Documentation de Visual Studio Code, consulté le [date de consultation]

GLOSSAIRE

AEAD : Chiffrement authentifié avec données associées : garantit à la fois la confidentialité et l'intégrité du message.

AES : Algorithme de chiffrement symétrique par blocs de 128 bits, avec des clés de 128, 192 ou 256 bits.

Attaque de l'homme du milieu : Attaque dans laquelle une personne se place entre deux interlocuteurs pour lire ou modifier leurs échanges.

Authentification : Vérification de l'identité d'un utilisateur ou d'un serveur.

Canal chiffré : Connexion dont les messages sont chiffrés avec des clés de session.

Chiffrement asymétrique : Chiffrement utilisant une clé publique et une clé privée distinctes.

Chiffrement symétrique : Chiffrement utilisant la même clé pour chiffrer et déchiffrer.

Confidentialité persistante : Propriété qui garantit que la fuite d'une clé à long terme ne permet pas de déchiffrer les sessions passées.

Défi/réponse : Méthode d'authentification dans laquelle le serveur envoie une valeur aléatoire et le client prouve sa connaissance du secret sans le transmettre.

Diffie-Hellman : Méthode permettant à deux parties de calculer un secret commun en n'échangeant que des valeurs publiques.

Empreinte : Valeur obtenue par une fonction de hachage, utilisée ici pour identifier la clé publique d'un serveur.

Handshake : Phase initiale de la connexion pendant laquelle les clés de session sont négociées.

HKDF : Fonction de dérivation de plusieurs clés à partir d'un secret partagé.

HMAC : Code d'authentification de message calculé avec une clé secrète et une fonction de hachage.

Intégrité : Garantie qu'un message n'a pas été modifié pendant son transport.

Nonce : Valeur unique utilisée une seule fois avec une clé donnée lors d'un chiffrement.

PBKDF2 : Fonction qui dérive une clé à partir d'un mot de passe en répétant un calcul de nombreuses fois.

Rejeu : Attaque consistant à renvoyer un message valide capturé précédemment.

RSA : Algorithme de cryptographie asymétrique utilisé pour le chiffrement et la signature.

Sel : Valeur aléatoire ajoutée à un mot de passe avant le calcul de son empreinte.

Socket : Point d'accès d'un programme au réseau, permettant d'envoyer et de recevoir des données.

TOFU : Principe de confiance à la première connexion : l'empreinte du serveur est mémorisée puis comparée lors des connexions ultérieures.

X25519 : Échange de clés Diffie-Hellman sur la courbe elliptique Curve25519.

ANNEXES

Les annexes regroupent le guide de démonstration, le code source des fichiers principaux et la feuille de route des captures d'écran.

Annexe A : Guide de démonstration express entre deux machines

1. **Avant la démonstration** : les deux machines sont connectées au même réseau ; Python et la bibliothèque `cryptography` sont installés ; le dossier du projet d'origine est présent sur chaque machine (sans `server_key.pem` ni `users.json` sur la machine cliente).
2. **Machine A** : lancer `python gui_server.py`, autoriser le pare-feu, relever l'adresse IP et l'empreinte, puis ajouter un utilisateur.
3. **Machine A** : ouvrir Wireshark, choisir l'interface Wi-Fi, saisir le filtre `tcp.port == 5000` et démarrer la capture.
4. **Machine B** : lancer `python gui_client.py` et saisir l'adresse de la machine A, le port 5000, le nom d'utilisateur et le mot de passe.
5. **Comparer les empreintes** affichées sur les deux écrans, puis faire confiance au serveur.
6. **Échanger des messages** dans les deux sens, avec la case « Afficher les messages chiffrés » cochée.
7. **Envoi ciblé** : connecter un deuxième client, sélectionner un client dans la liste du serveur et montrer que seul ce client reçoit le message.
8. **Wireshark** : arrêter la capture, puis Suivre > Flux TCP : le contenu est illisible et le mot de passe n'apparaît pas.
9. **Comparaison** : montrer la capture de la version en clair (`serveur_clair.py` et `client_clair.py <IP>`), où le mot de passe est lisible.
10. **Attaques** : lancer `pirate.py` (modification, puis `rejeu`) et montrer l'alerte du serveur ; montrer l'alerte d'empreinte modifiée.

TABLE DES MATIERES

CURRICULUM VITAE 2
CURRICULUM VITAE 4
CURRICULUM VITAE I
CURRICULUM VITAE III
SOMMAIRE V
REMERCIEMENTS VIII
LISTE DES FIGURES IX
LISTE DES TABLEAUX XII
LISTE DES ABREVIATIONS XIV
INTRODUCTION GENERALE 1
PARTIE I : PRESENTATIONS 2
Chapitre 1. Présentation de l'Ecole Nationale d'Informatique 3
1.1. Information d'ordre générale 3
1.2. Missions et historiques 3
1.3. Organigramme institutionnel 5
1.4. Domaine de spécialisation 6
1.5. Architecture des formations pédagogiques 7
1.6. Relation de l'ENI avec les organismes externes 8
1.7. Débouchés professionnels et diplômés 9
1.8. Ressources humaines 11
Chapitre 2. Description du projet 12
3.1. Formulation 12
3.2. Objectif et besoin de l'utilisateur 12
3.2.1. Cahier des charges et objectifs complémentaires 13
3.3. Moyens nécessaires à la réalisation du projet 13

3.3.1. Moyens humains 13
3.3.2. Moyens matériels 14
3.3.3. Moyens logiciels 14
3.4. Résultats attendus 15
3.5. Chronogramme de travail 15
PARTIE II : ANALYSE PREALABLE 17
Chapitre 4. Analyse de l'existant 18
4.1. Analyse de l'existant 18
4.1.1. Topologie réseau 18
4.1.2. Types de réseau et connexion Internet 18
4.1.3. Équipements d'interconnexion 19
4.1.4. Serveurs 19
4.1.5. Postes de travail 19
4.1.6. Passerelle 20
4.1.7. Partage des données 20
4.1.8. Routage 20
4.1.9. Surveillance 20
4.1.10. Disponibilité 20
4.1.11. Protection des échanges existante 20
4.2. Critique de l'existant 21
4.2.1. Points faibles 21
4.2.2. Points forts 21
4.2.3. Analyse des menaces 22
4.2.3.1. Modèle de l'attaquant 22
4.2.3.2. Menaces considérées et contre-mesures 22
4.2.3.3. Ce qui est hors périmètre 23
4.3. Proposition de solution 24

4.3.1. Solutions envisagées 24
4.3.2. Solution retenue 24
Chapitre 5. Études et choix des outils 25
5.1. Étude des notions de sécurité et de cryptographie 25
5.1.1. Les objectifs de la sécurité des communications 25
5.1.2. Communication par sockets TCP 26
5.1.3. Chiffrement symétrique : AES et le mode GCM 26
5.1.4. Chiffrement asymétrique : RSA 27
5.1.5. Échange de clés Diffie-Hellman et X25519 28
5.1.6. Hachage, HMAC et dérivation de clés 28
5.1.7. Authentification : mot de passe, défi/réponse et empreinte de clé 29
5.1.8. Récapitulatif des algorithmes utilisés 29
5.2. Comparaison et choix des outils 30
5.2.1. Algorithmes cryptographiques 30
5.2.2. Langage de programmation et bibliothèques 31
5.2.3. Éditeur de code 31
5.2.4. Interface graphique 32
5.2.5. Analyseur de trafic 32
5.3. Présentation des outils retenus 32
5.3.1. Python 3 32
5.3.2. La bibliothèque cryptography et pip 33
5.3.3. Tkinter 33
5.3.4. Visual Studio Code 33
5.3.5. Wireshark et Npcap 33
5.3.6. Outils réseau du système 33
PARTIE III : REALISATION 34
Chapitre 6. Présentation de l'environnement 35

6.1. Environnement matériel 35
6.2. Environnement réseau 35
6.3. Environnement logiciel 36
6.4. Architecture de la solution 36
6.4.1. Architecture générale 36
6.4.2. Structure du projet 37
Chapitre 7. Installation et configuration 39
7.1. Installation de Python 39
7.1.1. Installation sous Windows 39
7.1.2. Vérification 40
7.2. Installation et utilisation de Visual Studio Code 41
7.2.1. Installation 41
7.2.2. Utilisation de Visual Studio Code pour le projet 42
7.3. Installation de la bibliothèque cryptography et de Tkinter 43
7.4. Installation et utilisation de Wireshark 44
7.4.1. Installation 44
7.4.2. Utilisation pour le projet 45
7.5. Outils réseau du système 46
7.5.1. Trouver l'adresse IP 46
7.5.2. Tester la connectivité avec ping 47
7.5.3. Pare-feu Windows 47
7.6. Conception du protocole de communication 48
7.6.1. Le protocole de communication 48
7.6.1.1. Phase 1 : handshake 49
7.6.1.2. Phase 2 : authentification 50
7.6.1.3. Phase 3 : échange de messages 51
7.6.2. Format des messages sur le réseau 51

7.6.3. Comptes, empreintes et fichiers 52
7.6.4. Gestion de la concurrence 52
7.6.5. Choix de conception justifiés 53
7.7. Développement du code 53
7.7.1. Le module common.py 53
7.7.1.1. Envoi et réception de messages délimités 54
7.7.1.2. Chiffrement AES-GCM 55
7.7.1.3. Handshake : X25519 éphémère et signature RSA-PSS 56
7.7.1.4. Le canal chiffré : la classe Channel 57
7.7.1.5. Mots de passe et preuve d'authentification 58
7.7.2. Le serveur : server.py 59
7.7.2.1. Paramètres de sécurité et clé du serveur 59
7.7.2.2. Limitation des tentatives et comptes 60
7.7.2.3. Authentification défi/réponse 61
7.7.2.4. Gestion d'un client 62
7.7.2.5. Envoi à tous ou à un seul client 63
7.7.3. Le client : client.py 64
7.7.4. Le module chat_core.py 66
7.8. Interfaces graphiques 66
7.8.1. Objectifs et choix 66
7.8.2. Principe : un thread réseau et une file d'événements 67
7.8.3. Le client graphique 68
7.8.4. Le serveur graphique 70
7.8.5. Répondre à tous ou à un seul client 72
7.9. Déploiement sur deux machines 74
7.9.1. Test sur une seule machine 75
7.9.2. Version en terminal 75

7.9.3. Démonstration entre deux machines 76
7.9.3.1. Préparation et fichiers à transmettre 76
7.9.3.2. Étapes sur la machine A (serveur) 77
7.9.3.3. Étapes sur la machine B (client) 77
7.9.3.4. Problèmes fréquents 80
Chapitre 8. Test et validation 81
8.1. Tests 81
8.1.1. Environnement de test 81
8.1.2. Stratégie de test 81
8.1.3. Tests unitaires automatiques 81
8.1.4. Tests fonctionnels 83
8.1.5. 8.1.6. Analyse du trafic avec Wireshark 85
8.1.6.1. Protocole de la capture (deux machines) 85
8.1.6.2. Version en clair 85
8.1.6.3. Version chiffrée 86
8.1.6.4. Comparaison 88
8.1.7. Résultats obtenus 88
8.1.8. Interprétation des résultats 89
8.1.9. Conclusion de la phase de test 89
8.2. Validation 89
8.2.1. Bilan du projet 89
8.2.1.1. Évolution de la version 1 à la version finale 89
8.2.1.2. Difficultés rencontrées et solutions 90
8.2.2. Limites restantes 91
8.2.3. Pistes d'amélioration 92
CONCLUSION GENERALE 93
REFERENCES BIBLIOGRAPHIQUES IX

REFERENCES WEBOGRAPHIQUES X
GLOSSAIRE XI
ANNEXES XIII
Annexe A : Guide de démonstration express entre deux machines XIV
TABLE DES MATIERES XV
RESUME XXII
ABSTRACT XXIII

RESUME

Ce mémoire présente la conception, la réalisation et la validation d'un système de communication sécurisée entre deux machines, réalisé dans le cadre du projet de fin d'année (sujet 8 : chiffrement des communications client-serveur). Le projet consiste en un serveur TCP multi-clients et un client, écrits en Python, qui échangent des messages à travers une couche de chiffrement négociée par un handshake sécurisé.

Le protocole repose sur un échange de clés Diffie-Hellman sur courbe elliptique (X25519) dont les clés éphémères sont signées par la clé RSA-2048 du serveur, ce qui garantit l'authenticité du serveur et la confidentialité persistante. Les clés de session sont dérivées avec HKDF-SHA-256. Les messages sont chiffrés et authentifiés avec AES-256-GCM, avec une clé par sens de communication et un numéro de message qui interdit le rejeu. L'utilisateur s'authentifie par un défi/réponse HMAC : son mot de passe ne circule jamais sur le réseau et n'est stocké que sous forme dérivée (PBKDF2-HMAC-SHA-256 salé).

Deux interfaces graphiques Tkinter permettent de piloter le client et le serveur, ce dernier pouvant répondre à tous les clients ou à un seul. La validation repose sur 19 tests automatiques, des attaques simulées (modification, rejeu, faux serveur) et l'analyse du trafic avec Wireshark, qui montre qu'un espion lit le mot de passe et les messages dans la version non protégée, mais uniquement des octets illisibles dans la version chiffrée.

Mots-clés : AES-GCM, Authentification, Cryptographie, Handshake, RSA, Sockets TCP, Wireshark, X25519.

ABSTRACT

This thesis describes the design, implementation and validation of a secure communication system between two machines (project topic 8: encryption of client-server communications). The system is a multi-client TCP server and a client written in Python. Keys are negotiated with an X25519 key exchange signed by the server's RSA-2048 key, which provides server authentication and forward secrecy; session keys are derived with HKDF-SHA-256; messages are protected with AES-256-GCM using one key per direction and an authenticated message counter against replay. Users authenticate with an HMAC challenge-response, so passwords are never sent over the network. Two Tkinter graphical interfaces are provided, and the server can answer all clients or a single selected one. The system is validated by 19 automated tests, simulated attacks (tampering, replay, impersonation) and a Wireshark traffic analysis comparing an unprotected version with the encrypted one.

Keywords: AES-GCM, Authentication, Cryptography, Handshake, RSA, TCP sockets, Wireshark, X25519.